

Fundamentals of Image Processing

Ian T. Young
Jan J. Gerbrands
Lucas J. van Vliet
Delft University of Technology

1. Introduction	1
2. Digital Image Definitions	2
3. Tools	6
4. Perception	22
5. Image Sampling	28
6. Noise	32
7. Cameras	35
8. Displays	44
9. Algorithms	44
10. Techniques	86
11. Acknowledgments	109
12. References	109

1. Introduction

Modern digital technology has made it possible to manipulate multi-dimensional signals with systems that range from simple digital circuits to advanced parallel computers. The goal of this manipulation can be divided into three categories:

- Image Processing *image in → image out*
- Image Analysis *image in → measurements out*
- Image Understanding *image in → high-level description out*

We will focus on the fundamental concepts of *image processing*. Space does not permit us to make more than a few introductory remarks about *image analysis*. *Image understanding* requires an approach that differs fundamentally from the theme of this book. Further, we will restrict ourselves to two-dimensional (2D) image processing although most of the concepts and techniques that are to be described can be extended easily to three or more dimensions. Readers interested in either greater detail than presented here or in other aspects of image processing are referred to [1-10]

We begin with certain basic definitions. An image defined in the “real world” is considered to be a function of two real variables, for example, $a(x,y)$ with a as the amplitude (e.g. brightness) of the image at the *real* coordinate position (x,y) . An image may be considered to contain sub-images sometimes referred to as *regions-of-interest*, *ROIs*, or simply *regions*. This concept reflects the fact that images frequently contain collections of objects each of which can be the basis for a region. In a sophisticated image processing system it should be possible to apply specific image processing operations to selected regions. Thus one part of an image (region) might be processed to suppress motion blur while another part might be processed to improve color rendition.

The amplitudes of a given image will almost always be either real numbers or integer numbers. The latter is usually a result of a quantization process that converts a continuous range (say, between 0 and 100%) to a discrete number of levels. In certain image-forming processes, however, the signal may involve photon counting which implies that the amplitude would be inherently quantized. In other image forming procedures, such as magnetic resonance imaging, the direct physical measurement yields a complex number in the form of a real magnitude and a real phase. For the remainder of this book we will consider amplitudes as reals or integers unless otherwise indicated.

2. Digital Image Definitions

A digital image $a[m,n]$ described in a 2D discrete space is derived from an analog image $a(x,y)$ in a 2D continuous space through a *sampling* process that is frequently referred to as digitization. The mathematics of that sampling process will be described in Section 5. For now we will look at some basic definitions associated with the digital image. The effect of digitization is shown in Figure 1.

The 2D continuous image $a(x,y)$ is divided into N rows and M columns. The intersection of a row and a column is termed a *pixel*. The value assigned to the integer coordinates $[m,n]$ with $\{m=0,1,2,\dots,M-1\}$ and $\{n=0,1,2,\dots,N-1\}$ is $a[m,n]$. In fact, in most cases $a(x,y)$ – which we might consider to be the physical signal that impinges on the face of a 2D sensor – is actually a function of many variables including depth (z), color (λ), and time (t). Unless otherwise stated, we will consider the case of 2D, monochromatic, static images in this chapter.

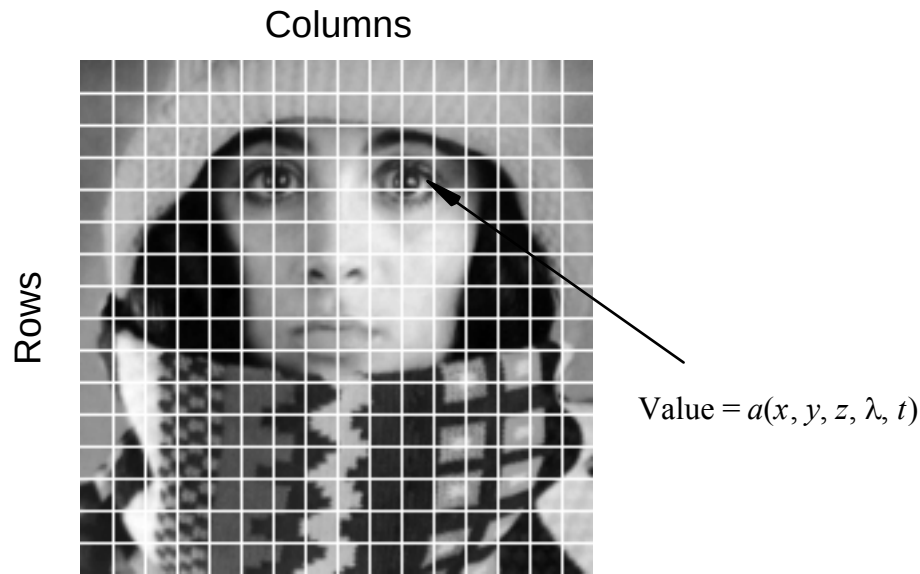


Figure 1: Digitization of a continuous image. The pixel at coordinates $[m=10, n=3]$ has the integer brightness value 110.

The image shown in Figure 1 has been divided into $N = 16$ rows and $M = 16$ columns. The value assigned to every pixel is the average brightness in the pixel rounded to the nearest integer value. The process of representing the amplitude of the 2D signal at a given coordinate as an integer value with L different gray levels is usually referred to as amplitude quantization or simply *quantization*.

2.1 COMMON VALUES

There are standard values for the various parameters encountered in digital image processing. These values can be caused by video standards, by algorithmic requirements, or by the desire to keep digital circuitry simple. Table 1 gives some commonly encountered values.

<i>Parameter</i>	<i>Symbol</i>	<i>Typical values</i>
Rows	N	256,512,525,625,1024,1080
Columns	M	256,512,768,1024,1920
Gray Levels	L	2,64,256,1024,4096,16384

Table 1: Common values of digital image parameters

Quite frequently we see cases of $M=N=2^K$ where $\{K = 8,9,10,11,12\}$. This can be motivated by digital circuitry or by the use of certain algorithms such as the (fast) Fourier transform (see Section 3.3).

The number of distinct gray levels is usually a power of 2, that is, $L=2^B$ where B is the number of bits in the binary representation of the brightness levels. When $B>1$ we speak of a *gray-level image*; when $B=1$ we speak of a *binary image*. In a binary image there are just two gray levels which can be referred to, for example, as “black” and “white” or “0” and “1”.

2.2 CHARACTERISTICS OF IMAGE OPERATIONS

There is a variety of ways to classify and characterize image operations. The reason for doing so is to understand what type of results we might expect to achieve with a given type of operation or what might be the computational burden associated with a given operation.

2.2.1 Types of operations

The types of operations that can be applied to digital images to transform an input image $a[m,n]$ into an output image $b[m,n]$ (or another representation) can be classified into three categories as shown in Table 2.

Operation	Characterization	Generic Complexity/Pixel
• <i>Point</i>	– the output value at a specific coordinate is dependent only on the input value at that same coordinate.	<i>constant</i>
• <i>Local</i>	– the output value at a specific coordinate is dependent on the input values in the <i>neighborhood</i> of that same coordinate.	P^2
• <i>Global</i>	– the output value at a specific coordinate is dependent on all the values in the input image.	N^2

Table 2: Types of image operations. Image size = $N \times N$; neighborhood size = $P \times P$. Note that the complexity is specified in operations *per pixel*.

This is shown graphically in Figure 2.

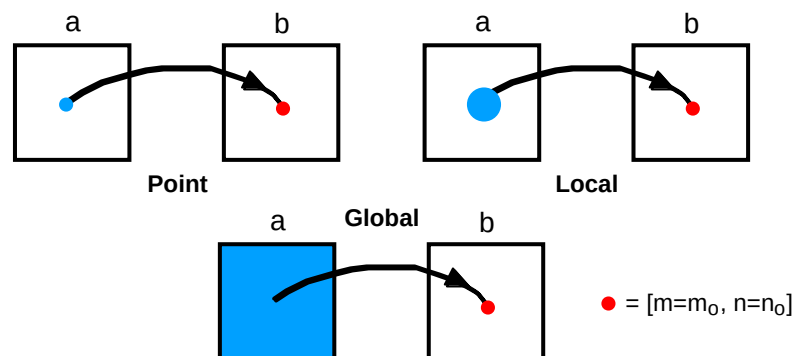


Figure 2: Illustration of various types of image operations

2.2.2 Types of neighborhoods

Neighborhood operations play a key role in modern digital image processing. It is therefore important to understand how images can be sampled and how that relates to the various neighborhoods that can be used to process an image.

- Rectangular sampling – In most cases, images are sampled by laying a rectangular grid over an image as illustrated in Figure 1. This results in the type of sampling shown in Figure 3ab.
- Hexagonal sampling – An alternative sampling scheme is shown in Figure 3c and is termed hexagonal sampling.

Both sampling schemes have been studied extensively [1] and both represent a possible periodic tiling of the continuous image space. We will restrict our attention, however, to only rectangular sampling as it remains, due to hardware and software considerations, the method of choice.

Local operations produce an output pixel value $b[m=m_o, n=n_o]$ based upon the pixel values in the *neighborhood* of $a[m=m_o, n=n_o]$. Some of the most common neighborhoods are the 4-connected neighborhood and the 8-connected neighborhood in the case of rectangular sampling and the 6-connected neighborhood in the case of hexagonal sampling illustrated in Figure 3.

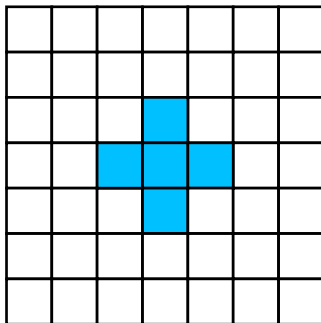


Figure 3a
Rectangular sampling
4-connected

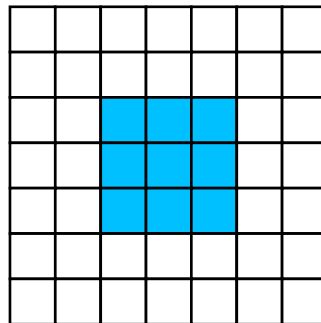


Figure 3b
Rectangular sampling
8-connected

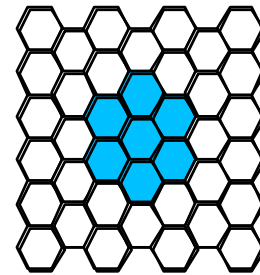


Figure 3c
Hexagonal sampling
6-connected

2.3 VIDEO PARAMETERS

We do not propose to describe the processing of dynamically changing images in this introduction. It is appropriate—given that many static images are derived from video cameras and frame grabbers—to mention the standards that are associated with the three standard video schemes that are currently in worldwide use – NTSC, PAL, and SECAM. This information is summarized in Table 3.

<i>Property</i>	<i>Standard</i>	NTSC	PAL	SECAM
images / second		29.97	25	25
ms / image		33.37	40.0	40.0
lines / image		525	625	625
(horiz./vert.) = aspect ratio		4:3	4:3	4:3
interlace		2:1	2:1	2:1
μs / line		63.56	64.00	64.00

Table 3: Standard video parameters

In an interlaced image the odd numbered lines (1,3,5,...) are scanned in half of the allotted time (e.g. 20 ms in PAL) and the even numbered lines (2,4,6,...) are scanned in the remaining half. The image display must be coordinated with this scanning format. (See Section 8.2.) The reason for interlacing the scan lines of a video image is to reduce the perception of flicker in a displayed image. If one is planning to use images that have been scanned from an interlaced video source, it is important to know if the two half-images have been appropriately “shuffled” by the digitization hardware or if that should be implemented in software. Further, the analysis of moving objects requires special care with interlaced video to avoid “zigzag” edges.

The number of rows (N) from a video source generally corresponds one-to-one with lines in the video image. The number of columns, however, depends on the nature of the electronics that is used to digitize the image. Different frame grabbers for the same video camera might produce $M = 384, 512$, or 768 columns (pixels) per line.

3. Tools

Certain tools are central to the processing of digital images. These include mathematical tools such as *convolution*, *Fourier analysis*, and *statistical* descriptions, and manipulative tools such as *chain codes* and *run codes*. We will present these tools without any specific motivation. The motivation will follow in later sections.

3.1 CONVOLUTION

There are several possible notations to indicate the convolution of two (multi-dimensional) signals to produce an output signal. The most common are: $\ominus$

$$c = a \otimes b = a * b \quad (1)$$

We shall use the first form, $c = a \otimes b$, with the following formal definitions.

In 2D continuous space:

$$c(x, y) = a(x, y) \otimes b(x, y) = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} a(\chi, \zeta) b(x - \chi, y - \zeta) d\chi d\zeta \quad (2)$$

In 2D discrete space:

$$c[m, n] = a[m, n] \otimes b[m, n] = \sum_{j=-\infty}^{+\infty} \sum_{k=-\infty}^{+\infty} a[j, k] b[m - j, n - k] \quad (3)$$

3.2 PROPERTIES OF CONVOLUTION

There are a number of important mathematical properties associated with convolution.

- Convolution is *commutative*.

$$c = a \otimes b = b \otimes a \quad (4)$$

- Convolution is *associative*.

$$c = a \otimes (b \otimes d) = (a \otimes b) \otimes d = a \otimes b \otimes d \quad (5)$$

- Convolution is *distributive*.

$$c = a \otimes (b + d) = (a \otimes b) + (a \otimes d) \quad (6)$$

where a , b , c , and d are all images, either continuous or discrete.

3.3 FOURIER TRANSFORMS

The Fourier transform produces another representation of a signal, specifically a representation as a weighted sum of complex exponentials. Because of Euler's formula:

$$e^{jq} = \cos(q) + j \sin(q) \quad (7)$$

where $j^2 = -1$, we can say that the Fourier transform produces a representation of a (2D) signal as a weighted sum of sines and cosines. The defining formulas for

the forward Fourier and the inverse Fourier transforms are as follows. Given an image a and its Fourier transform A , then the forward transform goes from the spatial domain (either continuous or discrete) to the frequency domain which is always continuous.

$$\text{Forward} - \quad A = \mathcal{F}\{a\} \quad (8)$$

The inverse Fourier transform goes from the frequency domain back to the spatial domain.

$$\text{Inverse} - \quad a = \mathcal{F}^{-1}\{A\} \quad (9)$$

The Fourier transform is a unique and invertible operation so that:

$$a = \mathcal{F}^{-1}\{\mathcal{F}\{a\}\} \quad \text{and} \quad A = \mathcal{F}\{\mathcal{F}^{-1}\{A\}\} \quad (10)$$

The specific formulas for transforming back and forth between the spatial domain and the frequency domain are given below.

In 2D continuous space:

$$\text{Forward} - \quad A(u, v) = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} a(x, y) e^{-j(ux+vy)} dx dy \quad (11)$$

$$\text{Inverse} - \quad a(x, y) = \frac{1}{4\pi^2} \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} A(u, v) e^{+j(ux+vy)} du dv \quad (12)$$

In 2D discrete space:

$$\text{Forward} - \quad A(\Omega, \Psi) = \sum_{m=-\infty}^{+\infty} \sum_{n=-\infty}^{+\infty} a[m, n] e^{-j(\Omega m + \Psi n)} \quad (13)$$

$$\text{Inverse} - \quad a[m, n] = \frac{1}{4\pi^2} \int_{-\pi}^{+\pi} \int_{-\pi}^{+\pi} A(\Omega, \Psi) e^{+j(\Omega m + \Psi n)} d\Omega d\Psi \quad (14)$$

3.4 PROPERTIES OF FOURIER TRANSFORMS

There are a variety of properties associated with the Fourier transform and the inverse Fourier transform. The following are some of the most relevant for digital image processing.

- The Fourier transform is, in general, a complex function of the real frequency variables. As such the transform can be written in terms of its magnitude and phase.

$$A(u, v) = |A(u, v)| e^{j\varphi(u, v)} \quad A(\Omega, \Psi) = |A(\Omega, \Psi)| e^{j\varphi(\Omega, \Psi)} \quad (15)$$

- A 2D signal can also be complex and thus written in terms of its magnitude and phase.

$$a(x, y) = |a(x, y)| e^{j\vartheta(x, y)} \quad a[m, n] = |a[m, n]| e^{j\vartheta[m, n]} \quad (16)$$

- If a 2D signal is real, then the Fourier transform has certain symmetries.

$$A(u, v) = A^*(-u, -v) \quad A(\Omega, \Psi) = A^*(-\Omega, -\Psi) \quad (17)$$

The symbol (*) indicates complex conjugation. For real signals eq. (17) leads directly to:

$$\begin{aligned} |A(u, v)| &= |A(-u, -v)| & \varphi(u, v) &= -\varphi(-u, -v) \\ |A(\Omega, \Psi)| &= |A(-\Omega, -\Psi)| & \varphi(\Omega, \Psi) &= -\varphi(-\Omega, -\Psi) \end{aligned} \quad (18)$$

- If a 2D signal is real and even, then the Fourier transform is real and even.

$$A(u, v) = A(-u, -v) \quad A(\Omega, \Psi) = A(-\Omega, -\Psi) \quad (19)$$

- The Fourier and the inverse Fourier transforms are linear operations.

$$\begin{aligned} \mathcal{F}\{w_1 a + w_2 b\} &= \mathcal{F}\{w_1 a\} + \mathcal{F}\{w_2 b\} = w_1 A + w_2 B \\ \mathcal{F}^{-1}\{w_1 A + w_2 B\} &= \mathcal{F}^{-1}\{w_1 A\} + \mathcal{F}^{-1}\{w_2 B\} = w_1 a + w_2 b \end{aligned} \quad (20)$$

where a and b are 2D signals (images) and w_1 and w_2 are arbitrary, complex constants.

- The Fourier transform in discrete space, $A(\Omega, \Psi)$, is periodic in both Ω and Ψ . Both periods are 2π .

$$A(\Omega + 2\pi j, \Psi + 2\pi k) = A(\Omega, \Psi) \quad j, k \text{ integers} \quad (21)$$

- The energy, E , in a signal can be measured either in the spatial domain or the frequency domain. For a signal with finite energy:

Parseval's theorem (2D continuous space):

$$E = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} |a(x, y)|^2 dx dy = \frac{1}{4\pi^2} \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} |A(u, v)|^2 du dv \quad (22)$$

Parseval's theorem (2D discrete space):

$$E = \sum_{m=-\infty}^{+\infty} \sum_{n=-\infty}^{+\infty} |a[m, n]|^2 = \frac{1}{4\pi^2} \int_{-\pi}^{+\pi} \int_{-\pi}^{+\pi} |A(\Omega, \Psi)|^2 d\Omega d\Psi \quad (23)$$

This “signal energy” is not to be confused with the physical energy in the phenomenon that produced the signal. If, for example, the value $a[m, n]$ represents a photon count, then the *physical* energy is proportional to the amplitude, a , and not the square of the amplitude. This is generally the case in video imaging.

- Given three, multi-dimensional signals a , b , and c and their Fourier transforms A , B , and C :

$$c = a \otimes b \quad \xleftrightarrow{\mathcal{F}} \quad C = A \bullet B \quad (24)$$

and

$$c = a \bullet b \quad \xleftrightarrow{\mathcal{F}} \quad C = \frac{1}{4\pi^2} A \otimes B$$

In words, convolution in the spatial domain is equivalent to multiplication in the Fourier (frequency) domain and vice-versa. This is a central result which provides not only a methodology for the implementation of a convolution but also insight into how two signals interact with each other—under convolution—to produce a third signal. We shall make extensive use of this result later.

- If a two-dimensional signal $a(x, y)$ is scaled in its spatial coordinates then:

$$\begin{aligned} \text{If} \quad a(x, y) &\rightarrow a(M_x \bullet x, M_y \bullet y) \\ \text{Then} \quad A(u, v) &\rightarrow A\left(\frac{u}{M_x}, \frac{v}{M_y}\right) / |M_x \bullet M_y| \end{aligned} \quad (25)$$

- If a two-dimensional signal $a(x,y)$ has Fourier spectrum $A(u,v)$ then:

$$\begin{aligned} A(u=0, v=0) &= \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} a(x,y) dx dy \\ a(x=0, y=0) &= \frac{1}{4\pi^2} \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} A(u,v) dx dy \end{aligned} \quad (26)$$

- If a two-dimensional signal $a(x,y)$ has Fourier spectrum $A(u,v)$ then:

$$\begin{aligned} \frac{\partial a(x,y)}{\partial x} &\overset{\mathcal{F}}{\leftrightarrow} juA(u,v) & \frac{\partial a(x,y)}{\partial y} &\overset{\mathcal{F}}{\leftrightarrow} jvA(u,v) \\ \frac{\partial^2 a(x,y)}{\partial x^2} &\overset{\mathcal{F}}{\leftrightarrow} -u^2 A(u,v) & \frac{\partial^2 a(x,y)}{\partial y^2} &\overset{\mathcal{F}}{\leftrightarrow} -v^2 A(u,v) \end{aligned} \quad (27)$$

3.4.1 Importance of phase and magnitude

Equation (15) indicates that the Fourier transform of an image can be complex. This is illustrated below in Figures 4a-c. Figure 4a shows the original image $a[m,n]$, Figure 4b the magnitude in a scaled form as $\log(|A(\Omega,\Psi)|)$, and Figure 4c the phase $\phi(\Omega,\Psi)$.



Figure 4a

Original

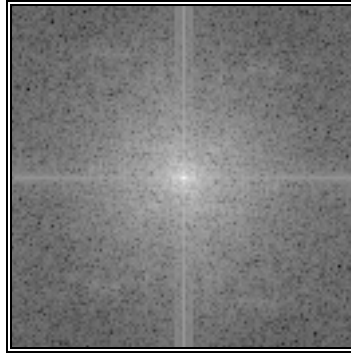


Figure 4b

$\log(|A(\Omega,\Psi)|)$

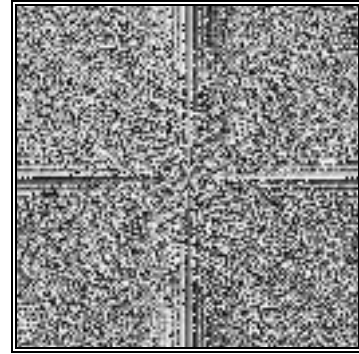


Figure 4c

$\phi(\Omega,\Psi)$

Both the magnitude and the phase functions are necessary for the complete reconstruction of an image from its Fourier transform. Figure 5a shows what happens when Figure 4a is restored solely on the basis of the magnitude information and Figure 5b shows what happens when Figure 4a is restored solely on the basis of the phase information.

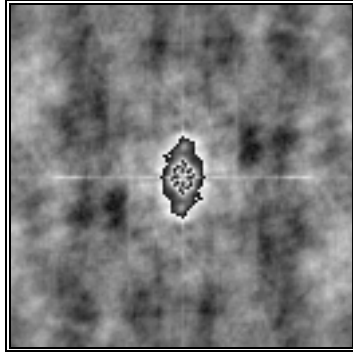


Figure 5a

$$\varphi(\Omega, \Psi) = 0$$



Figure 5b

$$|A(\Omega, \Psi)| = \text{constant}$$

Neither the magnitude information nor the phase information is sufficient to restore the image. The magnitude-only image (Figure 5a) is unrecognizable and has severe dynamic range problems. The phase-only image (Figure 5b) is barely recognizable, that is, severely degraded in quality.

3.4.2 Circularly symmetric signals

An arbitrary 2D signal $a(x, y)$ can always be written in a polar coordinate system as $a(r, \theta)$. When the 2D signal exhibits a circular symmetry this means that:

$$a(x, y) = a(r, \theta) = a(r) \quad (28)$$

where $r^2 = x^2 + y^2$ and $\tan \theta = y/x$. As a number of physical systems such as lenses exhibit circular symmetry, it is useful to be able to compute an appropriate Fourier representation.

The Fourier transform $A(u, v)$ can be written in polar coordinates $A(q, \xi)$ and then, for a circularly symmetric signal, rewritten as a *Hankel transform*:

$$A(u, v) = \mathcal{F}\{a(x, y)\} = 2\pi \int_0^\infty a(r) J_0(rq) r dr = A(q) \quad (29)$$

where $q^2 = u^2 + v^2$ and $\tan \xi = v/u$ and $J_0(\cdot)$ is a Bessel function of the first kind of order zero.

The inverse *Hankel transform* is given by:

$$a(r) = \frac{1}{2\pi} \int_0^\infty A(q) J_0(rq) q dq \quad (30)$$

The Fourier transform of a circularly symmetric 2D signal is a function of only the radial frequency, q . The dependence on the angular frequency, ξ , has vanished. Further, if $a(x,y) = a(r)$ is real, then it is automatically even due to the circular symmetry. According to equation (19), $A(q)$ will then be real and even.

3.4.3 Examples of 2D signals and transforms

Table 4 shows some basic and useful signals and their 2D Fourier transforms. In using the table entries in the remainder of this chapter we will refer to a spatial domain term as the *point spread function (PSF)* or the *2D impulse response* and its Fourier transforms as the *optical transfer function (OTF)* or simply *transfer function*. Two standard signals used in this table are $u(\bullet)$, the unit step function, and $J_1(\bullet)$, the Bessel function of the first kind. Circularly symmetric signals are treated as functions of r as in eq. (28).

3.5 STATISTICS

In image processing it is quite common to use simple statistical descriptions of images and sub-images. The notion of a statistic is intimately connected to the concept of a probability distribution, generally the distribution of signal amplitudes. For a given region—which could conceivably be an entire image—we can define the probability *distribution* function of the brightnesses in that region and the probability *density* function of the brightnesses in that region. We will assume in the discussion that follows that we are dealing with a digitized image $a[m,n]$.

3.5.1 Probability distribution function of the brightnesses

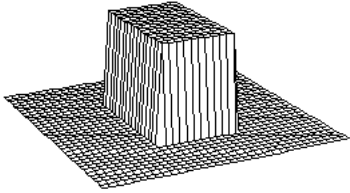
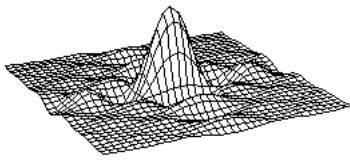
The probability distribution function, $P(a)$, is the probability that a brightness chosen from the region is less than or equal to a given brightness value a . As a increases from $-\infty$ to $+\infty$, $P(a)$ increases from 0 to 1. $P(a)$ is monotonic, non-decreasing in a and thus $dP/da \geq 0$.

3.5.2 Probability density function of the brightnesses

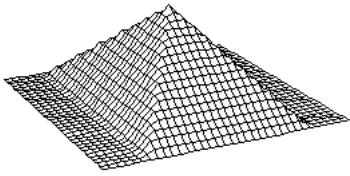
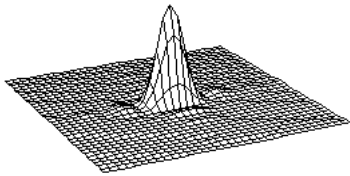
The probability that a brightness in a region falls between a and $a+\Delta a$, given the probability distribution function $P(a)$, can be expressed as $p(a)\Delta a$ where $p(a)$ is the probability density function:

$$p(a)\Delta a = \left(\frac{dP(a)}{da} \right) \Delta a \quad (31)$$

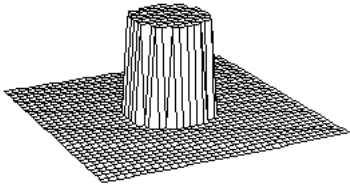
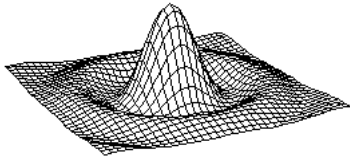
T.1 Rectangle $R_{a,b}(x,y) = \frac{1}{4ab} u(a^2 - x^2) u(b^2 - y^2)$ $\xleftrightarrow{\mathcal{F}} \left(\frac{\sin(au)}{au} \right) \left(\frac{\sin(bv)}{bv} \right)$

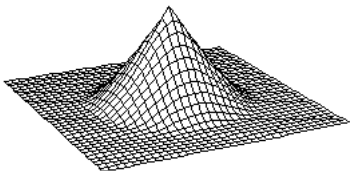
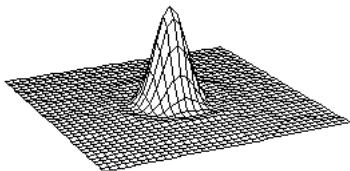
T.2 Pyramid $R_{a,b}(x,y) \otimes R_{a,b}(x,y)$ $\xleftrightarrow{\mathcal{F}} \left(\frac{\sin(au)}{au} \frac{\sin(bv)}{bv} \right)^2$

T.3 Pill Box $P_a(r) = \frac{u(a^2 - r^2)}{\pi a^2}$ $\xleftrightarrow{\mathcal{F}} \frac{2J_1(aq)}{aq}$

T.4 Cone $P_a(r) \otimes P_a(r)$ $\xleftrightarrow{\mathcal{F}} \left(\frac{2J_1(aq)}{aq} \right)^2$

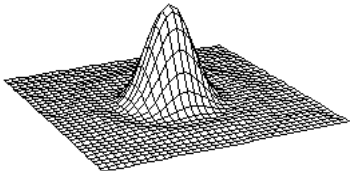
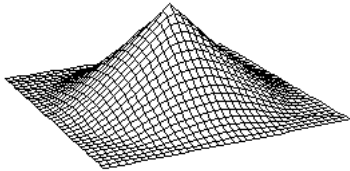
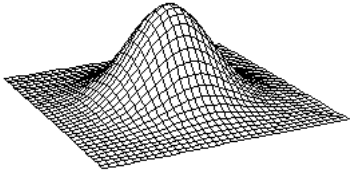
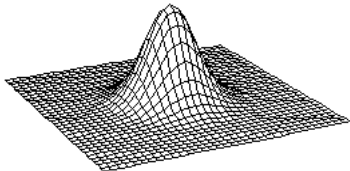
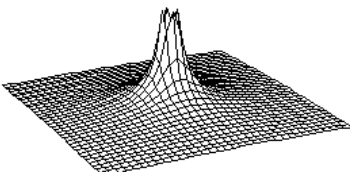
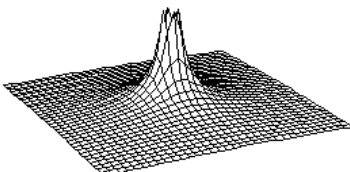
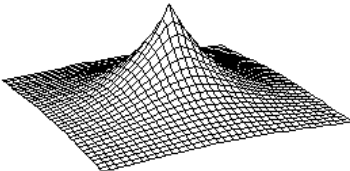
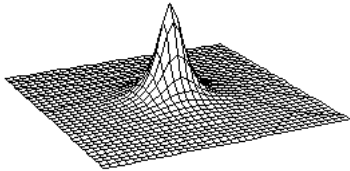
T.5 <i>Airy PSF</i>	$PSF(r) = \frac{1}{\pi} \left(\frac{J_1(\frac{1}{2} q_c r)}{r} \right)^2$	$\begin{aligned} &\mathcal{F} \\ &\leftrightarrow \frac{2}{\pi} \left(\cos^{-1} \frac{q}{q_c} - \frac{q}{q_c} \sqrt{1 - \left(\frac{q}{q_c} \right)^2} \right) u(q_c^2 - q^2) \\ &\text{with } q_c = 2\pi NA / \lambda \end{aligned}$
		
T.6 <i>Gaussian</i>	$g_{2D}(r, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{r^2}{2\sigma^2}\right)$	$\begin{aligned} &\mathcal{F} \\ &\leftrightarrow G_{2D}(q, \sigma) = \exp(-q^2 \sigma^2 / 2) \end{aligned}$
		
T.7 <i>Peak</i>	$\frac{1}{r}$	$\begin{aligned} &\mathcal{F} \\ &\leftrightarrow \frac{2\pi}{q} \end{aligned}$
		
T.8 <i>Exponential Decay</i>	e^{-ar}	$\begin{aligned} &\mathcal{F} \\ &\leftrightarrow \frac{2\pi a}{(q^2 + a^2)^{3/2}} \end{aligned}$
		

Table 4: 2D Images and their Fourier Transforms

Because of the monotonic, non-decreasing character of $P(a)$ we have that:

$$p(a) \geq 0 \quad \text{and} \quad \int_{-\infty}^{+\infty} p(a) da = 1 \quad (32)$$

For an image with quantized (integer) brightness amplitudes, the interpretation of Δa is the width of a brightness interval. We assume constant width intervals. The brightness probability *density* function is frequently estimated by counting the number of times that each brightness occurs in the region to generate a *histogram*, $h[a]$. The histogram can then be normalized so that the total area under the histogram is 1 (eq. (32)). Said another way, the $p[a]$ for a region is the normalized count of the number of pixels, Λ , in a region that have quantized brightness a :

$$p[a] = \frac{1}{\Lambda} h[a] \quad \text{with} \quad \Lambda = \sum_a h[a] \quad (33)$$

The brightness probability *distribution* function for the image shown in Figure 4a is shown in Figure 6a. The (unnormalized) brightness histogram of Figure 4a which is proportional to the estimated brightness probability density function is shown in Figure 6b. The height in this histogram corresponds to the number of pixels with a given brightness.

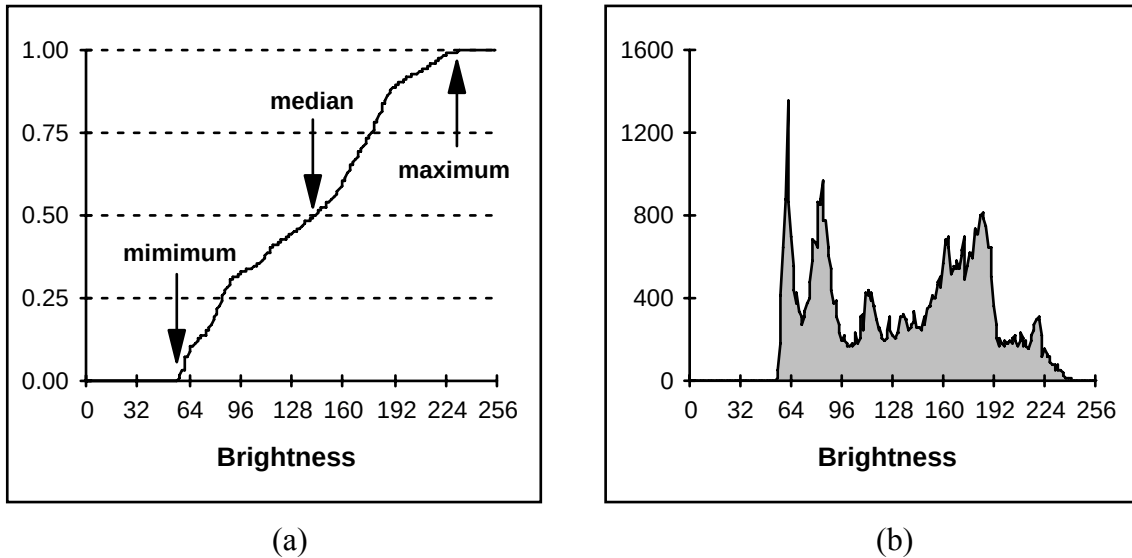


Figure 6: (a) Brightness distribution function of Figure 4a with *minimum*, *median*, and *maximum* indicated. See text for explanation. (b) Brightness histogram of Figure 4a.

Both the distribution function and the histogram as measured from a region are a statistical description of that region. It must be emphasized that both $P[a]$ and $p[a]$ should be viewed as *estimates* of true distributions when they are computed from

a specific region. That is, we view an image and a specific region as one realization of the various random processes involved in the formation of that image and that region. In the same context, the statistics defined below must be viewed as estimates of the underlying parameters.

3.5.3 Average

The average brightness of a region is defined as the *sample mean* of the pixel brightnesses within that region. The average, m_a , of the brightnesses over the Λ pixels within a region ($\mathfrak{R}$) is given by:

$$m_a = \frac{1}{\Lambda} \sum_{(m,n) \in \mathfrak{R}} a[m,n] \quad (34)$$

Alternatively, we can use a formulation based upon the (unnormalized) brightness histogram, $h(a) = \Lambda \cdot p(a)$, with discrete brightness values a . This gives:

$$m_a = \frac{1}{\Lambda} \sum_a a \cdot h[a] \quad (35)$$

The average brightness, m_a , is an estimate of the mean brightness, μ_a , of the underlying brightness probability distribution.

3.5.4 Standard deviation

The *unbiased estimate* of the standard deviation, s_a , of the brightnesses within a region ($\mathfrak{R}$) with Λ pixels is called the *sample standard deviation* and is given by:

$$\begin{aligned} s_a &= \sqrt{\frac{1}{\Lambda-1} \sum_{m,n \in \mathfrak{R}} (a[m,n] - m_a)^2} \\ &= \sqrt{\frac{\sum_{m,n \in \mathfrak{R}} a^2[m,n] - \Lambda m_a^2}{\Lambda-1}} \end{aligned} \quad (36)$$

Using the histogram formulation gives:

$$s_a = \sqrt{\frac{\left(\sum_a a^2 \cdot h[a] \right) - \Lambda \cdot m_a^2}{\Lambda-1}} \quad (37)$$

The standard deviation, s_a , is an estimate of σ_a of the underlying brightness probability distribution.

3.5.5 Coefficient-of-variation

The dimensionless *coefficient-of-variation*, CV , is defined as:

$$CV = \frac{s_a}{m_a} \times 100\% \quad (38)$$

3.5.6 Percentiles

The percentile, $p\%$, of an *unquantized* brightness distribution is defined as that value of the brightness a such that:

$$P(a) = p\%$$

or equivalently

$$\int_{-\infty}^a p(\alpha) d\alpha = p\% \quad (39)$$

Three special cases are frequently used in digital image processing.

- 0% the *minimum* value in the region
- 50% the *median* value in the region
- 100% the *maximum* value in the region

All three of these values can be determined from Figure 6a.

3.5.7 Mode

The mode of the distribution is the most frequent brightness value. There is no guarantee that a mode exists or that it is unique.

3.5.8 Signal-to-Noise ratio

The signal-to-noise ratio, SNR , can have several definitions. The noise is characterized by its standard deviation, s_n . The characterization of the signal can differ. If the signal is known to lie between two boundaries, $a_{min} \leq a \leq a_{max}$, then the SNR is defined as:

$$\text{Bounded signal} - \quad SNR = 20 \log_{10} \left(\frac{a_{max} - a_{min}}{s_n} \right) dB \quad (40)$$

If the signal is not bounded but has a statistical distribution then two other definitions are known:

Stochastic signal –

$$S \text{ \& } N \text{ inter-dependent} \quad SNR = 20 \log_{10} \left(\frac{m_a}{s_n} \right) dB \quad (41)$$

$$S \text{ \& } N \text{ independent} \quad SNR = 20 \log_{10} \left(\frac{s_a}{s_n} \right) dB \quad (42)$$

where m_a and s_a are defined above.

The various statistics are given in Table 5 for the image and the region shown in Figure 7.



Figure 7

Region is the interior of the circle.

Statistic	Image	ROI
Average	137.7	219.3
Standard Deviation	49.5	4.0
Minimum	56	202
Median	141	220
Maximum	241	226
Mode	62	220
SNR (db)	NA	33.3

Table 5

Statistics from Figure 7

A *SNR* calculation for the *entire* image based on eq. (40) is not directly available. The variations in the image brightnesses that lead to the large value of s ($=49.5$) are not, in general, due to noise but to the variation in local information. With the help of the region there is a way to estimate the *SNR*. We can use the $s_{\mathfrak{R}}$ ($=4.0$) and the dynamic range, $a_{max} - a_{min}$, for the image ($=241-56$) to calculate a global *SNR* ($=33.3$ dB). The underlying assumptions are that 1) the signal is approximately constant in that region and the variation in the region is therefore due to noise, and, 2) that the noise is the same over the entire image with a standard deviation given by $s_n = s_{\mathfrak{R}}$.

3.6 CONTOUR REPRESENTATIONS

When dealing with a region or object, several compact representations are available that can facilitate manipulation of and measurements on the object. In each case we assume that we begin with an image representation of the object as shown in Figure 8a,b. Several techniques exist to represent the region or object by describing its contour.

3.6.1 Chain code

This representation is based upon the work of Freeman [11]. We follow the contour in a clockwise manner and keep track of the directions as we go from one

contour pixel to the next. For the standard implementation of the chain code we consider a contour pixel to be an object pixel that has a background (non-object) pixel as one or more of its 4-connected neighbors. See Figures 3a and 8c.

The codes associated with eight possible directions are the chain codes and, with x as the current contour pixel position, the codes are generally defined as:

$$\begin{array}{rcc} & 3 & 2 & 1 \\ \text{Chain codes} & = & 4 & x & 0 \\ & & 5 & 6 & 7 \end{array} \quad (43)$$

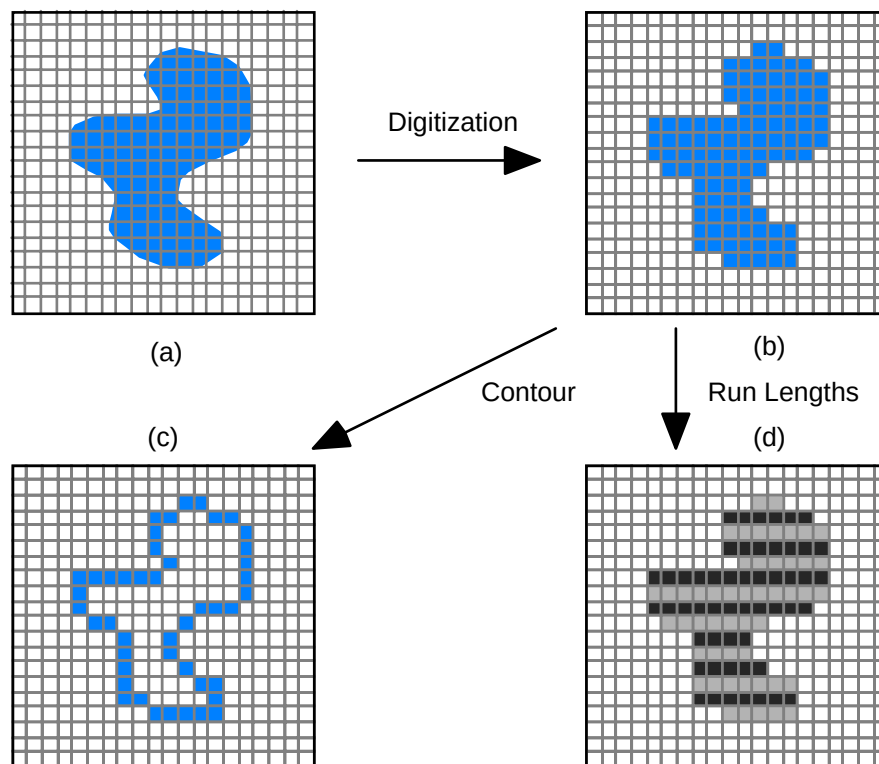


Figure 8: Region (shaded) as it is transformed from (a) continuous to (b) discrete form and then considered as a (c) contour or (d) run lengths illustrated in alternating colors.

3.6.2 Chain code properties

- Even codes $\{0,2,4,6\}$ correspond to horizontal and vertical directions; odd codes $\{1,3,5,7\}$ correspond to the diagonal directions.
- Each code can be considered as the angular direction, in multiples of 45° , that we must move to go from one contour pixel to the next.
- The absolute coordinates $[m,n]$ of the first contour pixel (e.g. top, leftmost) together with the chain code of the contour represent a complete description of the discrete region contour.

- When there is a change between two consecutive chain codes, then the contour has changed direction. This point is defined as a *corner*.

3.6.3 “Crack” code

An alternative to the chain code for contour encoding is to use neither the contour pixels associated with the object nor the contour pixels associated with background but rather the line, the “crack”, in between. This is illustrated with an enlargement of a portion of Figure 8 in Figure 9.

The “crack” code can be viewed as a chain code with four possible directions instead of eight.

$$\text{Crack codes} = \begin{matrix} & 1 \\ 2 & x & 0 \\ & 3 \end{matrix} \quad (44)$$

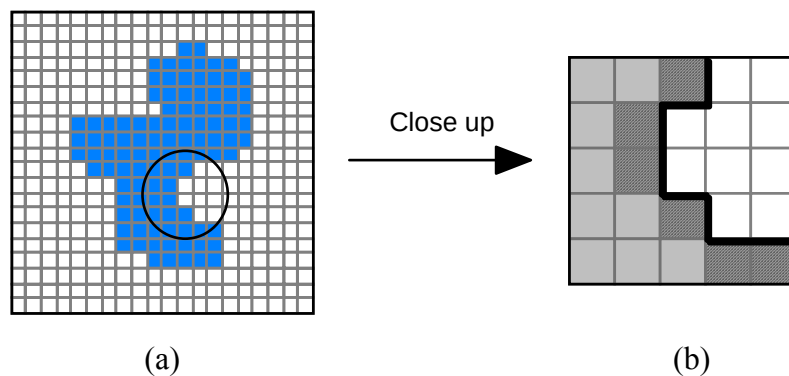


Figure 9: (a) Object including part to be studied. (b) Contour pixels as used in the chain code are diagonally shaded. The “crack” is shown with the thick black line.

The chain code for the enlarged section of Figure 9b, from top to bottom, is {5,6,7,7,0}. The crack code is {3,2,3,3,0,3,0,0}.

3.6.4 Run codes

A third representation is based on coding the consecutive pixels along a row—a run—that belong to an object by giving the starting position of the run and the ending position of the run. Such runs are illustrated in Figure 8d. There are a number of alternatives for the precise definition of the positions. Which alternative should be used depends upon the application and thus will not be discussed here.

4. Perception

Many image processing applications are intended to produce images that are to be viewed by human observers (as opposed to, say, automated industrial inspection.) It is therefore important to understand the characteristics and limitations of the human visual system—to understand the “receiver” of the 2D signals. At the outset it is important to realize that 1) the human visual system is not well understood, 2) no objective measure exists for judging the quality of an image that corresponds to human assessment of image quality, and, 3) the “typical” human observer does not exist. Nevertheless, research in perceptual psychology has provided some important insights into the visual system. See, for example, Stockham [12].

4.1 BRIGHTNESS SENSITIVITY

There are several ways to describe the sensitivity of the human visual system. To begin, let us assume that a homogeneous region in an image has an intensity as a function of wavelength (color) given by $I(\lambda)$. Further let us assume that $I(\lambda) = I_o$, a constant.

4.1.1 Wavelength sensitivity

The perceived intensity as a function of λ , the spectral sensitivity, for the “typical observer” is shown in Figure 10 [13].

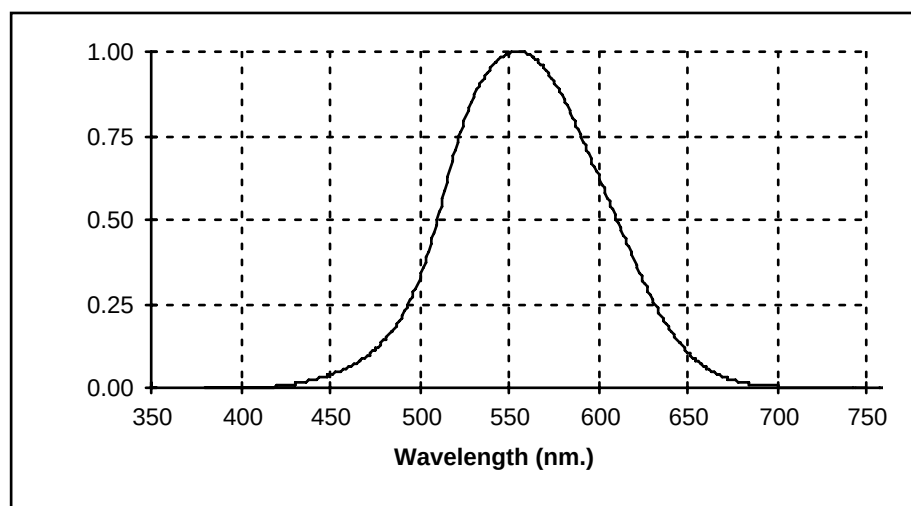


Figure 10: Spectral Sensitivity of the “typical” human observer

4.1.2 Stimulus sensitivity

If the constant intensity (brightness) I_o is allowed to vary then, to a good approximation, the visual response, R , is proportional to the logarithm of the intensity. This is known as the Weber–Fechner law:

$$R = \log(I_o) \quad (45)$$

The implications of this are easy to illustrate. Equal *perceived* steps in brightness, $\Delta R = k$, require that the physical brightness (the stimulus) increases exponentially. This is illustrated in Figure 11ab.

A horizontal line through the top portion of Figure 11a shows a linear increase in objective brightness (Figure 11b) but a logarithmic increase in subjective brightness. A horizontal line through the bottom portion of Figure 11a shows an exponential increase in objective brightness (Figure 11b) but a linear increase in subjective brightness.

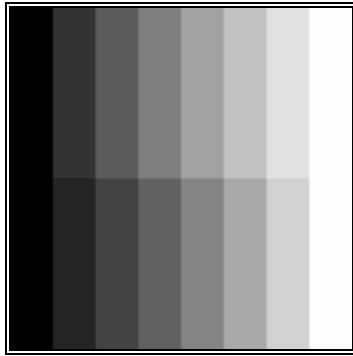


Figure 11a

(top) Brightness step $\Delta I = k$
(bottom) Brightness step $\Delta I = k \cdot I$

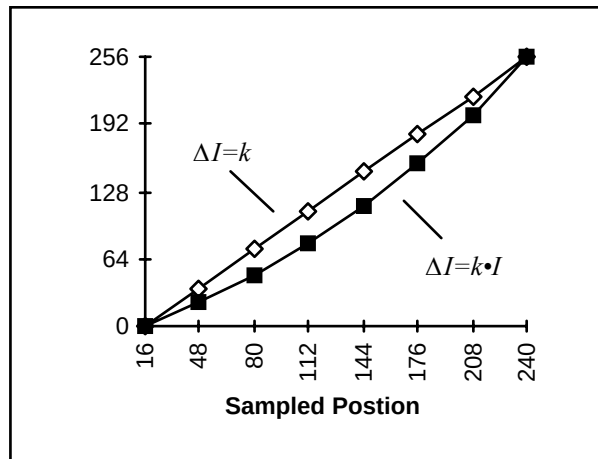


Figure 11b

Actual brightnesses plus interpolated values

The *Mach band effect* is visible in Figure 11a. Although the physical brightness is constant across each vertical stripe, the human observer perceives an “undershoot” and “overshoot” in brightness at what is physically a step edge. Thus, just before the step, we see a slight decrease in brightness compared to the true physical value. After the step we see a slight overshoot in brightness compared to the true physical value. The total effect is one of increased, local, *perceived* contrast at a step edge in brightness.

4.2 SPATIAL FREQUENCY SENSITIVITY

If the constant intensity (brightness) I_o is replaced by a sinusoidal grating with increasing spatial frequency (Figure 12a), it is possible to determine the spatial frequency sensitivity. The result is shown in Figure 12b [14, 15].

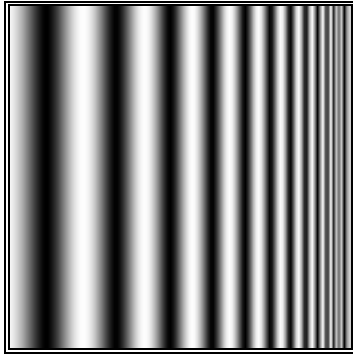


Figure 12a

Sinusoidal test grating

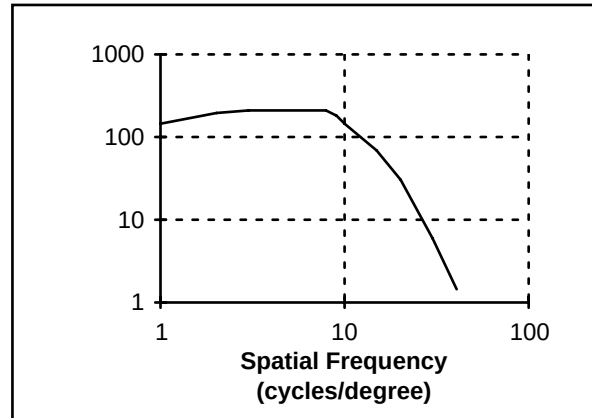


Figure 12b

Spatial frequency sensitivity

To translate these data into common terms, consider an “ideal” computer monitor at a viewing distance of 50 cm. The spatial frequency that will give maximum response is at 10 cycles per degree. (See Figure 12b.) The one degree at 50 cm translates to $50 \tan(1^\circ) = 0.87$ cm on the computer screen. Thus the spatial frequency of maximum response $f_{\max} = 10 \text{ cycles} / 0.87 \text{ cm} = 11.46 \text{ cycles/cm}$ at this viewing distance. Translating this into a general formula gives:

$$f_{\max} = \frac{10}{d \cdot \tan(1^\circ)} = \frac{572.9}{d} \text{ cycles/cm} \quad (46)$$

where d = viewing distance measured in cm.

4.3 COLOR SENSITIVITY

Human color perception is an exceedingly complex topic. As such we can only present a brief introduction here. The physical perception of color is based upon three color pigments in the retina.

4.3.1 Standard observer

Based upon psychophysical measurements, standard curves have been adopted by the CIE (Commission Internationale de l’Eclairage) as the sensitivity curves for the “typical” observer for the three “pigments” $\bar{x}(\lambda)$, $\bar{y}(\lambda)$, and $\bar{z}(\lambda)$. These are shown in Figure 13. These are not the *actual* pigment absorption characteristics found in the “standard” human retina but rather sensitivity curves derived from actual data [10].

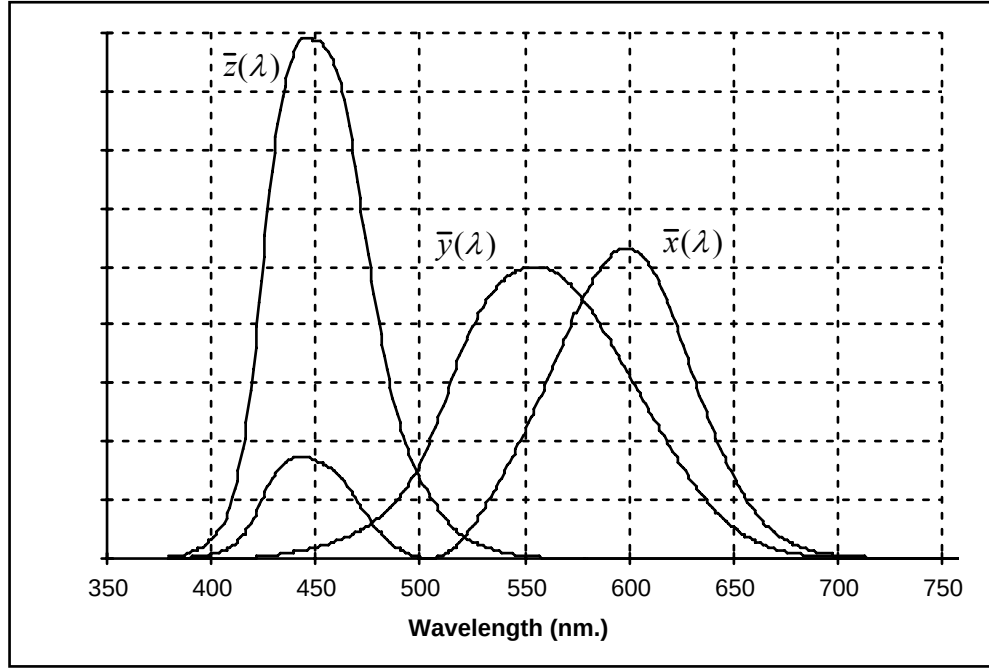


Figure 13: Standard observer spectral sensitivity curves.

For an arbitrary homogeneous region in an image that has an intensity as a function of wavelength (color) given by $I(\lambda)$, the three responses are called the *tristimulus values*:

$$X = \int_0^{\infty} I(\lambda) \bar{x}(\lambda) d\lambda \quad Y = \int_0^{\infty} I(\lambda) \bar{y}(\lambda) d\lambda \quad Z = \int_0^{\infty} I(\lambda) \bar{z}(\lambda) d\lambda \quad (47)$$

4.3.2 CIE chromaticity coordinates

The *chromaticity coordinates* which describe the perceived color information are defined as:

$$x = \frac{X}{X+Y+Z} \quad y = \frac{Y}{X+Y+Z} \quad z = 1 - (x + y) \quad (48)$$

The red chromaticity coordinate is given by x and the green chromaticity coordinate by y . The tristimulus values are linear in $I(\lambda)$ and thus the absolute intensity information has been lost in the calculation of the chromaticity coordinates $\{x, y\}$. All color distributions, $I(\lambda)$, that appear to an observer as having the same color will have the same chromaticity coordinates.

If we use a tunable source of pure color (such as a dye laser), then the intensity can be modeled as $I(\lambda) = \delta(\lambda - \lambda_o)$ with $\delta(\bullet)$ as the impulse function. The collection of chromaticity coordinates $\{x, y\}$ that will be generated by varying λ_o gives the *CIE chromaticity triangle* as shown in Figure 14.

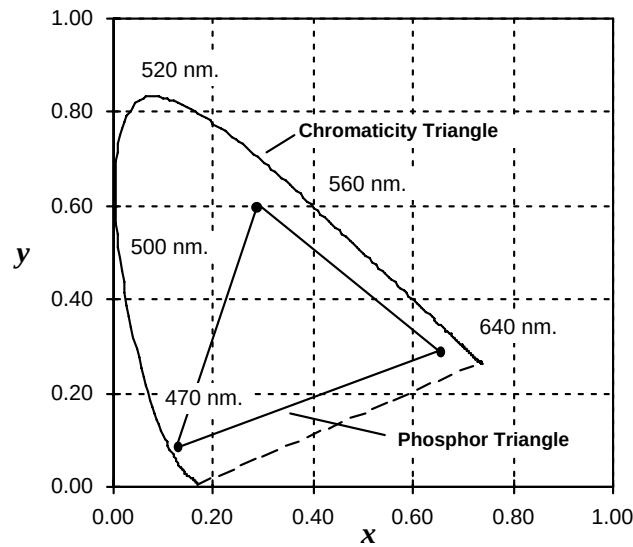


Figure 14: Chromaticity diagram containing the *CIE chromaticity triangle* associated with pure spectral colors and the triangle associated with CRT phosphors.

Pure spectral colors are along the boundary of the chromaticity triangle. All other colors are inside the triangle. The chromaticity coordinates for some standard sources are given in Table 6.

<i>Source</i>	<i>x</i>	<i>y</i>
Fluorescent lamp @ 4800 °K	0.35	0.37
Sun @ 6000 °K	0.32	0.33
Red Phosphor (europium yttrium vanadate)	0.68	0.32
Green Phosphor (zinc cadmium sulfide)	0.28	0.60
Blue Phosphor (zinc sulfide)	0.15	0.07

Table 6: Chromaticity coordinates for standard sources.

The description of color on the basis of chromaticity coordinates not only permits an analysis of color but provides a synthesis technique as well. Using a mixture of two color sources, it is possible to generate any of the colors along the line connecting their respective chromaticity coordinates. Since we cannot have a negative number of photons, this means the mixing coefficients must be positive. Using three color sources such as the red, green, and blue phosphors on CRT monitors leads to the set of colors defined by the *interior* of the “phosphor triangle” shown in Figure 14.

The formulas for converting from the tristimulus values (X,Y,Z) to the well-known CRT colors (R,G,B) and back are given by:

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.9107 & -0.5326 & -0.2883 \\ -0.9843 & 1.9984 & -0.0283 \\ 0.0583 & -0.1185 & 0.8986 \end{bmatrix} \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (49)$$

and

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.6067 & 0.1736 & 0.2001 \\ 0.2988 & 0.5868 & 0.1143 \\ 0.0000 & 0.0661 & 1.1149 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (50)$$

As long as the position of a desired color (X,Y,Z) is inside the phosphor triangle in Figure 14, the values of R , G , and B as computed by eq. (49) will be positive and can therefore be used to drive a CRT monitor.

It is incorrect to assume that a small displacement anywhere in the chromaticity diagram (Figure 14) will produce a proportionally small change in the *perceived* color. An empirically-derived chromaticity space where this property is approximated is the (u',v') space:

$$\begin{aligned} u' &= \frac{4x}{-2x+12y+3} & v' &= \frac{9y}{-2x+12y+3} \\ \text{and} & & & \\ x &= \frac{9u'}{6u'-16v'+12} & y &= \frac{4v'}{6u'-16v'+12} \end{aligned} \quad (51)$$

Small changes almost anywhere in the (u',v') chromaticity space produce equally small changes in the perceived colors.

4.4 OPTICAL ILLUSIONS

The description of the human visual system presented above is couched in standard engineering terms. This could lead one to conclude that there is sufficient knowledge of the human visual system to permit modeling the visual system with standard system analysis techniques. Two simple examples of optical illusions, shown in Figure 15, illustrate that this system approach would be a gross oversimplification. Such models should only be used with extreme care.

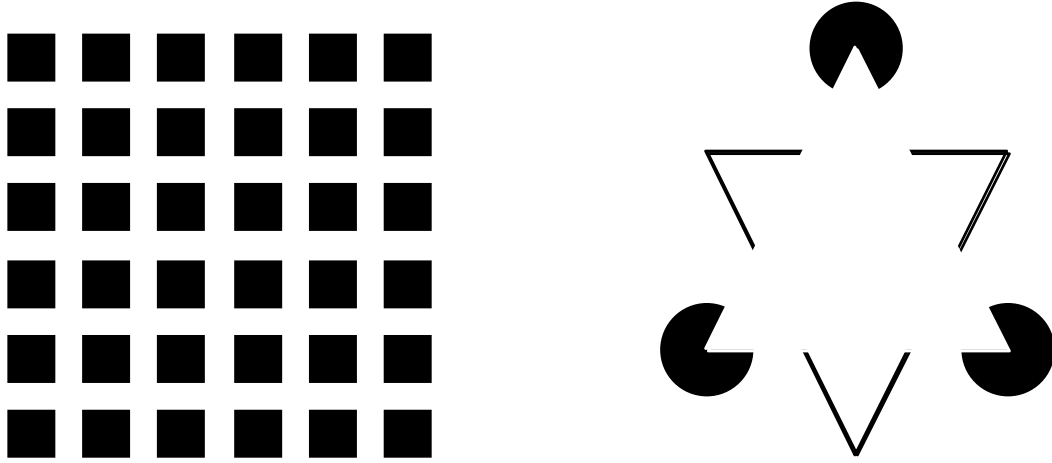


Figure 15: Optical Illusions

The left illusion induces the illusion of gray values in the eye that the brain “knows” does not exist. Further, there is a sense of dynamic change in the image due, in part, to the saccadic movements of the eye. The right illusion, Kanizsa’s triangle, shows enhanced contrast and false contours [14] neither of which can be explained by the system-oriented aspects of visual perception described above.

5. Image Sampling

Converting from a continuous image $a(x,y)$ to its digital representation $b[m,n]$ requires the process of sampling. In the ideal sampling system $a(x,y)$ is multiplied by an ideal 2D impulse train:

$$\begin{aligned} b_{ideal}[m,n] &= a(x,y) \cdot \sum_{m=-\infty}^{+\infty} \sum_{n=-\infty}^{+\infty} \delta(x-mX_o, y-nY_o) \\ &= \sum_{m=-\infty}^{+\infty} \sum_{n=-\infty}^{+\infty} a(mX_o, nY_o) \delta(x-mX_o, y-nY_o) \end{aligned} \quad (52)$$

where X_o and Y_o are the sampling distances or intervals and $\delta(\bullet, \bullet)$ is the ideal impulse function. (At some point, of course, the impulse function $\delta(x,y)$ is converted to the discrete impulse function $\delta[m,n]$.) *Square sampling* implies that $X_o = Y_o$. Sampling with an impulse function corresponds to sampling with an infinitesimally small point. This, however, does not correspond to the usual situation as illustrated in Figure 1. To take the effects of a *finite* sampling aperture $p(x,y)$ into account, we can modify the sampling model as follows:

$$b[m,n] = (a(x,y) \otimes p(x,y)) \cdot \sum_{m=-\infty}^{+\infty} \sum_{n=-\infty}^{+\infty} \delta(x-mX_o, y-nY_o) \quad (53)$$

The combined effect of the aperture and sampling are best understood by examining the Fourier domain representation.

$$B(\Omega, \Psi) = \frac{1}{4\pi^2} \sum_{m=-\infty}^{+\infty} \sum_{n=-\infty}^{+\infty} A(\Omega - m\Omega_s, \Psi - n\Psi_s) \cdot P(\Omega - m\Omega_s, \Psi - n\Psi_s) \quad (54)$$

where $\Omega_s = 2\pi/X_o$ is the sampling frequency in the x direction and $\Psi_s = 2\pi/Y_o$ is the sampling frequency in the y direction. The aperture $p(x,y)$ is frequently square, circular, or Gaussian with the associated $P(\Omega, \Psi)$. (See Table 4.) The periodic nature of the spectrum, described in eq. (21) is clear from eq. (54).

5.1 SAMPLING DENSITY FOR IMAGE PROCESSING

To prevent the possible *aliasing* (overlapping) of spectral terms that is inherent in eq. (54) two conditions must hold:

- *Bandlimited* $A(u,v)$ –

$$|A(u,v)| \equiv 0 \quad \text{for} \quad |u| > u_c \quad \text{and} \quad |v| > v_c \quad (55)$$

- *Nyquist sampling frequency* –

$$\Omega_s > 2 \cdot u_c \quad \text{and} \quad \Psi_s > 2 \cdot v_c \quad (56)$$

where u_c and v_c are the *cutoff frequencies* in the x and y direction, respectively. Images that are acquired through lenses that are circularly-symmetric, aberration-free, and diffraction-limited will, in general, be bandlimited. The lens acts as a lowpass filter with a cutoff frequency in the frequency domain (eq. (11)) given by:

$$u_c = v_c = \frac{2NA}{\lambda} \quad (57)$$

where NA is the numerical aperture of the lens and λ is the shortest wavelength of light used with the lens [16]. If the lens does not meet one or more of these assumptions then it will still be bandlimited but at lower cutoff frequencies than those given in eq. (57). When working with the F-number (F) of the optics instead of the NA and in air (with *index of refraction* = 1.0), eq. (57) becomes:

$$u_c = v_c = \frac{2}{\lambda} \left(\frac{1}{\sqrt{4F^2 + 1}} \right) \quad (58)$$

5.1.1 Sampling aperture

The aperture $p(x,y)$ described above will have only a marginal effect on the final signal if the two conditions eqs. (56) and (57) are satisfied. Given, for example, the distance between samples X_o equals Y_o and a sampling aperture that is not wider than X_o , the effect on the overall spectrum—due to the $A(u,v)P(u,v)$ behavior implied by eq.(53)—is illustrated in Figure 16 for square and Gaussian apertures.

The spectra are evaluated along one axis of the 2D Fourier transform. The Gaussian aperture in Figure 16 has a width such that the sampling interval X_o contains $\pm 3\sigma$ (99.7%) of the Gaussian. The rectangular apertures have a width such that one occupies 95% of the sampling interval and the other occupies 50% of the sampling interval. The 95% width translates to a *fill factor* of 90% and the 50% width to a *fill factor* of 25%. The *fill factor* is discussed in Section 7.5.2.

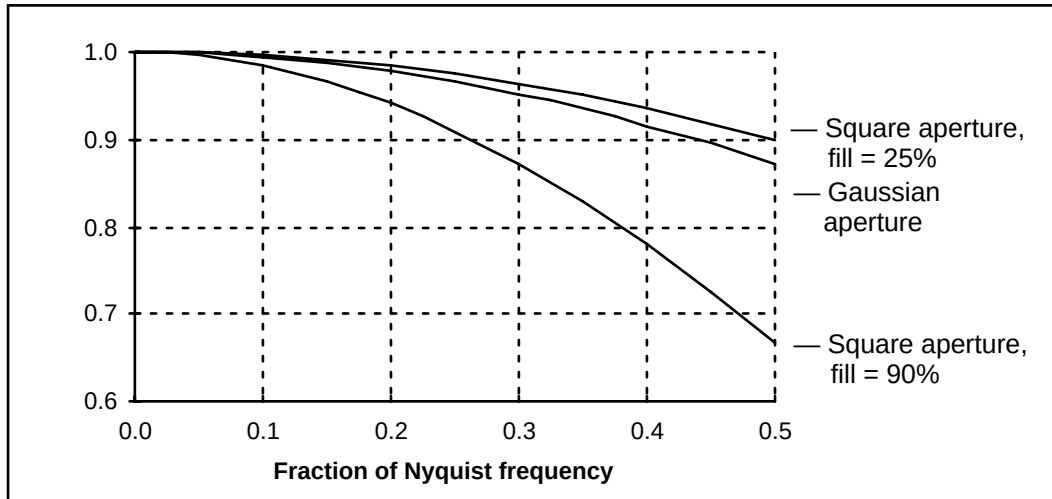


Figure 16: Aperture spectra $P(u,v=0)$ for frequencies up to half the Nyquist frequency. For explanation of “fill” see text.

5.2 SAMPLING DENSITY FOR IMAGE ANALYSIS

The “rules” for choosing the sampling density when the goal is image analysis—as opposed to image processing—are different. The fundamental difference is that the digitization of objects in an image into a collection of pixels introduces a form of spatial quantization noise that is not bandlimited. This leads to the following results for the choice of sampling density when one is interested in the measurement of area and (perimeter) length.

5.2.1 Sampling for area measurements

Assuming square sampling, $X_o = Y_o$ and the unbiased algorithm for estimating area which involves simple pixel counting, the CV (see eq. (38)) of the area measurement is related to the sampling density by [17]:

$$2D: \lim_{S \rightarrow \infty} CV(S) = k_2 S^{-3/2} \quad 3D: \lim_{S \rightarrow \infty} CV(S) = k_3 S^{-2} \quad (59)$$

and in D dimensions:

$$\lim_{S \rightarrow \infty} CV(S) = k_D S^{-(D+1)/2} \quad (60)$$

where S is the number of samples *per object diameter*. In 2D the measurement is area, in 3D volume, and in D -dimensions hypervolume.

5.2.2 Sampling for length measurements

Again assuming square sampling and algorithms for estimating length based upon the Freeman chain-code representation (see Section 3.6.1), the CV of the length measurement is related to the sampling density *per unit length* as shown in Figure 17 (see [18, 19].)

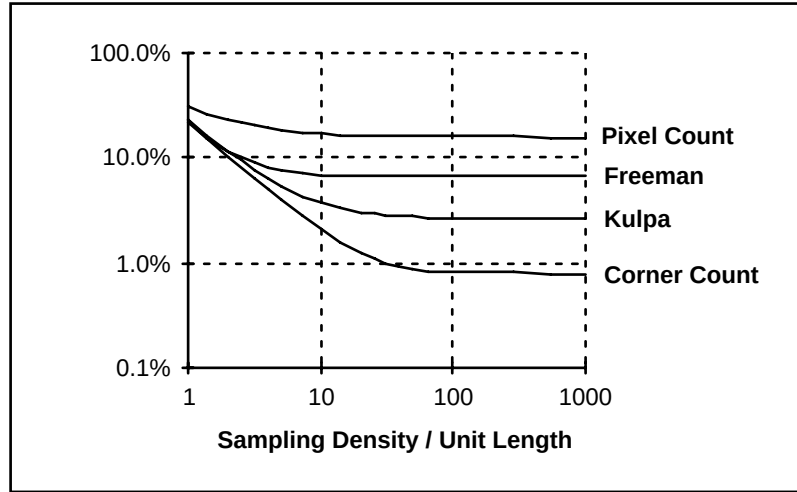


Figure 17: CV of length measurement for various algorithms.

The curves in Figure 17 were developed in the context of straight lines but similar results have been found for curves and closed contours. The specific formulas for length estimation use a chain code representation of a line and are based upon a linear combination of three numbers:

$$L = \alpha \cdot N_e + \beta \cdot N_o + \gamma \cdot N_c \quad (61)$$

where N_e is the number of even chain codes, N_o the number of odd chain codes, and N_c the number of corners. The specific formulas are given in Table 7.

<i>Coefficients Formula</i>	α	β	γ	<i>Reference</i>
Pixel count	1	1	0	[18]
Freeman	1	$\sqrt{2}$	0	[11]
Kulpa	0.9481	$0.9481 \cdot \sqrt{2}$	0	[20]
Corner count	0.980	1.406	-0.091	[21]

Table 7: Length estimation formulas based on chain code counts (N_e , N_o , N_c)

5.2.3 Conclusions on sampling

If one is interested in image processing, one should choose a sampling density based upon classical signal theory, that is, the Nyquist sampling theory. If one is interested in image analysis, one should choose a sampling density based upon the desired measurement accuracy (*bias*) and precision (*CV*). In a case of uncertainty, one should choose the higher of the two sampling densities (frequencies).

6. Noise

Images acquired through modern sensors may be contaminated by a variety of noise sources. By noise we refer to stochastic variations as opposed to deterministic distortions such as shading or lack of focus. We will assume for this section that we are dealing with images formed from light using modern electro-optics. In particular we will assume the use of modern, charge-coupled device (CCD) cameras where photons produce electrons that are commonly referred to as photoelectrons. Nevertheless, most of the observations we shall make about noise and its various sources hold equally well for other imaging modalities.

While modern technology has made it possible to reduce the noise levels associated with various electro-optical devices to almost negligible levels, one noise source can never be eliminated and thus forms the limiting case when all other noise sources are “eliminated”.

6.1 PHOTON NOISE

When the physical signal that we observe is based upon light, then the quantum nature of light plays a significant role. A single photon at $\lambda = 500$ nm carries an energy of $E = h\nu = hc/\lambda = 3.97 \times 10^{-19}$ Joules. Modern CCD cameras are sensitive enough to be able to count individual photons. (Camera sensitivity will be discussed in Section 7.2.) The noise problem arises from the fundamentally

statistical nature of photon production. We cannot assume that, in a given pixel for two consecutive but independent observation intervals of length T , the same number of photons will be counted. Photon production is governed by the laws of quantum physics which restrict us to talking about an average number of photons within a given observation window. The probability distribution for p photons in an observation window of length T seconds is known to be Poisson:

$$P(p | \rho, T) = \frac{(\rho T)^p e^{-\rho T}}{p!} \quad (62)$$

where ρ is the rate or intensity parameter measured in photons per second. It is critical to understand that even if there were no other noise sources in the imaging chain, the statistical fluctuations associated with photon counting over a finite time interval T would still lead to a finite signal-to-noise ratio (SNR). If we use the appropriate formula for the SNR (eq. (41)), then due to the fact that the average value and the standard deviation are given by:

$$\begin{aligned} \text{Poisson process} - \quad & \text{average} = \rho T \\ & \sigma = \sqrt{\rho T} \end{aligned} \quad (63)$$

we have for the SNR :

$$\text{Photon noise} - \quad SNR = 10 \log_{10}(\rho T) \text{ dB} \quad (64)$$

The three traditional assumptions about the relationship between signal and noise do not hold for photon noise:

- photon noise is not independent of the signal;
- photon noise is not Gaussian, and;
- photon noise is not additive.

For very bright signals, where ρT exceeds 10^5 , the noise fluctuations due to photon statistics can be ignored if the sensor has a sufficiently high saturation level. This will be discussed further in Section 7.3 and, in particular, eq. (73).

6.2 THERMAL NOISE

An additional, stochastic source of electrons in a CCD well is thermal energy. Electrons can be freed from the CCD material itself through thermal vibration and then, trapped in the CCD well, be indistinguishable from “true” photoelectrons. By cooling the CCD chip it is possible to reduce significantly the number of “thermal electrons” that give rise to thermal noise or *dark current*. As the

integration time T increases, the number of thermal electrons increases. The probability distribution of thermal electrons is also a Poisson process where the rate parameter is an increasing function of temperature. There are alternative techniques (to cooling) for suppressing dark current and these usually involve estimating the *average* dark current for the given integration time and then subtracting this value from the CCD pixel values before the A/D converter. While this does reduce the dark current *average*, it does not reduce the dark current *standard deviation* and it also reduces the possible dynamic range of the signal.

6.3 ON-CHIP ELECTRONIC NOISE

This noise originates in the process of reading the signal from the sensor, in this case through the field effect transistor (FET) of a CCD chip. The general form of the power spectral density of readout noise is:

$$\text{Readout noise} - S_{nn}(\omega) \propto \begin{cases} \omega^{-\beta} & \omega < \omega_{\min} \quad \beta > 0 \\ k & \omega_{\min} < \omega < \omega_{\max} \\ \omega^{\alpha} & \omega > \omega_{\max} \quad \alpha > 0 \end{cases} \quad (65)$$

where α and β are constants and ω is the (radial) frequency at which the signal is transferred from the CCD chip to the “outside world.” At very low readout rates ($\omega < \omega_{\min}$) the noise has a $1/f$ character. Readout noise can be reduced to manageable levels by appropriate readout rates and proper electronics. At very low signal levels (see eq. (64)), however, readout noise can still become a significant component in the overall *SNR* [22].

6.4 KTC NOISE

Noise associated with the gate capacitor of an FET is termed *KTC noise* and can be non-negligible. The output RMS value of this noise voltage is given by:

$$\text{KTC noise (voltage)} - \sigma_{KTC} = \sqrt{\frac{kT}{C}} \quad (66)$$

where C is the FET gate switch capacitance, k is Boltzmann’s constant, and T is the absolute temperature of the CCD chip measured in K. Using the relationships $Q = C \cdot V = N_{e^-} \cdot e^-$, the output RMS value of the KTC noise expressed in terms of the number of photoelectrons (N_{e^-}) is given by:

$$\text{KTC noise (electrons)} - \sigma_{N_e} = \frac{\sqrt{kTC}}{e^-} \quad (67)$$

where e^- is the electron charge. For $C = 0.5$ pF and $T = 233$ K this gives $N_{e^-} = 252$ electrons. This value is a “one time” noise per pixel that occurs during signal readout and is thus independent of the integration time (see Sections 6.1 and 7.7). Proper electronic design that makes use, for example, of correlated double sampling and dual-slope integration can almost completely eliminate KTC noise [22].

6.5 AMPLIFIER NOISE

The standard model for this type of noise is additive, Gaussian, and independent of the signal. In modern well-designed electronics, amplifier noise is generally negligible. The most common exception to this is in color cameras where more amplification is used in the blue color channel than in the green channel or red channel leading to more noise in the blue channel. (See also Section 7.6.)

6.6 QUANTIZATION NOISE

Quantization noise is inherent in the amplitude quantization process and occurs in the analog-to-digital converter, ADC. The noise is additive and independent of the signal when the number of levels $L \geq 16$. This is equivalent to $B \geq 4$ bits. (See Section 2.1.) For a signal that has been converted to electrical form and thus has a minimum and maximum electrical value, eq. (40) is the appropriate formula for determining the SNR . If the ADC is adjusted so that 0 corresponds to the minimum electrical value and $2^B - 1$ corresponds to the maximum electrical value then:

$$\text{Quantization noise} - \quad SNR = 6B + 11 \text{ dB} \quad (68)$$

For $B \geq 8$ bits, this means a $SNR \geq 59$ dB. Quantization noise can usually be ignored as the total SNR of a complete system is typically dominated by the smallest SNR . In CCD cameras this is photon noise.

7. Cameras

The cameras and recording media available for modern digital image processing applications are changing at a significant pace. To dwell too long in this section on one major type of camera, such as the CCD camera, and to ignore developments in areas such as charge injection device (CID) cameras and CMOS cameras is to run the risk of obsolescence. Nevertheless, the techniques that are used to characterize the CCD camera remain “universal” and the presentation that

follows is given in the context of modern CCD technology for purposes of illustration.

7.1 LINEARITY

It is generally desirable that the relationship between the input physical signal (e.g. photons) and the output signal (e.g. voltage) be linear. Formally this means (as in eq. (20)) that if we have two images, a and b , and two arbitrary complex constants, w_1 and w_2 and a linear camera response, then:

$$c = \mathcal{R}\{w_1 a + w_2 b\} = w_1 \mathcal{R}\{a\} + w_2 \mathcal{R}\{b\} \quad (69)$$

where $\mathcal{R}\{\cdot\}$ is the camera response and c is the camera output. In practice the relationship between input a and output c is frequently given by:

$$c = gain \cdot a^\gamma + offset \quad (70)$$

where γ is the *gamma* of the recording medium. For a truly linear recording system we must have $\gamma = 1$ and *offset* = 0. Unfortunately, the offset is almost never zero and thus we must compensate for this if the intention is to extract intensity measurements. Compensation techniques are discussed in Section 10.1.

Typical values of γ that may be encountered are listed in Table 8. Modern cameras often have the ability to switch electronically between various values of γ .

<i>Sensor</i>	<i>Surface</i>	γ	<i>Possible advantages</i>
CCD chip	Silicon	1.0	Linear
Vidicon Tube	Sb ₂ S ₃	0.6	Compresses dynamic range → high contrast scenes
Film	Silver halide	< 1.0	Compresses dynamic range → high contrast scenes
Film	Silver halide	> 1.0	Expands dynamic range → low contrast scenes

Table 8: Comparison of γ of various sensors

7.2 SENSITIVITY

There are two ways to describe the sensitivity of a camera. First, we can determine the minimum number of detectable photoelectrons. This can be termed the *absolute* sensitivity. Second, we can describe the number of photoelectrons necessary to change from one digital brightness level to the next, that is, to change one *analog-to-digital unit* (ADU). This can be termed the *relative* sensitivity.

7.2.1 Absolute sensitivity

To determine the absolute sensitivity we need a characterization of the camera in terms of its noise. If the total noise has a σ of, say, 100 photoelectrons, then to ensure detectability of a signal we could then say that, at the 3σ level, the minimum detectable signal (or absolute sensitivity) would be 300 photoelectrons. If all the noise sources listed in Section 6, with the exception of photon noise, can be reduced to negligible levels, this means that an absolute sensitivity of less than 10 photoelectrons is achievable with modern technology

7.2.2 Relative sensitivity

The definition of relative sensitivity, S , given above when coupled to the linear case, eq. (70) with $\gamma = 1$, leads immediately to the result:

$$S = 1/\text{gain} = \text{gain}^{-1} \quad (71)$$

The measurement of the *sensitivity* or *gain* can be performed in two distinct ways.

- If, following eq. (70), the input signal a can be precisely controlled by either “shutter” time or intensity (through neutral density filters), then the gain can be estimated by estimating the slope of the resulting straight-line curve. To translate this into the desired units, however, a standard source must be used that emits a known number of photons onto the camera sensor and the quantum efficiency (η) of the sensor must be known. The quantum efficiency refers to how many photoelectrons are produced—on the average—per photon at a given wavelength. In general $0 \leq \eta(\lambda) \leq 1$.

- If, however, the limiting effect of the camera is only the photon (Poisson) noise (see Section 6.1), then an easy-to-implement, alternative technique is available to determine the sensitivity. Using equations (63), (70), and (71) and after compensating for the *offset* (see Section 10.1), the *sensitivity* measured from an image c is given by:

$$S = \frac{E\{c\}}{\text{Var}\{c\}} = \frac{m_c}{s_c^2} \quad (72)$$

where m_c and s_c are defined in equations (34) and (36).

Measured data for five modern (1995) CCD camera configurations are given in Table 9.

Camera <i>Label</i>	Pixels	Pixel size $\mu m \times \mu m$	Temp. <i>K</i>	S e^- / ADU	Bits
C-1	1320 x 1035	6.8 x 6.8	231	7.9	12
C-2	578 x 385	22.0 x 22.0	227	9.7	16
C-3	1320 x 1035	6.8 x 6.8	293	48.1	10
C-4	576 x 384	23.0 x 23.0	238	90.9	12
C-5	756 x 581	11.0 x 5.5	300	109.2	8

Table 9: Sensitivity measurements. Note that a more sensitive camera has a lower value of S.

The extraordinary sensitivity of modern CCD cameras is clear from these data. In a scientific-grade CCD camera (C-1), only 8 photoelectrons (approximately 16 photons) separate two gray levels in the digital representation of the image. For a considerably less expensive video camera (C-5), only about 110 photoelectrons (approximately 220 photons) separate two gray levels.

7.3 SNR

As described in Section 6, in modern camera systems the noise is frequently limited by:

- amplifier noise in the case of color cameras;
- thermal noise which, itself, is limited by the chip temperature K and the exposure time T , and/or;
- photon noise which is limited by the photon production rate ρ and the exposure time T .

7.3.1 Thermal noise (Dark current)

Using cooling techniques based upon Peltier cooling elements it is straightforward to achieve chip temperatures of 230 to 250 K. This leads to low thermal electron production rates. As a measure of the thermal noise, we can look at the number of seconds necessary to produce a sufficient number of thermal electrons to go from one brightness level to the next, an ADU, in the absence of photoelectrons. This last condition—the absence of photoelectrons—is the reason for the name *dark current*. Measured data for the five cameras described above are given in Table 10.

Camera <i>Label</i>	Temp. <i>K</i>	Dark Current <i>Seconds / ADU</i>
C-1	231	526.3
C-2	227	0.2
C-3	293	8.3
C-4	238	2.4
C-5	300	23.3

Table 10: Thermal noise characteristics

The *video* camera (C–5) has on-chip dark current suppression. (See Section 6.2.) Operating at room temperature this camera requires more than 20 seconds to produce one ADU change due to thermal noise. This means at the conventional video frame and integration rates of 25 to 30 images per second (see Table 3), the thermal noise is negligible.

7.3.2 Photon noise

From eq. (64) we see that it should be possible to increase the *SNR* by increasing the integration time of our image and thus “capturing” more photons. The pixels in CCD cameras have, however, a finite well capacity. This finite capacity, *C*, means that the maximum *SNR* for a CCD camera per pixel is given by:

$$\text{Capacity-limited photon noise} - \quad SNR = 10 \log_{10}(C) \text{ dB} \quad (73)$$

Theoretical as well as measured data for the five cameras described above are given in Table 11.

Camera Label	C # e^-	Theor. SNR dB	Meas. SNR dB	Pixel size $\mu m \times \mu m$	Well Depth # $e^- / \mu m^2$
C–1	32,000	45	45	6.8 x 6.8	692
C–2	340,000	55	55	22.0 x 22.0	702
C–3	32,000	45	43	6.8 x 6.8	692
C–4	400,000	56	52	23.0 x 23.0	756
C–5	40,000	46	43	11.0 x 5.5	661

Table 11: Photon noise characteristics

Note that for certain cameras, the measured *SNR* achieves the theoretical, maximum indicating that the *SNR* is, indeed, photon and well capacity limited. Further, the curves of *SNR* versus *T* (integration time) are consistent with equations (64) and (73). (Data not shown.) It can also be seen that, as a consequence of CCD technology, the “depth” of a CCD pixel well is constant at about $0.7 \text{ ke}^- / \mu m^2$.

7.4 SHADING

Virtually all imaging systems produce shading. By this we mean that if the physical input image $a(x,y) = \text{constant}$, then the digital version of the image will not be constant. The source of the shading might be outside the camera such as in the scene illumination or the result of the camera itself where a *gain* and *offset* might vary from pixel to pixel. The model for shading is given by:

$$c[m,n] = \text{gain}[m,n] \cdot a[m,n] + \text{offset}[m,n] \quad (74)$$

where $a[m,n]$ is the digital image that would have been recorded if there were no shading in the image, that is, $a[m,n] = \text{constant}$. Techniques for reducing or removing the effects of shading are discussed in Section 10.1.

7.5 PIXEL FORM

While the pixels shown in Figure 1 appear to be square and to “cover” the continuous image, it is important to know the geometry for a given camera/digitizer system. In Figure 18 we define possible parameters associated with a camera and digitizer and the effect they have upon the pixel.

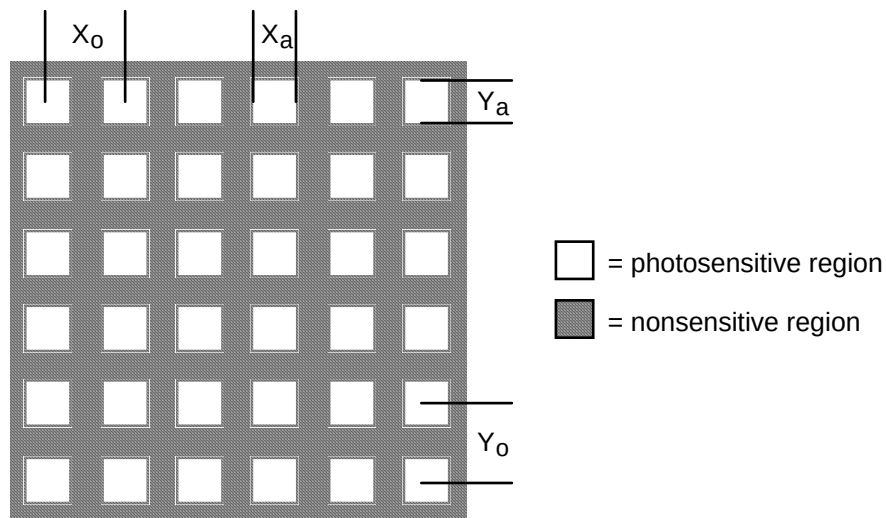


Figure 18: Pixel form parameters

The parameters X_o and Y_o are the spacing between the pixel centers and represent the sampling distances from equation (52). The parameters X_a and Y_a are the dimensions of that portion of the camera’s surface that is sensitive to light. As mentioned in Section 2.3, different video digitizers (frame grabbers) can have different values for X_o while they have a common value for Y_o .

7.5.1 Square pixels

As mentioned in Section 5, square sampling implies that $X_o = Y_o$ or alternatively $X_o / Y_o = 1$. It is not uncommon, however, to find frame grabbers where $X_o / Y_o = 1.1$ or $X_o / Y_o = 4/3$. (This latter format matches the format of commercial television. See Table 3) The risk associated with non-square pixels is that isotropic objects scanned with non-square pixels might appear isotropic on a camera-compatible monitor but analysis of the objects (such as length-to-width ratio) will yield non-isotropic results. This is illustrated in Figure 19.

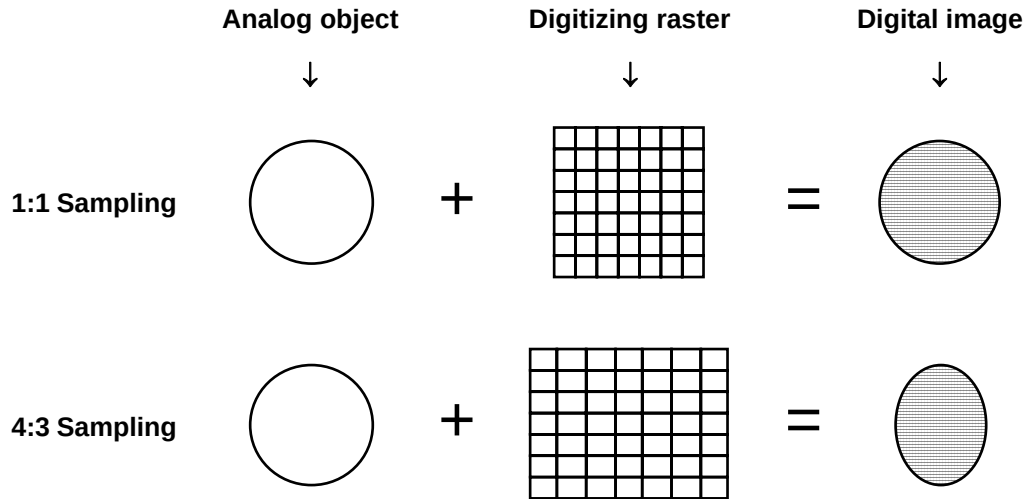


Figure 19: Effect of non-square pixels

The ratio X_o / Y_o can be determined for any specific camera/digitizer system by using a calibration test chart with known distances in the horizontal and vertical direction. These are straightforward to make with modern laser printers. The test chart can then be scanned and the sampling distances X_o and Y_o determined.

7.5.2 Fill factor

In modern CCD cameras it is possible that a portion of the camera surface is not sensitive to light and is instead used for the CCD electronics or to prevent *blooming*. Blooming occurs when a CCD well is filled (see Table 11) and additional photoelectrons spill over into adjacent CCD wells. Anti-blooming regions between the active CCD sites can be used to prevent this. This means, of course, that a fraction of the incoming photons are lost as they strike the non-sensitive portion of the CCD chip. The fraction of the surface that is sensitive to light is termed the *fill factor* and is given by:

$$fill\ factor = \frac{X_a \cdot Y_a}{X_o \cdot Y_o} \times 100\% \quad (75)$$

The larger the *fill factor* the more light will be captured by the chip up to the maximum of 100%. This helps improve the *SNR*. As a tradeoff, however, larger values of the fill factor mean more spatial smoothing due to the aperture effect described in Section 5.1.1. This is illustrated in Figure 16.

7.6 SPECTRAL SENSITIVITY

Sensors, such as those found in cameras and film, are not equally sensitive to all wavelengths of light. The spectral sensitivity for the CCD sensor is given in Figure 20.

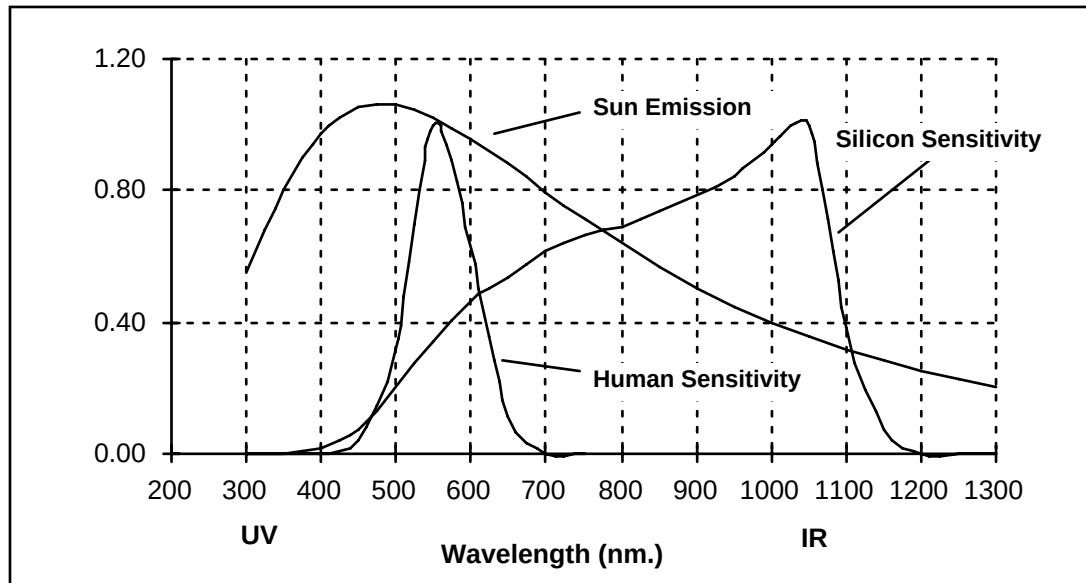


Figure 20: Spectral characteristics of silicon, the sun, and the human visual system. UV = ultraviolet and IR = infra-red.

The high sensitivity of silicon in the infra-red means that, for applications where a CCD (or other silicon-based) camera is to be used as a source of images for digital image processing and analysis, consideration should be given to using an IR blocking filter. This filter blocks wavelengths above 750 nm. and thus prevents “fogging” of the image from the longer wavelengths found in sunlight. Alternatively, a CCD-based camera can make an excellent sensor for the near infrared wavelength range of 750 nm to 1000 nm.

7.7 SHUTTER SPEEDS (INTEGRATION TIME)

The length of time that an image is exposed—that photons are collected—may be varied in some cameras or may vary on the basis of video formats (see Table 3). For reasons that have to do with the parameters of photography, this exposure time is usually termed *shutter speed* although integration time would be a more appropriate description.

7.7.1 Video cameras

Values of the shutter speed as low as 500 ns are available with commercially available CCD *video* cameras although the more conventional speeds for video are 33.37 ms (NTSC) and 40.0 ms (PAL, SECAM). Values as high as 30 s may also be achieved with certain video cameras although this means sacrificing a continuous stream of video images that contain signal in favor of a single integrated image amongst a stream of otherwise empty images. Subsequent digitizing hardware must be capable of handling this situation.

7.7.2 Scientific cameras

Again values as low as 500 ns are possible and, with cooling techniques based on Peltier-cooling or liquid nitrogen cooling, integration times in excess of one hour are readily achieved.

7.8 READOUT RATE

The rate at which data is read from the sensor chip is termed the *readout rate*. The readout rate for standard video cameras depends on the parameters of the frame grabber as well as the camera. For standard video, see Section 2.3, the readout rate is given by:

$$R = \left(\frac{\text{images}}{\text{sec}} \right) \cdot \left(\frac{\text{lines}}{\text{image}} \right) \cdot \left(\frac{\text{pixels}}{\text{line}} \right) \quad (76)$$

While the appropriate unit for describing the readout rate should be *pixels / second*, the term *Hz* is frequently found in the literature and in camera specifications; we shall therefore use the latter unit. For a video camera with square pixels (see Section 7.5), this means:

<i>Format</i>	<i>lines / sec</i>	<i>pixels / line</i>	<i>R (MHz.)</i>
NTSC	15,750	(4/3)*525	≈11.0
PAL / SECAM	15,625	(4/3)*625	≈13.0

Table 12: Video camera readout rates

Note that the values in Table 12 are approximate. Exact values for square-pixel systems require exact knowledge of the way the video digitizer (frame grabber) samples each video line.

The readout rates used in video cameras frequently means that the electronic noise described in Section 6.3 occurs in the region of the noise spectrum (eq. (65)) described by $\omega > \omega_{\max}$ where the noise power increases with increasing frequency. Readout noise can thus be significant in video cameras.

Scientific cameras frequently use a slower readout rate in order to reduce the readout noise. Typical values of readout rate for scientific cameras, such as those described in Tables 9, 10, and 11, are 20 kHz, 500 kHz, and 1 MHz to 8 MHz.

8. Displays

The displays used for image processing—particularly the display systems used with computers—have a number of characteristics that help determine the quality of the final image.

8.1 REFRESH RATE

The *refresh rate* is defined as the number of complete images that are written to the screen per second. For standard video the refresh rate is fixed at the values given in Table 3, either 29.97 or 25 images/s. For computer displays the refresh rate can vary with common values being 67 images/s and 75 images/s. At values above 60 images/s visual flicker is negligible at virtually all illumination levels.

8.2 INTERLACING

To prevent the appearance of visual flicker at refresh rates below 60 images/s, the display can be interlaced as described in Section 2.3. Standard interlace for video systems is 2:1. Since interlacing is not necessary at refresh rates above 60 images/s, an interlace of 1:1 is used with such systems. In other words, lines are drawn in an ordinary sequential fashion: 1,2,3,4,...,N.

8.3 RESOLUTION

The pixels stored in computer memory, although they are derived from regions of finite area in the original scene (see Sections 5.1 and 7.5), may be thought of as mathematical points having no physical extent. When displayed, the space between the points must be filled in. This generally happens as a result of the finite spot size of a cathode-ray tube (CRT). The brightness profile of a CRT spot is approximately Gaussian and the number of spots that can be resolved on the display depends on the quality of the system. It is relatively straightforward to obtain display systems with a resolution of 72 spots per inch (28.3 spots per cm.) This number corresponds to standard printing conventions. If printing is not a consideration then higher resolutions, in excess of 30 spots per cm, are attainable.

9. Algorithms

In this Section we will describe operations that are fundamental to digital image processing. These operations can be divided into four categories: operations based on the image histogram, on simple mathematics, on convolution, and on mathematical morphology. Further, these operations can also be described in

terms of their implementation as a point operation, a local operation, or a global operation as described in Section 2.2.1.

9.1 HISTOGRAM-BASED OPERATIONS

An important class of point operations is based upon the manipulation of an image histogram or a *region* histogram. The most important examples are described below.

9.1.1 Contrast stretching

Frequently, an image is scanned in such a way that the resulting brightness values do not make full use of the available dynamic range. This can be easily observed in the histogram of the brightness values shown in Figure 6. By stretching the histogram over the available dynamic range we attempt to correct this situation. If the image is intended to go from brightness 0 to brightness 2^B-1 (see Section 2.1), then one generally maps the 0% value (or *minimum* as defined in Section 3.5.2) to the value 0 and the 100% value (or *maximum*) to the value 2^B-1 . The appropriate transformation is given by:

$$b[m,n] = (2^B - 1) \cdot \frac{a[m,n] - \text{minimum}}{\text{maximum} - \text{minimum}} \quad (77)$$

This formula, however, can be somewhat sensitive to outliers and a less sensitive and more general version is given by:

$$b[m,n] = \begin{cases} 0 & a[m,n] \leq p_{\text{low}} \% \\ (2^B - 1) \cdot \frac{a[m,n] - p_{\text{low}} \%}{p_{\text{high}} \% - p_{\text{low}} \%} & p_{\text{low}} \% < a[m,n] < p_{\text{high}} \% \\ (2^B - 1) & a[m,n] \geq p_{\text{high}} \% \end{cases} \quad (78)$$

In this second version one might choose the 1% and 99% values for $p_{\text{low}}\%$ and $p_{\text{high}}\%$, respectively, instead of the 0% and 100% values represented by eq. (77). It is also possible to apply the contrast-stretching operation on a regional basis using the histogram from a region to determine the appropriate limits for the algorithm. Note that in eqs. (77) and (78) it is possible to suppress the term 2^B-1 and simply normalize the brightness range to $0 \leq b[m,n] \leq 1$. This means representing the final pixel brightnesses as reals instead of integers but modern computer speeds and RAM capacities make this quite feasible.

9.1.2 Equalization

When one wishes to compare two or more images on a specific basis, such as texture, it is common to first normalize their histograms to a “standard” histogram. This can be especially useful when the images have been acquired under different circumstances. The most common histogram normalization technique is *histogram equalization* where one attempts to change the histogram through the use of a function $b = f(a)$ into a histogram that is constant for all brightness values. This would correspond to a brightness distribution where all values are equally probable. Unfortunately, for an arbitrary image, one can only approximate this result.

For a “suitable” function $f(\bullet)$ the relation between the input probability density function, the output probability density function, and the function $f(\bullet)$ is given by:

$$p_b(b)db = p_a(a)da \quad \Rightarrow \quad df = \frac{p_a(a)da}{p_b(b)} \quad (79)$$

From eq. (79) we see that “suitable” means that $f(\bullet)$ is differentiable and that $df/da \geq 0$. For histogram equalization we desire that $p_b(b) = \text{constant}$ and this means that:

$$f(a) = (2^B - 1) \cdot P(a) \quad (80)$$

where $P(a)$ is the probability *distribution* function defined in Section 3.5.1 and illustrated in Figure 6a. In other words, the *quantized* probability distribution function normalized from 0 to $2^B - 1$ is the look-up table required for histogram equalization. Figures 21a-c illustrate the effect of contrast stretching and histogram equalization on a standard image. The histogram equalization procedure can also be applied on a regional basis.



Figure 21a
Original



Figure 21b
Contrast Stretched



Figure 21c
Histogram Equalized

9.1.3 Other histogram-based operations

The histogram derived from a local region can also be used to drive local filters that are to be applied to that region. Examples include *minimum* filtering, *median* filtering, and *maximum* filtering [23]. The concepts minimum, median, and maximum were introduced in Figure 6. The filters based on these concepts will be presented formally in Sections 9.4.2 and 9.6.10.

9.2 MATHEMATICS-BASED OPERATIONS

We distinguish in this section between binary arithmetic and ordinary arithmetic. In the binary case there are two brightness values “0” and “1”. In the ordinary case we begin with 2^B brightness values or levels but the processing of the image can easily generate many more levels. For this reason many *software* systems provide 16 or 32 bit representations for pixel brightnesses in order to avoid problems with arithmetic overflow.

9.2.1 Binary operations

Operations based on binary (Boolean) arithmetic form the basis for a powerful set of tools that will be described here and extended in Section 9.6, mathematical morphology. The operations described below are point operations and thus admit a variety of efficient implementations including simple look-up tables. The standard notation for the basic set of binary operations is:

$$\begin{aligned}
 NOT & \quad c = \bar{a} \\
 OR & \quad c = a + b \\
 AND & \quad c = a \cdot b \\
 XOR & \quad c = a \oplus b = a \cdot \bar{b} + \bar{a} \cdot b \\
 SUB & \quad c = a \setminus b = a - b = a \cdot \bar{b}
 \end{aligned} \tag{81}$$

The implication is that each operation is applied on a pixel-by-pixel basis. For example, $c[m,n] = a[m,n] \cdot \bar{b}[m,n] \quad \forall m,n$. The definition of each operation is:

NOT	a	
	0	1
	1	0

↑
input

↑
output

OR	a	b
		0 1
	0	0 1
	1	1 1

AND	a	b
		0 1
	0	0 0
	1	0 1

XOR	a	b
		0 1
	0	0 1
	1	1 0

SUB	a	b
		0 1
	0	0 0
	1	1 0

(82)

These operations are illustrated in Figure 22 where the binary value “1” is shown in black and the value “0” in white.

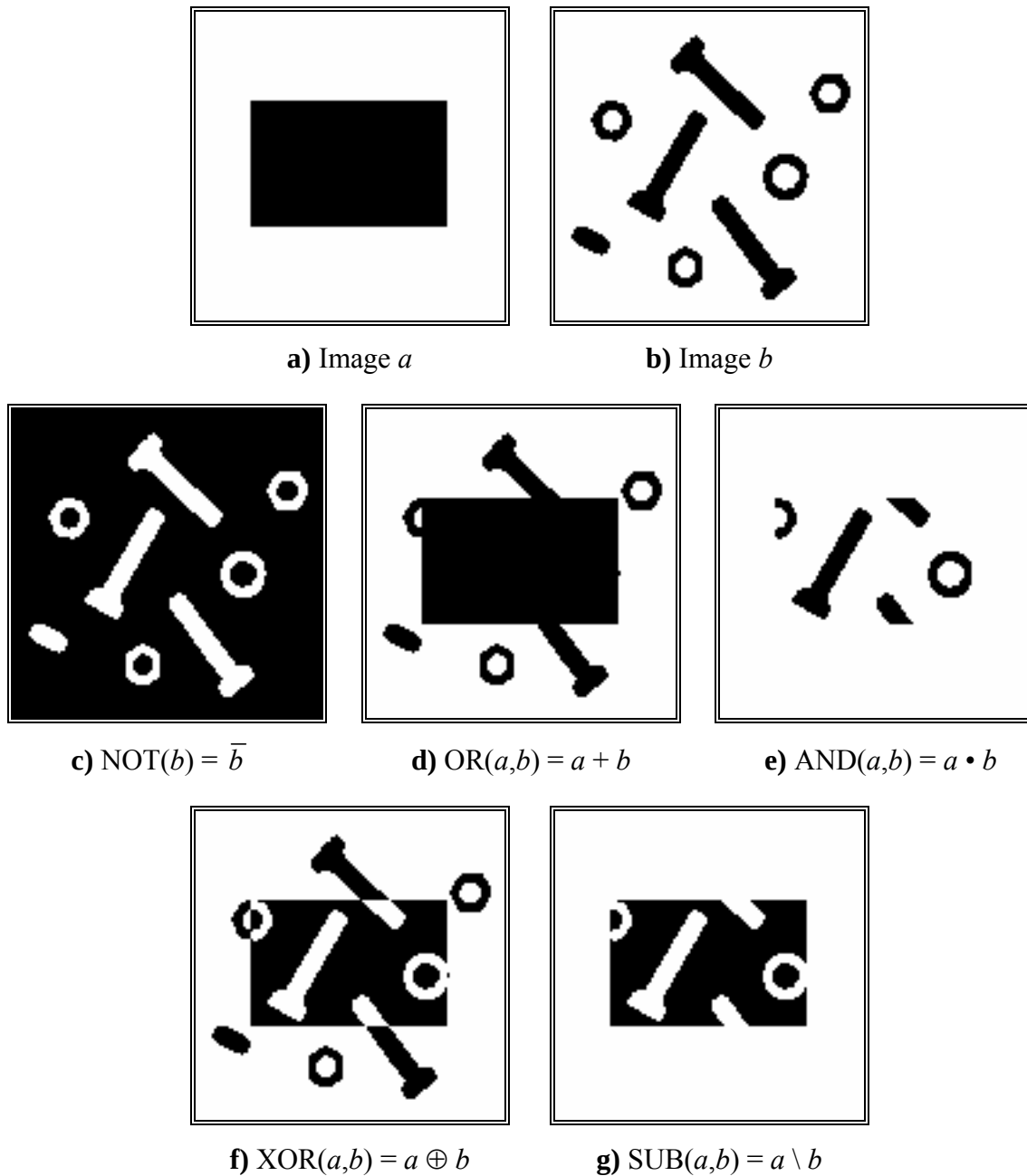


Figure 22: Examples of the various binary point operations.

The SUB($\bullet$) operation can be particularly useful when the image a represents a region-of-interest that we want to analyze systematically and the image b represents objects that, having been analyzed, can now be discarded, that is subtracted, from the region.

9.2.2 Arithmetic-based operations

The gray-value point operations that form the basis for image processing are based on ordinary mathematics and include:

<i>Operation</i>	<i>Definition</i>	<i>preferred data type</i>
ADD	$c = a + b$	integer
SUB	$c = a - b$	integer
MUL	$c = a \cdot b$	integer or floating point
DIV	$c = a / b$	floating point
LOG	$c = \log(a)$	floating point
EXP	$c = \exp(a)$	floating point
SQRT	$c = \text{sqrt}(a)$	floating point
TRIG.	$c = \sin/\cos/\tan(a)$	floating point
INVERT	$c = (2^B - 1) - a$	integer

(83)

9.3 CONVOLUTION-BASED OPERATIONS

Convolution, the mathematical, *local* operation defined in Section 3.1 is central to modern image processing. The basic idea is that a window of some finite size and shape—the *support*—is scanned across the image. The output pixel value is the weighted sum of the input pixels within the window where the weights are the values of the filter assigned to every pixel of the window itself. The window with its weights is called the *convolution kernel*. This leads directly to the following variation on eq. (3). If the filter $h[j,k]$ is zero outside the (odd sized rectangular) window of size $J \times K$ centered around the origin $\{j=-J_0, -J_0+1, \dots, -1, 0, 1, \dots, J_0-1, J_0; k=-K_0, -K_0+1, \dots, -1, 0, 1, \dots, K_0-1, K_0\}$, then, using eq. (4), the convolution can be written as the following finite sum:

$$c[m,n] = a[m,n] \otimes h[m,n] = \sum_{j=-J_0}^{J_0} \sum_{k=-K_0}^{K_0} h[j,k] a[m-j, n-k] \quad (84)$$

This equation can be viewed as more than just a pragmatic mechanism for smoothing or sharpening an image. Further, while eq. (84) illustrates the local character of this operation, eqs. (10) and (24) suggest that the operation can be implemented through the use of the Fourier domain which requires a global operation, the Fourier transform. Both of these aspects will be discussed below.

9.3.1 Background

In a variety of image-forming systems an appropriate model for the transformation of the physical signal $a(x,y)$ into an electronic signal $c(x,y)$ is the convolution of the input signal with the impulse response of the sensor system. This system might consist of both an optical as well as an electrical sub-system. If

each of these systems can be treated as a linear, shift-invariant (*LSI*) system then the convolution model is appropriate. The definitions of these two, possible, system properties are given below:

$$\begin{array}{ll} \text{Linearity} - & \begin{array}{l} \text{If } a_1 \rightarrow c_1 \quad \text{and} \quad a_2 \rightarrow c_2 \\ \text{Then } w_1 \cdot a_1 + w_2 \cdot a_2 \rightarrow w_1 \cdot c_1 + w_2 \cdot c_2 \end{array} \end{array} \quad (85)$$

$$\begin{array}{ll} \text{Shift-Invariance} - & \begin{array}{l} \text{If } a(x, y) \rightarrow c(x, y) \\ \text{Then } a(x - x_o, y - y_o) \rightarrow c(x - x_o, y - y_o) \end{array} \end{array} \quad (86)$$

where w_1 and w_2 are arbitrary complex constants and x_o and y_o are coordinates corresponding to arbitrary spatial translations.

Two remarks are appropriate at this point. First, linearity implies (by choosing $w_1 = w_2 = 0$) that “zero in” gives “zero out”. The offset described in eq. (70) means that such camera signals are not the output of a linear system and thus (strictly speaking) the convolution result is not applicable. Fortunately, it is straightforward to correct for this non-linear effect. (See Section 10.1.)

Second, optical lenses with a magnification, M , other than $1\times$ are not shift invariant; a translation of 1 unit in the input image $a(x,y)$ produces a translation of M units in the output image $c(x,y)$. Due to the Fourier property described in eq. (25) this case can still be handled by linear system theory.

If an impulse point of light $\delta(x,y)$ is imaged through an LSI system then the impulse response of that system is called the *point spread function (PSF)*. The output image then becomes the convolution of the input image with the *PSF*. The Fourier transform of the *PSF* is called the *optical transfer function (OTF)*. For optical systems that are circularly-symmetric, aberration-free, and diffraction-limited the *PSF* is given by the Airy disk shown in Table 4–T.5. The *OTF* of the Airy disk is also presented in Table 4–T.5.

If the convolution window is not the diffraction-limited PSF of the lens but rather the effect of defocusing a lens then an appropriate model for $h(x,y)$ is a pill box of radius a as described in Table 4–T.3. The effect on a test pattern is illustrated in Figure 23.

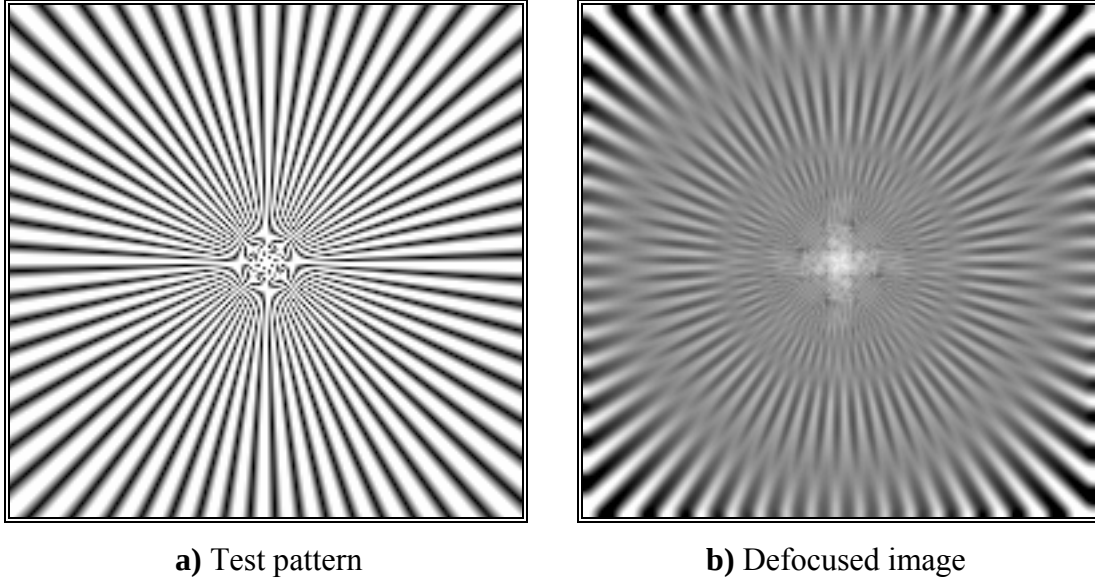


Figure 23: Convolution of test pattern with a pill box of radius $a=4.5$ pixels.

The effect of the defocusing is more than just simple blurring or smoothing. The almost periodic negative lobes in the transfer function in Table 4–T.3 produce a 180° phase shift in which black turns to white and vice-versa. The phase shift is clearly visible in Figure 23b.

9.3.2 Convolution in the spatial domain

In describing filters based on convolution we will use the following convention. Given a filter $h[j,k]$ of dimensions $J \times K = 2J_0+1 \times 2K_0+1$, we will consider the coordinate $[j=0,k=0]$ to be in the center of the filter matrix, $\mathbf{h}$. This is illustrated in Figure 24. The “center” is well-defined when the filter sizes are odd.

$$\mathbf{h} = \begin{bmatrix} h[-J_0, -K_0] & \dots & \dots & h[0, -K_0] & \dots & \dots & h[J_0, -K_0] \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \vdots & \dots & h[-1, -1] & h[0, -1] & h[1, -1] & \dots & \vdots \\ h[-J_0, 0] & \dots & h[-1, 0] & h[0, 0] & h[1, 0] & \dots & h[J_0, 0] \\ \vdots & \dots & h[-1, 1] & h[0, 1] & h[1, +1] & \dots & \vdots \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ h[-J_0, K_0] & \dots & \dots & h[0, K_0] & \dots & \dots & h[J_0, K_0] \end{bmatrix}$$

Figure 24: Coordinate system for describing $h[j,k]$

When we examine the convolution sum (eq. (84)) closely, several issues become evident.

- Evaluation of formula (84) for $m=n=0$ while rewriting the limits of the convolution sum based on the “centering” of $h[j,k]$ shows that values of $a[j,k]$ can be required that are outside the image boundaries:

$$c[0,0] = \sum_{j=-J_0}^{+J_0} \sum_{k=-K_0}^{+K_0} h[j,k]a[-j,-k] \quad (87)$$

The question arises – what values should we assign to the image $a[m,n]$ for $m<0$, $m\geq M$, $n<0$, and $n\geq N$? There is no “answer” to this question. There are only alternatives among which we are free to choose assuming we understand the possible consequences of our choice. The standard alternatives are a) extend the images with a constant (possibly zero) brightness value, b) extend the image periodically, c) extend the image by mirroring it at its boundaries, or d) extend the values at the boundaries indefinitely. These alternatives are illustrated in Figure 25.

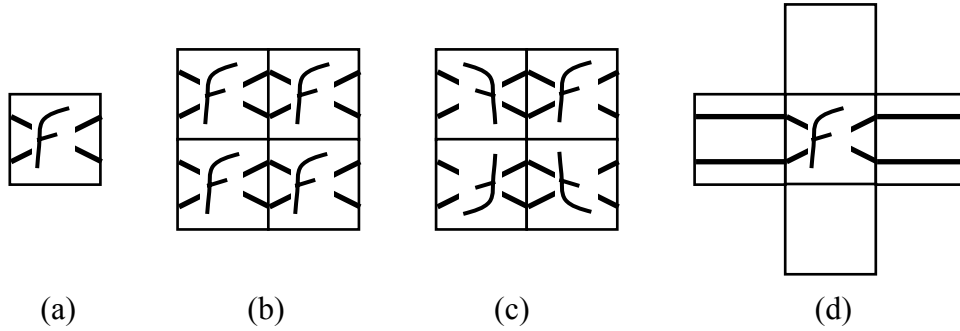


Figure 25: Examples of various alternatives to extend an image outside its formal boundaries. See text for explanation.

- When the convolution sum is written in the standard form (eq. (3)) for an image $a[m,n]$ of size $M \times N$:

$$c[m,n] = \sum_{j=0}^{M-1} \sum_{k=0}^{N-1} a[j,k]h[m-j,n-k] \quad (88)$$

we see that the convolution kernel $h[j,k]$ is mirrored around $j=k=0$ to produce $h[-j,-k]$ before it is translated by $[m,n]$ as indicated in eq. (88). While some convolution kernels in common use are symmetric in this respect, $h[j,k] = h[-j,-k]$, many are not. (See Section 9.5.) Care must therefore be taken in the implementation of filters with respect to the mirroring requirements.

- The computational complexity for a $J \times K$ convolution kernel implemented in the spatial domain on an image of $N \times N$ is $O(J \cdot K)$ where the complexity is measured *per pixel* on the basis of the number of multiplies-and-adds (MADDs).

- The value computed by a convolution that begins with integer brightnesses for $a[m,n]$ may produce a rational number or a floating point number in the result $c[m,n]$. Working exclusively with integer brightness values will, therefore, cause roundoff errors.

- Inspection of eq. (84) reveals another possibility for efficient implementation of convolution. If the convolution kernel $h[j,k]$ is *separable*, that is, if the kernel can be written as:

$$h[j,k] = h_{row}[k] \cdot h_{col}[j] \quad (89)$$

then the filtering can be performed as follows:

$$c[m,n] = \sum_{j=-J_0}^{J_0} \left\{ \sum_{k=-K_0}^{K_0} h_{row}[k] a[m-j, n-k] \right\} h_{col}[j] \quad (90)$$

This means that instead of applying one, two-dimensional filter it is possible to apply two, one-dimensional filters, the first one in the k direction and the second one in the j direction. For an $N \times N$ image this, in general, reduces the computational complexity per pixel from $O(J \cdot K)$ to $O(J+K)$.

An alternative way of writing separability is to note that the convolution kernel (Figure 24) is a matrix $\mathbf{h}$ and, if separable, $\mathbf{h}$ can be written as:

$$\begin{aligned} [\mathbf{h}] &= [\mathbf{h}_{col}] \cdot [\mathbf{h}_{row}]^t \\ (J \times K) &= (J \times 1) \cdot (1 \times K) \end{aligned} \quad (91)$$

where “ t ” denotes the matrix transpose operation. In other words, $\mathbf{h}$ can be expressed as the *outer product* of a column vector $[\mathbf{h}_{col}]$ and a row vector $[\mathbf{h}_{row}]$.

- For certain filters it is possible to find an *incremental implementation* for a convolution. As the convolution window moves over the image (see eq. (88)), the leftmost column of image data under the window is shifted out as a new column of image data is shifted in from the right. Efficient algorithms can take advantage of this [24] and, when combined with separable filters as described above, this can lead to algorithms where the computational complexity per pixel is $O(constant)$.

9.3.3 Convolution in the frequency domain

In Section 3.4 we indicated that there was an alternative method to implement the filtering of images through convolution. Based on eq. (24) it appears possible to achieve the same result as in eq. (84) by the following sequence of operations:

$$\begin{aligned}
 & i) \quad \text{Compute } A(\Omega, \Psi) = \mathcal{F}\{a[m, n]\} \\
 & ii) \quad \text{Multiply } A(\Omega, \Psi) \text{ by the precomputed } H(\Omega, \Psi) = \mathcal{F}\{h[m, n]\} \\
 & iii) \quad \text{Compute the result } c[m, n] = \mathcal{F}^{-1}\{A(\Omega, \Psi) \cdot H(\Omega, \Psi)\}
 \end{aligned} \tag{92}$$

• While it might seem that the “recipe” given above in eq. (92) circumvents the problems associated with direct convolution in the spatial domain—specifically, determining values for the image outside the boundaries of the image—the Fourier domain approach, in fact, simply “assumes” that the image is repeated periodically outside its boundaries as illustrated in Figure 25b. This phenomenon is referred to as *circular convolution*.

If circular convolution is not acceptable then the other possibilities illustrated in Figure 25 can be realized by embedding the image $a[m, n]$ and the filter $H(\Omega, \Psi)$ in larger matrices with the desired image extension mechanism for $a[m, n]$ being explicitly implemented.

• The computational complexity per pixel of the Fourier approach for an image of $N \times N$ and for a convolution kernel of $K \times K$ is $O(2^2 \log N)$ complex MADDs independent of K . Here we assume that $N > K$ and that N is a highly composite number such as a power of two. (See also 2.1.) This latter assumption permits use of the computationally-efficient Fast Fourier Transform (*FFT*) algorithm. Surprisingly then, the indirect route described by eq. (92) can be faster than the direct route given in eq. (84). This requires, in general, that $K^2 \gg 2^2 \log N$. The range of K and N for which this holds depends on the specifics of the implementation. For the machine on which this manuscript is being written and the specific image processing package that is being used, for an image of $N = 256$ the Fourier approach is faster than the convolution approach when $K \geq 15$. (It should be noted that in this comparison the direct convolution involves only integer arithmetic while the Fourier domain approach requires complex floating point arithmetic.)

9.4 SMOOTHING OPERATIONS

These algorithms are applied in order to reduce noise and/or to prepare images for further processing such as segmentation. We distinguish between linear and non-linear algorithms where the former are amenable to analysis in the Fourier domain and the latter are not. We also distinguish between implementations based on a rectangular support for the filter and implementations based on a circular support for the filter.

9.4.1 Linear Filters

Several filtering algorithms will be presented together with the most useful supports.

- *Uniform filter* – The output image is based on a local averaging of the input filter where all of the values within the filter support have the same weight. In the continuous spatial domain (x,y) the *PSF* and transfer function are given in Table 4–T.1 for the rectangular case and in Table 4–T.3 for the circular (pill box) case. For the discrete spatial domain $[m,n]$ the filter values are the samples of the continuous domain case. Examples for the rectangular case ($J=K=5$) and the circular case ($R=2.5$) are shown in Figure 26.

$$h_{rect}[j,k] = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \quad h_{circ}[j,k] = \frac{1}{21} \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

(a) Rectangular filter ($J=K=5$) (b) Circular filter ($R=2.5$)

Figure 26: Uniform filters for image smoothing

Note that in both cases the filter is normalized so that $\sum h[j,k] = 1$. This is done so that if the input $a[m,n]$ is a constant then the output image $c[m,n]$ is the same constant. The justification can be found in the Fourier transform property described in eq. (26). As can be seen from Table 4, both of these filters have transfer functions that have negative lobes and can, therefore, lead to phase reversal as seen in Figure 23. The square implementation of the filter is separable and incremental; the circular implementation is incremental [24, 25].

- *Triangular filter* – The output image is based on a local averaging of the input filter where the values within the filter support have differing weights. In general, the filter can be seen as the convolution of two (identical) uniform filters either rectangular or circular and this has direct consequences for the computational complexity [24, 25]. (See Table 13.) In the continuous spatial domain the *PSF* and transfer function are given in Table 4–T.2 for the rectangular support case and in Table 4–T.4 for the circular (pill box) support case. As seen in Table 4 the transfer functions of these filters do not have negative lobes and thus do not exhibit phase reversal.

Examples for the rectangular support case ($J=K=5$) and the circular support case ($R=2.5$) are shown in Figure 27. The filter is again normalized so that $\sum h[j,k]=1$.

$$h_{rect}[j, k] = \frac{1}{81} \begin{bmatrix} 1 & 2 & 3 & 2 & 1 \\ 2 & 4 & 6 & 4 & 2 \\ 3 & 6 & 9 & 6 & 3 \\ 2 & 4 & 6 & 4 & 2 \\ 1 & 2 & 3 & 2 & 1 \end{bmatrix} \quad h_{circ}[j, k] = \frac{1}{25} \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 2 & 2 & 2 & 0 \\ 1 & 2 & 5 & 2 & 1 \\ 0 & 2 & 2 & 2 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

(a) Pyramidal filter ($J=K=5$) (b) Cone filter ($R=2.5$)

Figure 27: Triangular filters for image smoothing

• *Gaussian filter* – The use of the Gaussian kernel for smoothing has become extremely popular. This has to do with certain properties of the Gaussian (e.g. the central limit theorem, minimum space-bandwidth product) as well as several application areas such as edge finding and scale space analysis. The *PSF* and transfer function for the continuous space Gaussian are given in Table 4–T6. The Gaussian filter is separable:

$$h(x, y) = g_{2D}(x, y) = \left(\frac{1}{\sqrt{2\pi}\sigma} e^{-\left(\frac{x^2}{2\sigma^2}\right)} \right) \cdot \left(\frac{1}{\sqrt{2\pi}\sigma} e^{-\left(\frac{y^2}{2\sigma^2}\right)} \right) \quad (93)$$

$$= g_{1D}(x) \cdot g_{1D}(y)$$

There are four distinct ways to implement the Gaussian:

– Convolution using a finite number of samples (N_o) of the Gaussian as the convolution kernel. It is common to choose $N_o = \lceil 3\sigma \rceil$ or $\lceil 5\sigma \rceil$.

$$g_{1D}[n] = \begin{cases} \frac{1}{\sqrt{2\pi}\sigma} e^{-\left(\frac{n^2}{2\sigma^2}\right)} & |n| \leq N_o \\ 0 & |n| > N_o \end{cases} \quad (94)$$

– Repetitive convolution using a uniform filter as the convolution kernel.

$$g_{1D}[n] \approx u[n] \otimes u[n] \otimes u[n]$$

$$u[n] = \begin{cases} \frac{1}{(2N_o + 1)} & |n| \leq N_o \\ 0 & |n| > N_o \end{cases} \quad (95)$$

The actual implementation (in each dimension) is usually of the form:

$$c[n] = ((a[n] \otimes u[n]) \otimes u[n]) \otimes u[n] \quad (96)$$

This implementation makes use of the approximation afforded by the central limit theorem. For a desired σ with eq. (96), we use $N_o = \lceil \sigma \rceil$ although this severely restricts our choice of σ 's to integer values.

– Multiplication in the frequency domain. As the Fourier transform of a Gaussian is a Gaussian (see Table –T.6), this means that it is straightforward to prepare a filter $H(\Omega, \Psi) = G_{2D}(\Omega, \Psi)$ for use with eq. (92). To avoid truncation effects in the frequency domain due to the infinite extent of the Gaussian it is important to choose a σ that is sufficiently large. Choosing $\sigma > k/\pi$ where $k = 3$ or 4 will usually be sufficient.

– Use of a recursive filter implementation. A recursive filter has an infinite impulse response and thus an infinite support. The separable Gaussian filter can be implemented [26] by applying the following recipe in each dimension when $\sigma \geq 0.5$.

- i) Choose the σ based on the desired goal of the filtering;
- ii) Determine the parameter q based on eq. (98);
- iii) Use eq. (99) to determine the filter coefficients $\{b_0, b_1, b_2, b_3, B\}$; (97)
- iv) Apply the forward difference equation, eq. (100);
- v) Apply the backward difference equation, eq. (101);

The relation between the desired σ and q is given by:

$$q = \begin{cases} .98711\sigma - 0.96330 & \sigma \geq 2.5 \\ 3.97156 - 4.14554\sqrt{1 - .26891\sigma} & 0.5 \leq \sigma \leq 2.5 \end{cases} \quad (98)$$

The filter coefficients $\{b_0, b_1, b_2, b_3, B\}$ are defined by:

$$\begin{aligned} b_0 &= 1.57825 + (2.44413q) + (1.4281q^2) + (0.422205q^3) \\ b_1 &= (2.44413q) + (2.85619q^2) + (1.26661q^3) \\ b_2 &= -(1.4281q^2) - (1.26661q^3) \\ b_3 &= 0.422205q^3 \\ B &= 1 - (b_1 + b_2 + b_3) / b_0 \end{aligned} \quad (99)$$

The one-dimensional *forward difference equation* takes an input row (or column) $a[n]$ and produces an intermediate output result $w[n]$ given by:

$$w[n] = Ba[n] + (b_1w[n-1] + b_2w[n-2] + b_3w[n-3]) / b_0 \quad (100)$$

The one-dimensional *backward difference equation* takes the intermediate result $w[n]$ and produces the output $c[n]$ given by:

$$c[n] = Bw[n] + (b_1c[n+1] + b_2c[n+2] + b_3c[n+3]) / b_0 \quad (101)$$

The forward equation is applied from $n = 0$ up to $n = N - 1$ while the backward equation is applied from $n = N - 1$ down to $n = 0$.

The relative performance of these various implementation of the Gaussian filter can be described as follows. Using the *root-square error* $\sqrt{\sum_{n=-\infty}^{+\infty} |g[n|\sigma] - h[n]|^2}$ between a true, infinite-extent Gaussian, $g[n|\sigma]$, and an approximated Gaussian, $h[n]$, as a measure of accuracy, the various algorithms described above give the results shown in Figure. 28a. The relative speed of the various algorithms in shown in Figure 28b.

The root-square error measure is extremely conservative and thus all filters, with the exception of “Uniform 3×” for large σ , are sufficiently accurate. The recursive implementation is the fastest independent of σ ; the other implementations can be significantly slower. The FFT implementation, for example, is 3.1 times slower for $N=256$. Further, the FFT requires that N be a highly composite number.

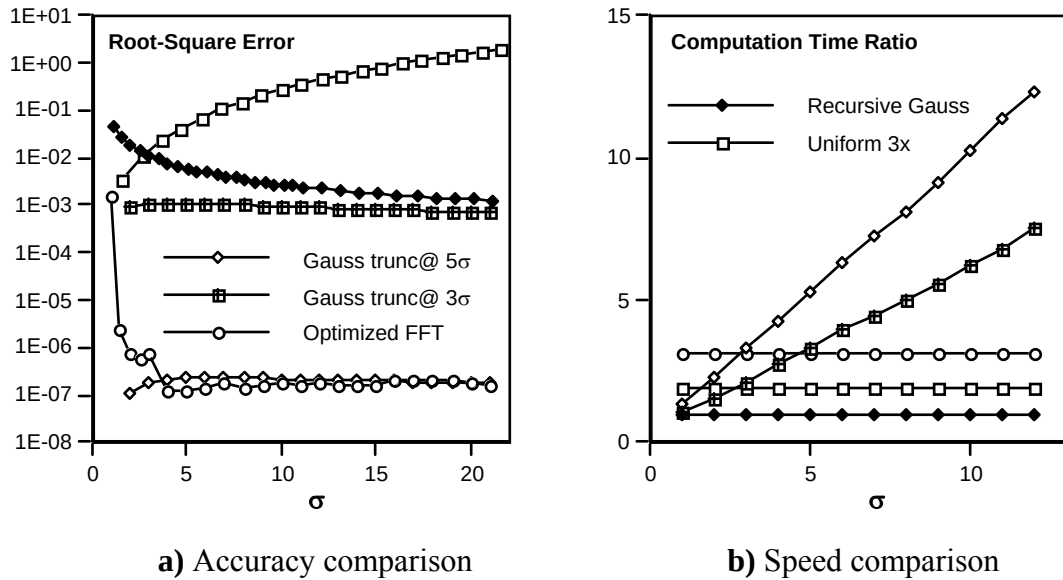


Figure 28: Comparison of various Gaussian algorithms with $N=256$.
The legend is spread across both graphs

- *Other* – The Fourier domain approach offers the opportunity to implement a variety of smoothing algorithms. The smoothing filters will then be *lowpass filters*. In general it is desirable to use a lowpass filter that has zero phase so as not to produce phase distortion when filtering the image. The importance of phase

was illustrated in Figures 5 and 23. When the frequency domain characteristics can be represented in an analytic form, then this can lead to relatively straightforward implementations of $H(\Omega, \Psi)$. Possible candidates include the lowpass filters “Airy” and “Exponential Decay” found in Table 4–T.5 and Table 4–T.8, respectively.

9.4.2 Non-Linear Filters

A variety of smoothing filters have been developed that are not linear. While they cannot, in general, be submitted to Fourier analysis, their properties and domains of application have been studied extensively.

- *Median filter* – The median statistic was described in Section 3.5.2. A median filter is based upon moving a window over an image (as in a convolution) and computing the output pixel as the median value of the brightnesses within the input window. If the window is $J \times K$ in size we can order the $J \cdot K$ pixels in brightness value from smallest to largest. If $J \cdot K$ is odd then the median will be the $(J \cdot K + 1)/2$ entry in the list of ordered brightnesses. Note that the value selected will be exactly equal to one of the existing brightnesses so that no roundoff error will be involved if we want to work exclusively with integer brightness values. The algorithm as it is described above has a generic complexity per pixel of $O(J \cdot K \cdot \log(J \cdot K))$. Fortunately, a fast algorithm (due to Huang et al. [23]) exists that reduces the complexity to $O(K)$ assuming $J \geq K$.

A useful variation on the theme of the median filter is the *percentile filter*. Here the center pixel in the window is replaced not by the 50% (median) brightness value but rather by the $p\%$ brightness value where $p\%$ ranges from 0% (the *minimum filter*) to 100% (the *maximum filter*). Values other than $(p=50)\%$ do not, in general, correspond to smoothing filters.

- *Kuwahara filter* – Edges play an important role in our perception of images (see Figure 15) as well as in the analysis of images. As such it is important to be able to smooth images without disturbing the sharpness and, if possible, the position of edges. A filter that accomplishes this goal is termed an *edge-preserving filter* and one particular example is the Kuwahara filter [27]. Although this filter can be implemented for a variety of different window shapes, the algorithm will be described for a square window of size $J = K = 4L + 1$ where L is an integer. The window is partitioned into four regions as shown in Figure 29.

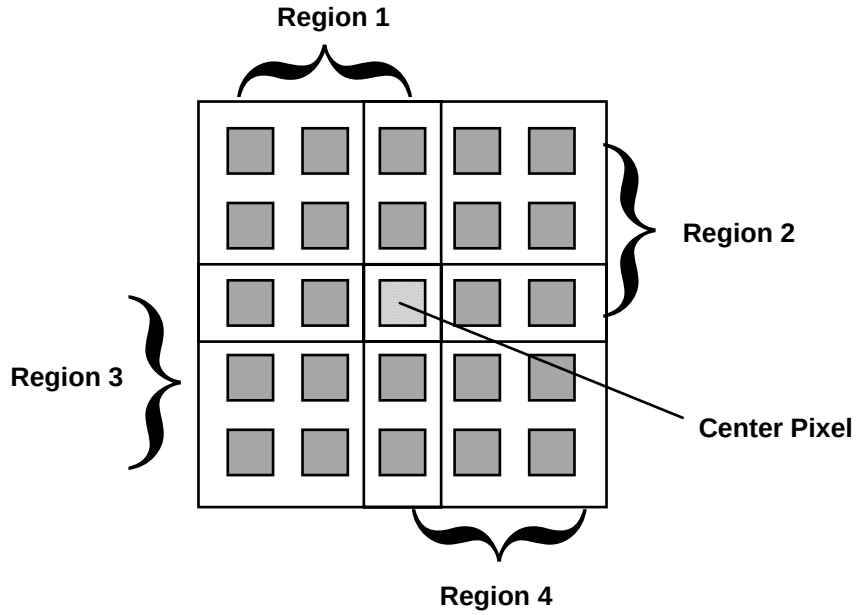


Figure 29: Four, square regions defined for the Kuwahara filter. In this example $L=1$ and thus $J=K=5$. Each region is $[(J+1)/2] \times [(K+1)/2]$.

In each of the four regions ($i=1,2,3,4$), the mean brightness, m_i in eq. (34), and the variance, s_i^2 in eq. (36), are measured. The output value of the center pixel in the window is the mean value of that region that has the smallest variance.

9.4.3 Summary of Smoothing Algorithms

The following table summarizes the various properties of the smoothing algorithms presented above. The filter size is assumed to be bounded by a rectangle of $J \times K$ where, without loss of generality, $J \geq K$. The image size is $N \times N$.

Algorithm	Domain	Type	Support	Separable / Incremental	Complexity/pixel
Uniform	Space	Linear	Square	Y / Y	$O(\text{constant})$
Uniform	Space	Linear	Circular	N / Y	$O(K)$
Triangle	Space	Linear	Square	Y / N	$O(\text{constant})^a$
Triangle	Space	Linear	Circular	N / N	$O(K)^a$
Gaussian	Space	Linear	∞^a	Y / N	$O(\text{constant})^a$
Median	Space	Non-Linear	Square	N / Y	$O(K)^a$
Kuwahara	Space	Non-Linear	Square ^a	N / N	$O(J \cdot K)$
Other	Frequency	Linear	—	— / —	$O(\log N)$

Table 13: Characteristics of smoothing filters. ^aSee text for additional explanation.

Examples of the effect of various smoothing algorithms are shown in Figure 30.

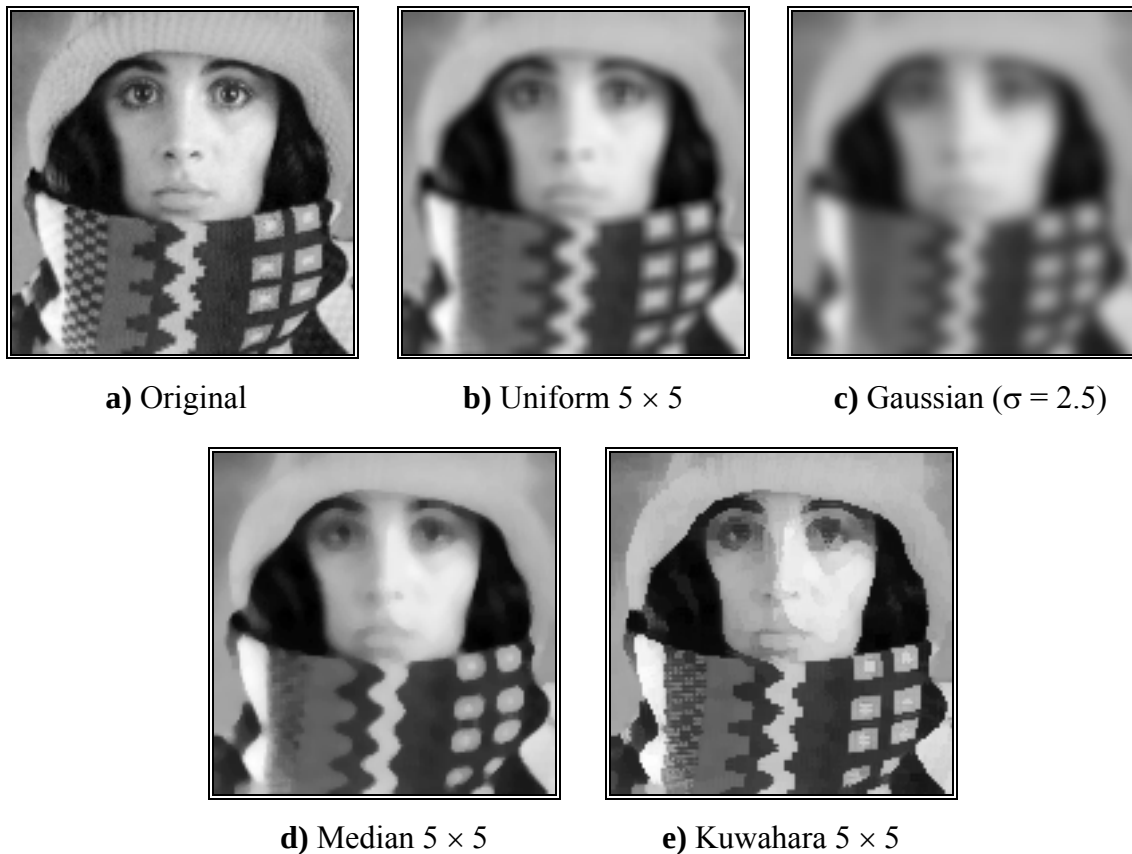


Figure 30: Illustration of various linear and non-linear smoothing filters

9.5 DERIVATIVE-BASED OPERATIONS

Just as smoothing is a fundamental operation in image processing so is the ability to take one or more spatial derivatives of the image. The fundamental problem is that, according to the mathematical definition of a derivative, this cannot be done. A digitized image is not a continuous function $a(x,y)$ of the spatial variables but rather a discrete function $a[m,n]$ of the integer spatial coordinates. As a result the algorithms we will present can only be seen as *approximations* to the true spatial derivatives of the original spatially-continuous image.

Further, as we can see from the Fourier property in eq. (27), taking a derivative multiplies the signal spectrum by either u or v . This means that high frequency noise will be emphasized in the resulting image. The general solution to this problem is to combine the derivative operation with one that suppresses high frequency noise, in short, smoothing in combination with the desired derivative operation.

9.5.1 First Derivatives

As an image is a function of two (or more) variables it is necessary to define the direction in which the derivative is taken. For the two-dimensional case we have

the horizontal direction, the vertical direction, or an arbitrary direction which can be considered as a combination of the two. If we use $\mathbf{h}_x$ to denote a horizontal derivative filter (matrix), $\mathbf{h}_y$ to denote a vertical derivative filter (matrix), and $\mathbf{h}_\theta$ to denote the arbitrary angle derivative filter (matrix), then:

$$[\mathbf{h}_\theta] = \cos \theta \cdot [\mathbf{h}_x] + \sin \theta \cdot [\mathbf{h}_y] \quad (102)$$

• *Gradient filters* – It is also possible to generate a vector derivative description as the *gradient*, $\nabla a[m,n]$, of an image:

$$\nabla a = \frac{\partial a}{\partial x} \vec{i}_x + \frac{\partial a}{\partial y} \vec{i}_y = (h_x \otimes a) \vec{i}_x + (h_y \otimes a) \vec{i}_y \quad (103)$$

where $\vec{i}_x$ and $\vec{i}_y$ are unit vectors in the horizontal and vertical direction, respectively. This leads to two descriptions:

$$\text{Gradient magnitude} - \quad |\nabla a| = \sqrt{(h_x \otimes a)^2 + (h_y \otimes a)^2} \quad (104)$$

and

$$\text{Gradient direction} - \quad \psi(\nabla a) = \arctan \left\{ \frac{(h_y \otimes a)}{(h_x \otimes a)} \right\} \quad (105)$$

The gradient magnitude is sometimes approximated by:

$$\text{Approx. Gradient magnitude} - \quad |\nabla a| \cong |h_x \otimes a| + |h_y \otimes a| \quad (106)$$

The final results of these calculations depend strongly on the choices of $\mathbf{h}_x$ and $\mathbf{h}_y$. A number of possible choices for $(\mathbf{h}_x, \mathbf{h}_y)$ will now be described.

• *Basic derivative filters* – These filters are specified by:

$$\begin{aligned} i) \quad [\mathbf{h}_x] &= [\mathbf{h}_y]^t = \begin{bmatrix} 1 & -1 \end{bmatrix} \\ ii) \quad [\mathbf{h}_x] &= [\mathbf{h}_y]^t = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} \end{aligned} \quad (107)$$

where “ t ” denotes matrix transpose. These two filters differ significantly in their Fourier magnitude and Fourier phase characteristics. For the frequency range $0 \leq \Omega \leq \pi$, these are given by:

$$\begin{aligned}
 i) \quad [\mathbf{h}] &= \begin{bmatrix} 1 & -1 \end{bmatrix} \xleftrightarrow{\mathcal{F}} |H(\Omega)| = 2|\sin(\Omega/2)|; \varphi(\Omega) = (\pi - \Omega)/2 \\
 ii) \quad [\mathbf{h}] &= \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} \xleftrightarrow{\mathcal{F}} |H(\Omega)| = 2|\sin \Omega|; \quad \varphi(\Omega) = \pi/2
 \end{aligned} \tag{108}$$

The second form (ii) gives suppression of high frequency terms ($\Omega \approx \pi$) while the first form (i) does not. The first form leads to a phase shift; the second form does not.

• *Prewitt gradient filters* – These filters are specified by:

$$\begin{aligned}
 [\mathbf{h}_x] &= \frac{1}{3} \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} \\
 [\mathbf{h}_y] &= \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}
 \end{aligned} \tag{109}$$

Both $\mathbf{h}_x$ and $\mathbf{h}_y$ are separable. Beyond the computational implications are the implications for the analysis of the filter. Each filter takes the derivative in one direction using eq. (107)ii and smoothes in the orthogonal direction using a one-dimensional version of a *uniform* filter as described in Section 9.4.1.

• *Sobel gradient filters* – These filters are specified by:

$$\begin{aligned}
 [\mathbf{h}_x] &= \frac{1}{4} \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \frac{1}{4} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} \\
 [\mathbf{h}_y] &= \frac{1}{4} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \frac{1}{4} \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}
 \end{aligned} \tag{110}$$

Again, $\mathbf{h}_x$ and $\mathbf{h}_y$ are separable. Each filter takes the derivative in one direction using eq. (107)ii and smoothes in the orthogonal direction using a one-dimensional version of a *triangular* filter as described in Section 9.4.1.

• *Alternative gradient filters* – The variety of techniques available from one-dimensional signal processing for the design of digital filters offers us powerful tools for designing one-dimensional versions of $\mathbf{h}_x$ and $\mathbf{h}_y$. Using the Parks-

McClellan filter design algorithm, for example, we can choose the frequency bands where we want the derivative to be taken and the frequency bands where we want the noise to be suppressed. The algorithm will then produce a real, odd filter with a minimum length that meets the specifications.

As an example, if we want a filter that has derivative characteristics in a passband (with weight 1.0) in the frequency range $0.0 \leq \Omega \leq 0.3\pi$ and a stopband (with weight 3.0) in the range $0.32\pi \leq \Omega \leq \pi$, then the algorithm produces the following optimized seven sample filter:

$$[\mathbf{h}_x] = [\mathbf{h}_y] = \frac{1}{16348} [-3571 \quad 8212 \quad -15580 \quad 0 \quad 15580 \quad -8212 \quad 3571] \quad (111)$$

The gradient can then be calculated as in eq. (103).

- *Gaussian gradient filters* – In modern digital image processing one of the most common techniques is to use a Gaussian filter (see Section 9.4.1) to accomplish the required smoothing and one of the derivatives listed in eq. (107). Thus, we might first apply the recursive Gaussian in eq. (97) followed by eq. (107)ii to achieve the desired, smoothed derivative filters $\mathbf{h}_x$ and $\mathbf{h}_y$. Further, for computational efficiency, we can combine these two steps as:

$$w[n] = \left(\frac{B}{2} \right) (a[n+1] - a[n-1]) + (b_1 w[n-1] + b_2 w[n-2] + b_3 w[n-3]) / b_0 \quad (112)$$

$$c[n] = Bw[n] + (b_1 c[n+1] + b_2 c[n+2] + b_3 c[n+3]) / b_0$$

where the various coefficients are defined in eq. (99). The first (forward) equation is applied from $n = 0$ up to $n = N - 1$ while the second (backward) equation is applied from $n = N - 1$ down to $n = 0$.

- *Summary* – Examples of the effect of various *derivative algorithms* on a noisy version of Figure 30a ($SNR = 29$ dB) are shown in Figure 31a-c. The effect of various *magnitude gradient algorithms* on Figure 30a are shown in Figure 32a-c. After processing, all images are contrast stretched as in eq. (77) for display purposes.



(a) Simple Derivative – eq. (107)ii (b) Sobel – eq. (110) (c) Gaussian ($\sigma=1.5$) & eq. (107)ii

Figure 31: Application of various algorithms for $\mathbf{h}_x$ – the horizontal derivative.



(a) Simple Derivative – eq. (107)ii (b) Sobel – eq. (110) (c) Gaussian ($\sigma=1.5$) & eq. (107)ii

Figure 32: Various algorithms for the magnitude gradient, $|\nabla a|$.

The magnitude gradient takes on large values where there are strong edges in the image. Appropriate choice of σ in the Gaussian-based derivative (Figure 31c) or gradient (Figure 32c) permits computation of virtually any of the other forms – simple, Prewitt, Sobel, etc. In that sense, the Gaussian derivative represents a superset of derivative filters.

9.5.2 Second Derivatives

It is, of course, possible to compute higher-order derivatives of functions of two variables. In image processing, as we shall see in Sections 10.2.1 and 10.3.2, the second derivatives or Laplacian play an important role. The Laplacian is defined as:

$$\nabla^2 a = \frac{\partial^2 a}{\partial x^2} + \frac{\partial^2 a}{\partial y^2} = (h_{2x} \otimes a) + (h_{2y} \otimes a) \quad (113)$$

where $\mathbf{h}_{2x}$ and $\mathbf{h}_{2y}$ are second derivative filters. In the frequency domain we have for the Laplacian filter (from eq. (27)):

$$\nabla^2 a \xleftrightarrow{\mathcal{F}} -(u^2 + v^2) A(u, v) \quad (114)$$

The transfer function of a Laplacian corresponds to a parabola $H(u, v) = -(u^2 + v^2)$.

• *Basic second derivative filter* – This filter is specified by:

$$[\mathbf{h}_{2x}] = [\mathbf{h}_{2y}]^t = [1 \quad -2 \quad 1] \quad (115)$$

and the frequency spectrum of this filter, in each direction, is given by:

$$H(\Omega) = \mathcal{F}\{1 \quad -2 \quad 1\} = -2(1 - \cos \Omega) \quad (116)$$

over the frequency range $-\pi \leq \Omega \leq \pi$. The two, one-dimensional filters can be used in the manner suggested by eq. (113) or combined into one, two-dimensional filter as:

$$[\mathbf{h}] = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad (117)$$

and used as in eq. (84).

• *Frequency domain Laplacian* – This filter is the implementation of the general recipe given in eq. (92) and for the Laplacian filter takes the form:

$$c[m, n] = \mathcal{F}^{-1} \left\{ -(\Omega^2 + \Psi^2) A(\Omega, \Psi) \right\} \quad (118)$$

• *Gaussian second derivative filter* – This is the straightforward extension of the Gaussian first derivative filter described above and can be applied independently in each dimension. We first apply Gaussian smoothing with a σ chosen on the basis of the problem specification. We then apply the desired second derivative filter eq. (115) or eq. (118). Again there is the choice among the various Gaussian smoothing algorithms.

For efficiency, we can use the recursive implementation and combine the two steps—smoothing and derivative operation—as follows:

$$\begin{aligned} w[n] &= B(a[n] - a[n-1]) + (b_1 w[n-1] + b_2 w[n-2] + b_3 w[n-3]) / b_0 \\ c[n] &= B(w[n+1] - w[n]) + (b_1 c[n+1] + b_2 c[n+2] + b_3 c[n+3]) / b_0 \end{aligned} \quad (119)$$

where the various coefficients are defined in eq. (99). Again, the first (forward) equation is applied from $n = 0$ up to $n = N - 1$ while the second (backward) equation is applied from $n = N - 1$ down to $n = 0$.

- *Alternative Laplacian filters* – Again one-dimensional digital filter design techniques offer us powerful methods to create filters that are optimized for a specific problem. Using the Parks-McClellan design algorithm, we can choose the frequency bands where we want the second derivative to be taken and the frequency bands where we want the noise to be suppressed. The algorithm will then produce a real, even filter with a minimum length that meets the specifications.

As an example, if we want a filter that has second derivative characteristics in a passband (with weight 1.0) in the frequency range $0.0 \leq \Omega \leq 0.3\pi$ and a stopband (with weight 3.0) in the range $0.32\pi \leq \Omega \leq \pi$, then the algorithm produces the following optimized seven sample filter:

$$[\mathbf{h}_x] = [\mathbf{h}_y] = \frac{1}{11043} \begin{bmatrix} -3448 & 10145 & 1495 & -16383 & 1495 & 10145 & -3448 \end{bmatrix} \quad (120)$$

The Laplacian can then be calculated as in eq. (113).

- *SDGD filter* – A filter that is especially useful in edge finding and object measurement is the *Second-Derivative-in-the-Gradient-Direction (SDGD)* filter. This filter uses five partial derivatives:

$$\begin{aligned} A_{xx} &= \frac{\partial^2 a}{\partial x^2} & A_{xy} &= \frac{\partial^2 a}{\partial x \partial y} & A_x &= \frac{\partial a}{\partial x} \\ A_{yx} &= \frac{\partial^2 a}{\partial x \partial y} & A_{yy} &= \frac{\partial^2 a}{\partial y^2} & A_y &= \frac{\partial a}{\partial y} \end{aligned} \quad (121)$$

Note that $A_{xy} = A_{yx}$ which accounts for the five derivatives.

This *SDGD* combines the different partial derivatives as follows:

$$SDGD(a) = \frac{A_{xx}A_x^2 + 2A_{xy}A_xA_y + A_{yy}A_y^2}{A_x^2 + A_y^2} \quad (122)$$

As one might expect, the large number of derivatives involved in this filter implies that noise suppression is important and that Gaussian derivative filters—

both first and second order—are highly recommended if not required [28]. It is also necessary that the first and second derivative filters have essentially the same passbands and stopbands. This means that if the first derivative filter h_{1x} is given by $[1 \ 0 \ -1]$ (eq. (107)*ii*) then the second derivative filter should be given by $h_{1x} \otimes h_{1x} = h_{2x} = [1 \ 0 \ -2 \ 0 \ 1]$.

• *Summary* – The effects of the various second derivative filters are illustrated in Figure 33a-e. All images were contrast stretched for display purposes using eq. (78) and the parameters 1% and 99%.

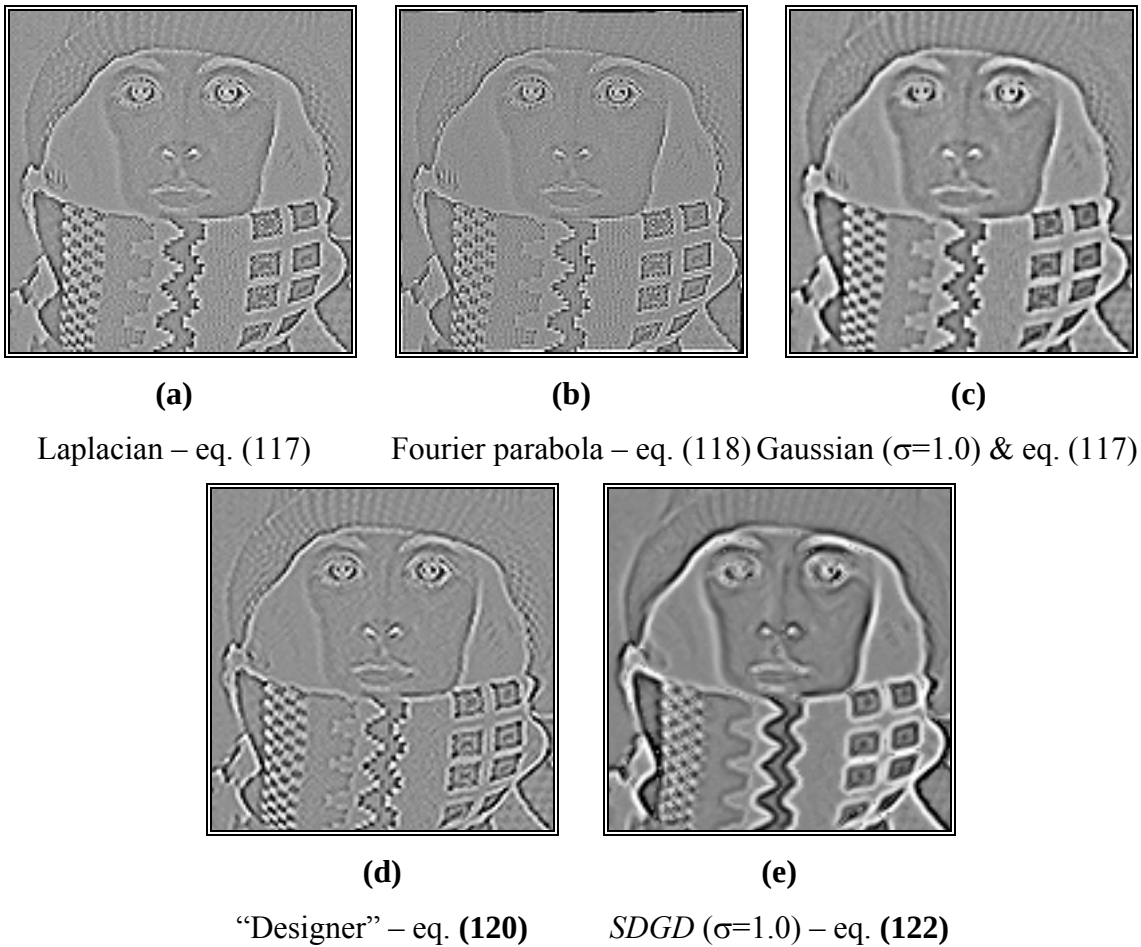


Figure 33: Various algorithms for the Laplacian and Laplacian-related filters.

9.5.3 Other Filters

An infinite number of filters, both linear and non-linear, are possible for image processing. It is therefore impossible to describe more than the basic types in this section. The description of others can be found in the reference literature (see Section 11) as well as in the applications literature. It is important to use a small consistent set of test images that are relevant to the application area to understand the effect of a given filter or class of filters. The effect of filters on images can be frequently understood by the use of images that have pronounced regions of

varying sizes to visualize the effect on edges or by the use of test patterns such as sinusoidal sweeps to visualize the effects in the frequency domain. The former have been used above (Figures 21, 23, and 30–33) and the latter are demonstrated below in Figure 34.

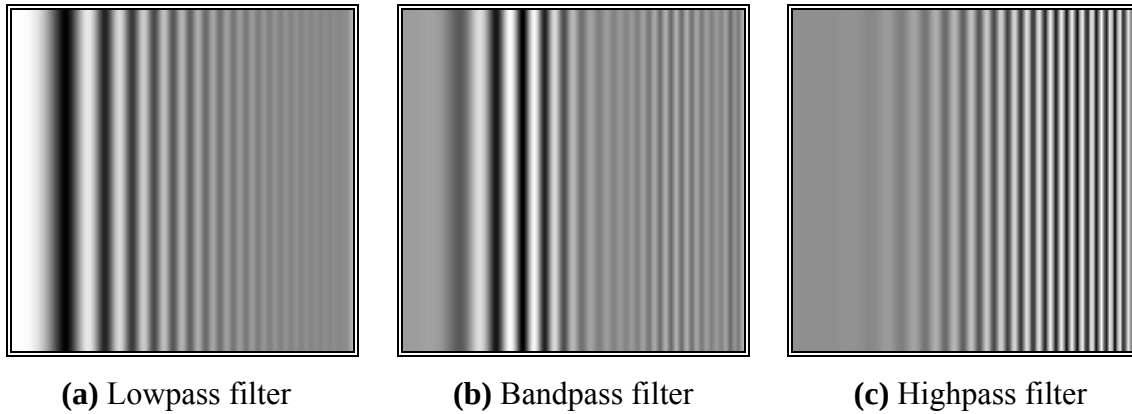


Figure 34: Various convolution algorithms applied to sinusoidal test image.

9.6 MORPHOLOGY-BASED OPERATIONS

In Section 1 we defined an image as an (amplitude) function of two, real (coordinate) variables $a(x,y)$ or two, discrete variables $a[m,n]$. An alternative definition of an image can be based on the notion that an image consists of a set (or collection) of either continuous or discrete coordinates. In a sense the set corresponds to the points or pixels that belong to the objects in the image. This is illustrated in Figure 35 which contains two objects or sets **A** and **B**. Note that the coordinate system is required. For the moment we will consider the pixel values to be binary as discussed in Section 2.1 and 9.2.1. Further we shall restrict our discussion to discrete space (Z^2). More general discussions can be found in [6, 7, 29].

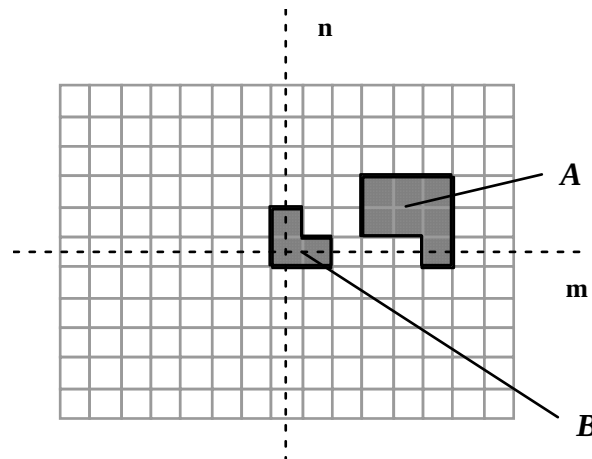


Figure 35: A binary image containing two object sets **A** and **B**.

The object $\mathbf{A}$ consists of those pixels α that share some common property:

$$\text{Object} - \quad \mathbf{A} = \{\alpha \mid \text{property}(\alpha) == \text{TRUE}\} \quad (123)$$

As an example, object $\mathbf{B}$ in Figure 35 consists of $\{[0,0], [1,0], [0,1]\}$.

The background of $\mathbf{A}$ is given by $\mathbf{A}^c$ (the *complement* of $\mathbf{A}$) which is defined as those elements that are not in $\mathbf{A}$:

$$\text{Background} - \quad \mathbf{A}^c = \{\alpha \mid \alpha \notin \mathbf{A}\} \quad (124)$$

In Figure 3 we introduced the concept of neighborhood connectivity. We now observe that if an object $\mathbf{A}$ is defined on the basis of C -connectivity ($C=4, 6$, or 8) then the background $\mathbf{A}^c$ has a connectivity given by $12 - C$. The necessity for this is illustrated for the Cartesian grid in Figure 36.

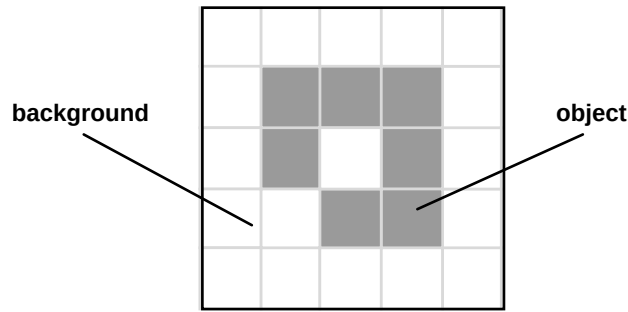


Figure 36: A binary image requiring careful definition of object and background connectivity.

9.6.1 Fundamental definitions

The fundamental operations associated with an object are the standard set operations *union*, *intersection*, and *complement* $\{\cup, \cap, ^c\}$ plus *translation*:

- *Translation* – Given a vector $\mathbf{x}$ and a set $\mathbf{A}$, the *translation*, $\mathbf{A} + \mathbf{x}$, is defined as:

$$\mathbf{A} + \mathbf{x} = \{\alpha + \mathbf{x} \mid \alpha \in \mathbf{A}\} \quad (125)$$

Note that, since we are dealing with a digital image composed of pixels at integer coordinate positions ($\mathbb{Z}^2$), this implies restrictions on the allowable translation vectors $\mathbf{x}$.

The basic *Minkowski set operations*—addition and subtraction—can now be defined. First we note that the individual elements that comprise $\mathbf{B}$ are not only

pixels but also *vectors* as they have a clear coordinate position with respect to $[0,0]$. Given two sets $\mathbf{A}$ and $\mathbf{B}$:

$$\text{Minkowski addition} - \quad \mathbf{A} \oplus \mathbf{B} = \bigcup_{\beta \in \mathbf{B}} (\mathbf{A} + \beta) \quad (126)$$

$$\text{Minkowski subtraction} - \quad \mathbf{A} \ominus \mathbf{B} = \bigcap_{\beta \in \mathbf{B}} (\mathbf{A} + \beta) \quad (127)$$

9.6.2 Dilation and Erosion

From these two Minkowski operations we define the fundamental mathematical morphology operations *dilation* and *erosion*:

$$\text{Dilation} - \quad D(\mathbf{A}, \mathbf{B}) = \mathbf{A} \oplus \mathbf{B} = \bigcup_{\beta \in \mathbf{B}} (\mathbf{A} + \beta) \quad (128)$$

$$\text{Erosion} - \quad E(\mathbf{A}, \mathbf{B}) = \mathbf{A} \ominus \tilde{\mathbf{B}} = \bigcap_{\beta \in \mathbf{B}} (\mathbf{A} - \beta) \quad (129)$$

where $\tilde{\mathbf{B}} = \{-\beta | \beta \in \mathbf{B}\}$. These two operations are illustrated in Figure 37 for the objects defined in Figure 35.

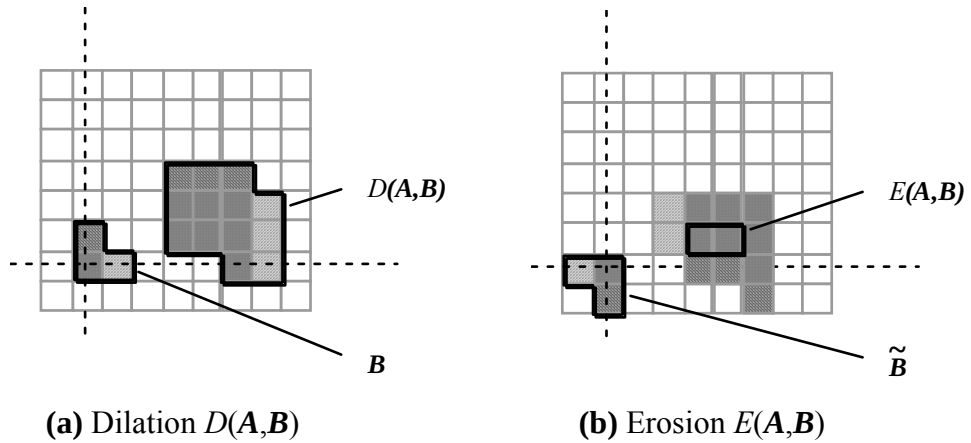


Figure 37: A binary image containing two object sets $\mathbf{A}$ and $\mathbf{B}$. The three pixels in $\mathbf{B}$ are “color-coded” as is their effect in the result.

While either set $\mathbf{A}$ or $\mathbf{B}$ can be thought of as an “image”, $\mathbf{A}$ is usually considered as the image and $\mathbf{B}$ is called a *structuring element*. The *structuring element* is to *mathematical morphology* what the *convolution kernel* is to *linear filter theory*.

Dilation, in general, causes objects to dilate or grow in size; *erosion* causes objects to shrink. The amount and the way that they grow or shrink depend upon the choice of the structuring element. Dilating or eroding without specifying the

structural element makes no more sense than trying to lowpass filter an image without specifying the filter. The two most common structuring elements (given a Cartesian grid) are the 4-connected and 8-connected sets, N_4 and N_8 . They are illustrated in Figure 38.

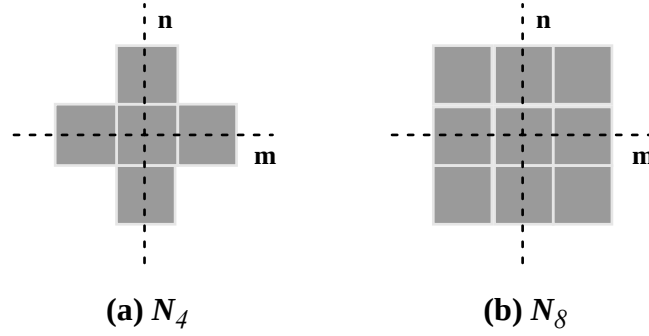


Figure 38: The standard structuring elements N_4 and N_8 .

Dilation and *erosion* have the following properties:

$$\text{Commutative} - \quad D(\mathbf{A}, \mathbf{B}) = \mathbf{A} \oplus \mathbf{B} = \mathbf{B} \oplus \mathbf{A} = D(\mathbf{B}, \mathbf{A}) \quad (130)$$

$$\text{Non-Commutative} - \quad E(\mathbf{A}, \mathbf{B}) \neq E(\mathbf{B}, \mathbf{A}) \quad (131)$$

$$\text{Associative} - \quad \mathbf{A} \oplus (\mathbf{B} \oplus \mathbf{C}) = (\mathbf{A} \oplus \mathbf{B}) \oplus \mathbf{C} \quad (132)$$

$$\text{Translation Invariance} - \quad \mathbf{A} \oplus (\mathbf{B} + \mathbf{x}) = (\mathbf{A} \oplus \mathbf{B}) + \mathbf{x} \quad (133)$$

$$\begin{aligned} \text{Duality} - \quad D^c(\mathbf{A}, \mathbf{B}) &= E(\mathbf{A}^c, \tilde{\mathbf{B}}) \\ E^c(\mathbf{A}, \mathbf{B}) &= D(\mathbf{A}^c, \tilde{\mathbf{B}}) \end{aligned} \quad (134)$$

With $\mathbf{A}$ as an object and $\mathbf{A}^c$ as the background, eq. (134) says that the *dilation* of an object is equivalent to the *erosion* of the background. Likewise, the *erosion* of the object is equivalent to the *dilation* of the background.

Except for special cases:

$$\text{Non-Inverses} - \quad D(E(\mathbf{A}, \mathbf{B}), \mathbf{B}) \neq \mathbf{A} \neq E(D(\mathbf{A}, \mathbf{B}), \mathbf{B}) \quad (135)$$

Erosion has the following translation property:

$$\text{Translation Invariance} - \quad \mathbf{A} \ominus (\mathbf{B} + \mathbf{x}) = (\mathbf{A} + \mathbf{x}) \ominus \mathbf{B} = (\mathbf{A} \ominus \mathbf{B}) + \mathbf{x} \quad (136)$$

Dilation and *erosion* have the following important properties. For any arbitrary structuring element $\mathbf{B}$ and two image objects $\mathbf{A}_1$ and $\mathbf{A}_2$ such that $\mathbf{A}_1 \subset \mathbf{A}_2$ ($\mathbf{A}_1$ is a proper subset of $\mathbf{A}_2$):

$$\begin{aligned} \text{Increasing in } \mathbf{A} - \quad & D(\mathbf{A}_1, \mathbf{B}) \subset D(\mathbf{A}_2, \mathbf{B}) \\ & E(\mathbf{A}_1, \mathbf{B}) \subset E(\mathbf{A}_2, \mathbf{B}) \end{aligned} \quad (137)$$

For two structuring elements $\mathbf{B}_1$ and $\mathbf{B}_2$ such that $\mathbf{B}_1 \subset \mathbf{B}_2$:

$$\text{Decreasing in } \mathbf{B} - \quad E(\mathbf{A}, \mathbf{B}_1) \supset E(\mathbf{A}, \mathbf{B}_2) \quad (138)$$

The *decomposition theorems* below make it possible to find efficient implementations for morphological filters.

$$\text{Dilation} - \quad \mathbf{A} \oplus (\mathbf{B} \cup \mathbf{C}) = (\mathbf{A} \oplus \mathbf{B}) \cup (\mathbf{A} \oplus \mathbf{C}) = (\mathbf{B} \cup \mathbf{C}) \oplus \mathbf{A} \quad (139)$$

$$\text{Erosion} - \quad \mathbf{A} \ominus (\mathbf{B} \cup \mathbf{C}) = (\mathbf{A} \ominus \mathbf{B}) \cap (\mathbf{A} \ominus \mathbf{C}) \quad (140)$$

$$\text{Erosion} - \quad (\mathbf{A} \ominus \mathbf{B}) \ominus \mathbf{C} = \mathbf{A} \ominus (\mathbf{B} \oplus \mathbf{C}) \quad (141)$$

$$\text{Multiple Dilations} - \quad n\mathbf{B} = \underbrace{(\mathbf{B} \oplus \mathbf{B} \oplus \mathbf{B} \oplus \cdots \oplus \mathbf{B})}_{n \text{ times}} \quad (142)$$

An important decomposition theorem is due to Vincent [30]. First, we require some definitions. A *convex* set (in R^2) is one for which the straight line joining any two points in the set consists of points that are also in the set. Care must obviously be taken when applying this definition to discrete pixels as the concept of a “straight line” must be interpreted appropriately in Z^2 . A set is *bounded* if each of its elements has a finite magnitude, in this case distance to the origin of the coordinate system. A set is *symmetric* if $\mathbf{B} = \tilde{\mathbf{B}}$. The sets $\mathbf{N}_4$ and $\mathbf{N}_8$ in Figure 38 are examples of convex, bounded, symmetric sets.

Vincent’s theorem, when applied to an image consisting of discrete pixels, states that for a bounded, symmetric structuring element $\mathbf{B}$ that contains no holes and contains its own center, $[0, 0] \in \mathbf{B}$:

$$D(\mathbf{A}, \mathbf{B}) = \mathbf{A} \oplus \mathbf{B} = \mathbf{A} \cup (\partial \mathbf{A} \oplus \mathbf{B}) \quad (143)$$

where $\partial \mathbf{A}$ is the contour of the object. That is, $\partial \mathbf{A}$ is the set of pixels that have a background pixel as a neighbor. The implication of this theorem is that it is not necessary to process all the pixels in an object in order to compute a *dilation* or (using eq. (134)) an *erosion*. We only have to process the boundary pixels. This

also holds for all operations that can be derived from *dilations* and *erosions*. The processing of boundary pixels instead of object pixels means that, except for pathological images, computational complexity can be reduced from $O(N^2)$ to $O(N)$ for an $N \times N$ image. A number of “fast” algorithms can be found in the literature that are based on this result [30-32]. The simplest dilation and erosion algorithms are frequently described as follows.

- *Dilation* – Take each binary object pixel (with value “1”) and set all background pixels (with value “0”) that are C -connected to that object pixel to the value “1”.
- *Erosion* – Take each binary object pixel (with value “1”) that is C -connected to a background pixel and set the object pixel value to “0”.

Comparison of these two procedures to eq. (143) where $B = N_{C=4}$ or $N_{C=8}$ shows that they are equivalent to the formal definitions for dilation and erosion. The procedure is illustrated for *dilation* in Figure 39.

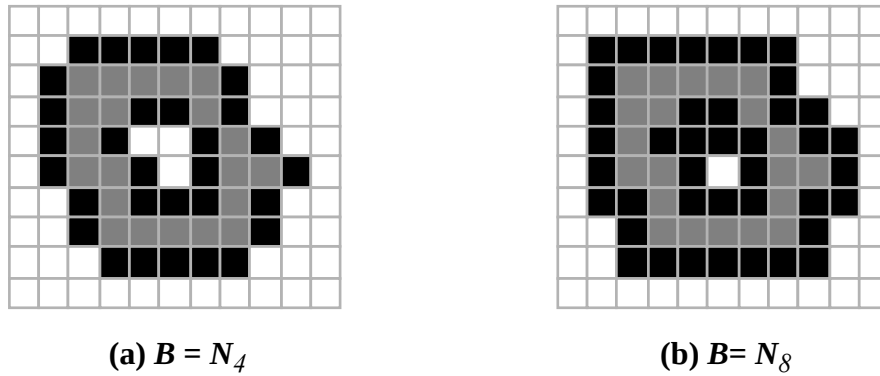


Figure 39: Illustration of *dilation*. Original object pixels are in gray; pixels added through *dilation* are in black.

9.6.3 Boolean Convolution

An arbitrary *binary* image object (or structuring element) A can be represented as:

$$A \leftrightarrow \sum_{k=-\infty}^{+\infty} \sum_{j=-\infty}^{+\infty} a[j,k] \cdot \delta[m-j, n-k] \quad (144)$$

where Σ and $\cdot$ are the Boolean operations *OR* and *AND* as defined in eqs. (81) and (82), $a[j,k]$ is a *characteristic function* that takes on the Boolean values “1” and “0” as follows:

$$a[j,k] = \begin{cases} 1 & a \in A \\ 0 & a \notin A \end{cases} \quad (145)$$

and $\delta[m,n]$ is a Boolean version of the Dirac delta function that takes on the Boolean values “1” and “0” as follows:

$$\delta[j,k] = \begin{cases} 1 & j = k = 0 \\ 0 & \text{otherwise} \end{cases} \quad (146)$$

Dilation for binary images can therefore be written as:

$$D(\mathbf{A}, \mathbf{B}) = \sum_{k=-\infty}^{+\infty} \sum_{j=-\infty}^{+\infty} a[j,k] \bullet b[m-j, n-k] = \mathbf{a} \otimes \mathbf{b} \quad (147)$$

which, because Boolean *OR* and *AND* are commutative, can also be written as

$$D(\mathbf{A}, \mathbf{B}) = \sum_{k=-\infty}^{+\infty} \sum_{j=-\infty}^{+\infty} a[m-j, n-k] \bullet b[j,k] = \mathbf{b} \otimes \mathbf{a} = D(\mathbf{B}, \mathbf{A}) \quad (148)$$

Using De Morgan’s theorem:

$$\overline{(a+b)} = \bar{a} \bullet \bar{b} \quad \text{and} \quad \overline{(a \bullet b)} = \bar{a} + \bar{b} \quad (149)$$

on eq. (148) together with eq. (134), *erosion* can be written as:

$$E(\mathbf{A}, \mathbf{B}) = \prod_{k=-\infty}^{+\infty} \prod_{j=-\infty}^{+\infty} (a[m-j, n-k] + \bar{b}[-j, -k]) \quad (150)$$

Thus, *dilation* and *erosion* on binary images can be viewed as a form of convolution over a Boolean algebra.

In Section 9.3.2 we saw that, when convolution is employed, an appropriate choice of the boundary conditions for an image is essential. Dilation and erosion—being a Boolean convolution—are no exception. The two most common choices are that either everything outside the binary image is “0” or everything outside the binary image is “1”.

9.6.4 Opening and Closing

We can combine *dilation* and *erosion* to build two important higher order operations:

$$\text{Opening} - \quad O(\mathbf{A}, \mathbf{B}) = \mathbf{A} \circ \mathbf{B} = D(E(\mathbf{A}, \mathbf{B}), \mathbf{B}) \quad (151)$$

$$\text{Closing} - \quad C(\mathbf{A}, \mathbf{B}) = \mathbf{A} \bullet \mathbf{B} = E(D(\mathbf{A}, \tilde{\mathbf{B}}), \tilde{\mathbf{B}}) \quad (152)$$

The *opening* and *closing* have the following properties:

$$\begin{aligned} \text{Duality –} \quad C^c(A, B) &= O(A^c, B) \\ O^c(A, B) &= C(A^c, B) \end{aligned} \quad (153)$$

$$\begin{aligned} \text{Translation –} \quad O(A + \mathbf{x}, B) &= O(A, B) + \mathbf{x} \\ C(A + \mathbf{x}, B) &= C(A, B) + \mathbf{x} \end{aligned} \quad (154)$$

For the *opening* with structuring element B and images A , A_1 , and A_2 , where A_1 is a subimage of A_2 ($A_1 \subseteq A_2$):

$$\text{Antiextensivity –} \quad O(A, B) \subseteq A \quad (155)$$

$$\text{Increasing monotonicity –} \quad O(A_1, B) \subseteq O(A_2, B) \quad (156)$$

$$\text{Idempotence –} \quad O(O(A, B), B) = O(A, B) \quad (157)$$

For the *closing* with structuring element B and images A , A_1 , and A_2 , where A_1 is a subimage of A_2 ($A_1 \subseteq A_2$):

$$\text{Extensivity –} \quad A \subseteq C(A, B) \quad (158)$$

$$\text{Increasing monotonicity –} \quad C(A_1, B) \subseteq C(A_2, B) \quad (159)$$

$$\text{Idempotence –} \quad C(C(A, B), B) = C(A, B) \quad (160)$$

The two properties given by eqs. (155) and (84) are so important to mathematical morphology that they can be considered as the reason for defining *erosion* with $\tilde{B}$ instead of B in eq. (129).

9.6.5 Hit-and-Miss operation

The *hit-or-miss operator* was defined by Serra but we shall refer to it as the *hit-and-miss operator* and define it as follows. Given an image A and two structuring elements B_1 and B_2 , the set definition and Boolean definition are:

$$\text{Hit-and-Miss –} \quad \text{HitMiss}(A, B_1, B_2) = \begin{cases} E(A, B_1) \cap E(A^c, B_2) \\ E(A, B_1) \cdot E(\bar{A}, B_2) \end{cases} \quad (161)$$

where $\mathbf{B}_1$ and $\mathbf{B}_2$ are bounded, disjoint structuring elements. (Note the use of the notation from eq. (81).) Two sets are *disjoint* if $\mathbf{B}_1 \cap \mathbf{B}_2 = \emptyset$, the empty set. In an important sense the *hit-and-miss operator* is the morphological equivalent of *template matching*, a well-known technique for matching patterns based upon cross-correlation. Here, we have a template $\mathbf{B}_1$ for the object and a template $\mathbf{B}_2$ for the background.

9.6.6 Summary of the basic operations

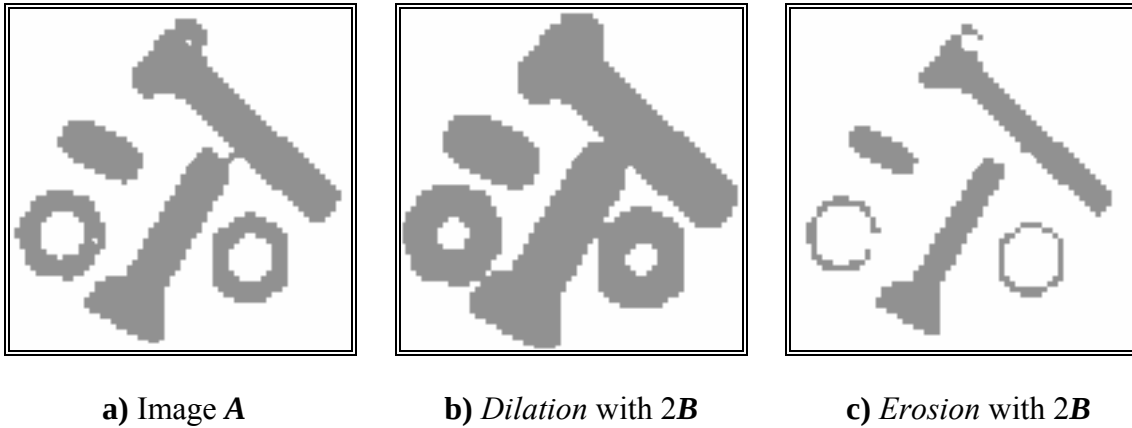
The results of the application of these basic operations on a test image are illustrated below. In Figure 40 the various structuring elements used in the processing are defined. The value “–” indicates a “don’t care”. All three structuring elements are symmetric.

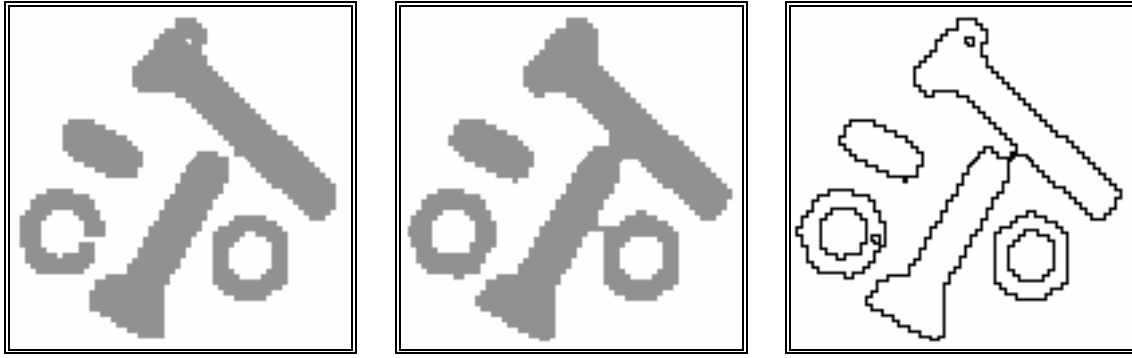
$$\mathbf{B} = \mathbf{N}_8 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \mathbf{B}_1 = \begin{bmatrix} - & - & - \\ - & 1 & - \\ - & - & - \end{bmatrix} \quad \mathbf{B}_2 = \begin{bmatrix} - & 1 & - \\ 1 & - & 1 \\ - & 1 & - \end{bmatrix}$$

(a) (b) (c)

Figure 40: Structuring elements $\mathbf{B}$, $\mathbf{B}_1$, and $\mathbf{B}_2$ that are 3×3 and symmetric.

The results of processing are shown in Figure 41 where the binary value “1” is shown in black and the value “0” in white.





d) Opening with $2B$

e) Closing with $2B$

f) 8-c contour: $A - E(A, N_8)$

Figure 41: Examples of various mathematical morphology operations.

The *opening* operation can separate objects that are connected in a binary image. The *closing* operation can fill in small holes. Both operations generate a certain amount of smoothing on an object contour given a “smooth” structuring element. The *opening* smooths from the inside of the object contour and the *closing* smooths from the outside of the object contour. The *hit-and-miss* example has found the 4-connected contour pixels. An alternative method to find the contour is simply to use the relation:

$$4\text{-connected contour} - \quad \partial A = A - E(A, N_8) \quad (162)$$

or

$$8\text{-connected contour} - \quad \partial A = A - E(A, N_4) \quad (163)$$

9.6.7 Skeleton

The informal definition of a skeleton is a line representation of an object that is:

- i) one-pixel thick,
 - ii) through the “middle” of the object, and,
 - iii) preserves the topology of the object.
- (164)

These are not always realizable. Figure 42 shows why this is the case.

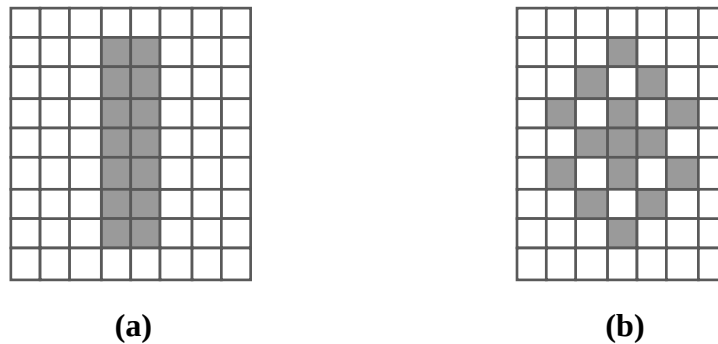


Figure 42: Counterexamples to the three requirements.

In the first example, Figure 42a, it is not possible to generate a line that is one pixel thick and in the center of an object while generating a path that reflects the simplicity of the object. In Figure 42b it is not possible to remove a pixel from the 8-connected object and simultaneously preserve the topology—the notion of connectedness—of the object. Nevertheless, there are a variety of techniques that attempt to achieve this goal and to produce a *skeleton*.

A basic formulation is based on the work of Lantuéjoul [33]. The *skeleton subset* $S_k(\mathbf{A})$ is defined as:

$$\text{Skeleton subsets} - \quad S_k(\mathbf{A}) = E(\mathbf{A}, k\mathbf{B}) - [E(\mathbf{A}, k\mathbf{B}) \circ \mathbf{B}] \quad k = 0, 1, \dots, K \quad (165)$$

where K is the largest value of k before the set $S_k(\mathbf{A})$ becomes empty. (From eq. (156), $E(\mathbf{A}, k\mathbf{B}) \circ \mathbf{B} \subseteq E(\mathbf{A}, k\mathbf{B})$). The structuring element $\mathbf{B}$ is chosen (in Z^2) to approximate a circular disc, that is, convex, bounded and symmetric. The *skeleton* is then the union of the skeleton subsets:

$$\text{Skeleton} - \quad S(\mathbf{A}) = \bigcup_{k=0}^K S_k(\mathbf{A}) \quad (166)$$

An elegant side effect of this formulation is that the original object can be reconstructed given knowledge of the skeleton subsets $S_k(\mathbf{A})$, the structuring element $\mathbf{B}$, and K :

$$\text{Reconstruction} - \quad \mathbf{A} = \bigcup_{k=0}^K (S_k(\mathbf{A}) \oplus k\mathbf{B}) \quad (167)$$

This formulation for the skeleton, however, does not preserve the topology, a requirement described in eq. (164).

An alternative point-of-view is to implement a *thinning*, an erosion that reduces the thickness of an object without permitting it to vanish. A general thinning algorithm is based on the *hit-and-miss* operation:

$$\text{Thinning} - \quad \text{Thin}(\mathbf{A}, \mathbf{B}_1, \mathbf{B}_2) = \mathbf{A} - \text{HitMiss}(\mathbf{A}, \mathbf{B}_1, \mathbf{B}_2) \quad (168)$$

Depending on the choice of $\mathbf{B}_1$ and $\mathbf{B}_2$, a large variety of thinning algorithms—and through repeated application skeletonizing algorithms—can be implemented.

A quite practical implementation can be described in another way. If we restrict ourselves to a 3×3 neighborhood, similar to the structuring element $\mathbf{B} = \mathbf{N}_8$ in Figure 40a, then we can view the thinning operation as a window that repeatedly scans over the (binary) image and sets the center pixel to “0” under certain conditions. The center pixel is *not* changed to “0” if and only if:

- i) an isolated pixel is found (e.g. Figure 43a),
 - ii) removing a pixel would change the connectivity (e.g. Figure 43b),
 - iii) removing a pixel would shorten a line (e.g. Figure 43c).
- (169)

As pixels are (potentially) removed in each iteration, the process is called a *conditional erosion*. Three test cases of eq. (169) are illustrated in Figure 43. In general all possible rotations and variations have to be checked. As there are only 512 possible combinations for a 3×3 window on a binary image, this can be done easily with the use of a lookup table.

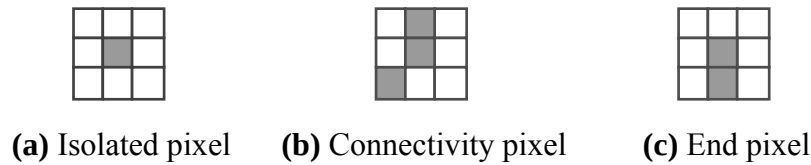


Figure 43: Test conditions for *conditional erosion* of the center pixel.

If only condition (i) is used then each object will be reduced to a single pixel. This is useful if we wish to count the number of objects in an image. If only condition (ii) is used then holes in the objects will be found. If conditions (i + ii) are used each object will be reduced to either a single pixel if it does not contain a hole or to closed rings if it does contain holes. If conditions (i + ii + iii) are used then the “complete skeleton” will be generated as an approximation to eq. (56). Illustrations of these various possibilities are given in Figure 44a,b.

9.6.8 Propagation

It is convenient to be able to reconstruct an image that has “survived” several erosions or to fill an object that is defined, for example, by a boundary. The formal mechanism for this has several names including *region-filling*, *reconstruction*, and *propagation*. The formal definition is given by the following algorithm. We start with a *seed image* $\mathbf{S}^{(0)}$, a *mask image* $\mathbf{A}$, and a structuring element $\mathbf{B}$. We then use dilations of $\mathbf{S}$ with structuring element $\mathbf{B}$ and masked by $\mathbf{A}$ in an iterative procedure as follows:

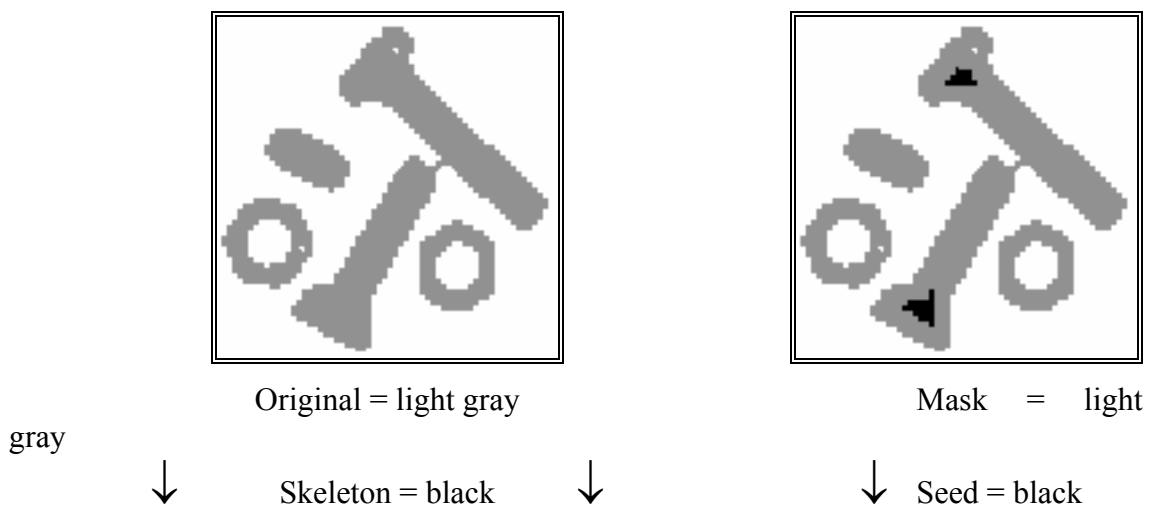
$$\text{Iteration } k - \quad \mathbf{S}^{(k)} = \left[\mathbf{S}^{(k-1)} \oplus \mathbf{B} \right] \cap \mathbf{A} \quad \text{until} \quad \mathbf{S}^{(k)} = \mathbf{S}^{(k-1)} \quad (170)$$

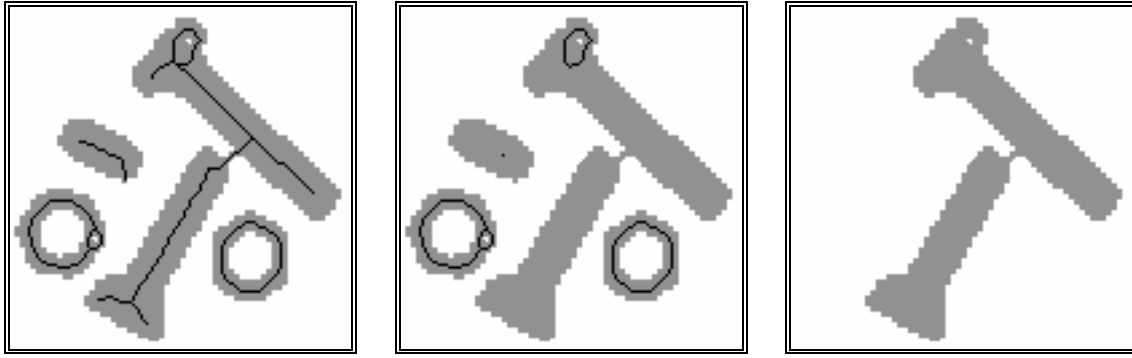
With each iteration the seed image grows (through dilation) but *within* the set (object) defined by A ; S *propagates* to fill A . The most common choices for B are N_4 or N_8 . Several remarks are central to the use of propagation. First, in a straightforward implementation, as suggested by eq. (170), the computational costs are extremely high. Each iteration requires $O(N^2)$ operations for an $N \times N$ image and with the required number of iterations this can lead to a complexity of $O(N^3)$. Fortunately, a recursive implementation of the algorithm exists in which one or two passes through the image are usually sufficient, meaning a complexity of $O(N^2)$. Second, although we have not paid much attention to the issue of object/background connectivity until now (see Figure 36), it is essential that the connectivity implied by B be matched to the connectivity associated with the boundary definition of A (see eqs. (162) and (163)). Finally, as mentioned earlier, it is important to make the correct choice (“0” or “1”) for the boundary condition of the image. The choice depends upon the application.

9.6.9 Summary of skeleton and propagation

The application of these two operations on a test image is illustrated in Figure 44. In Figure 44a,b the skeleton operation is shown with the endpixel condition (eq. (169) $i+ii+iii$) and without the end pixel condition (eq. (169) $i+ii$). The propagation operation is illustrated in Figure 44c. The original image, shown in light gray, was eroded by $E(A, 6N_8)$ to produce the *seed* image shown in black. The original was then used as the *mask* image to produce the final result. The border value in both images was “0”.

Several techniques based upon the use of *skeleton* and *propagation* operations in combination with other mathematical morphology operations will be given in Section 10.3.3.





a) *Skeleton with end pixels* b) *Skeleton without end pixels* c) *Propagation with N_8*
 Condition eq. (169) $i+ii+iii$ Condition eq. (169) $i+ii$

Figure 44: Examples of *skeleton* and *propagation*.

9.6.10 Gray-value morphological processing

The techniques of morphological filtering can be extended to gray-level images. To simplify matters we will restrict our presentation to structuring elements, $\mathbf{B}$, that comprise a finite number of pixels and are convex and bounded. Now, however, the structuring element has gray values associated with every coordinate position as does the image $\mathbf{A}$.

- *Gray-level dilation*, $D_G(\bullet)$, is given by:

$$\text{Dilation} - \quad D_G(\mathbf{A}, \mathbf{B}) = \max_{[j,k] \in \mathbf{B}} \{a[m-j, n-k] + b[j, k]\} \quad (171)$$

For a given output coordinate $[m, n]$, the structuring element is summed with a shifted version of the image and the maximum encountered over all shifts within the $J \times K$ domain of $\mathbf{B}$ is used as the result. Should the shifting require values of the image $\mathbf{A}$ that are outside the $M \times N$ domain of $\mathbf{A}$, then a decision must be made as to which model for image extension, as described in Section 9.3.2, should be used.

- *Gray-level erosion*, $E_G(\bullet)$, is given by:

$$\text{Erosion} - \quad E_G(\mathbf{A}, \mathbf{B}) = \min_{[j,k] \in \mathbf{B}} \{a[m+j, n+k] - b[j, k]\} \quad (172)$$

The duality between *gray-level erosion* and *gray-level dilation*—the gray-level counterpart of eq. (134) is:

$$\begin{aligned} \text{Duality} - \quad E_G(\mathbf{A}, \mathbf{B}) &= -D_G(-\mathbf{A}, \tilde{\mathbf{B}}) \\ D_G(\mathbf{A}, \mathbf{B}) &= -E_G(-\mathbf{A}, \tilde{\mathbf{B}}) \end{aligned} \quad (173)$$

where “ $\tilde{A}$ ” means that $a[j,k] \rightarrow a[-j,-k]$ and “ $-A$ ” means that $a[j,k] \rightarrow -a[j,k]$.

The definitions of higher order operations such as *gray-level opening* and *gray-level closing* are:

$$\text{Opening} - \quad O_G(A, B) = D_G(E_G(A, B), B) \quad (174)$$

$$\text{Closing} - \quad C_G(A, B) = E_G(D_G(A, \tilde{B}), \tilde{B}) \quad (175)$$

The duality between *gray-level opening* and *gray-level closing* is:

$$\begin{aligned} \text{Duality} - \quad O_G(A, B) &= -C_G(-A, B) \\ C_G(A, B) &= -O_G(-A, B) \end{aligned} \quad (176)$$

The important properties that were discussed earlier such as idempotence, translation invariance, increasing in A , and so forth are also applicable to gray level morphological processing. The details can be found in Giardina and Dougherty [6].

In many situations the seeming complexity of gray level morphological processing is significantly reduced through the use of symmetric structuring elements where $b[j,k] = b[-j,-k]$. The most common of these is based on the use of $B = \text{constant} = 0$. For this important case and using again the domain $[j,k] \in B$, the definitions above reduce to:

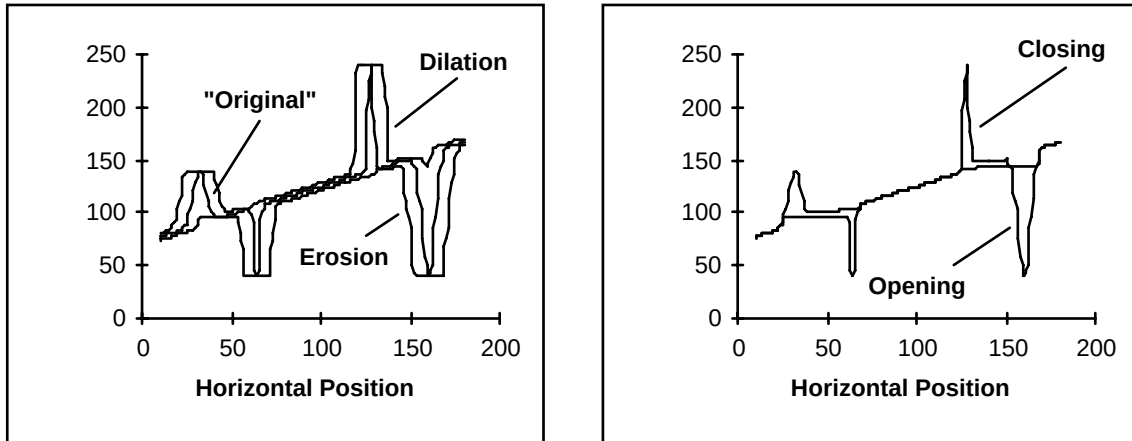
$$\text{Dilation} - \quad D_G(A, B) = \max_{[j,k] \in B} \{a[m-j, n-k]\} = \max_B(A) \quad (177)$$

$$\text{Erosion} - \quad E_G(A, B) = \min_{[j,k] \in B} \{a[m+j, n+k]\} = \min_B(A) \quad (178)$$

$$\text{Opening} - \quad O_G(A, B) = \max_B(\min_B(A)) \quad (179)$$

$$\text{Closing} - \quad C_G(A, B) = \min_B(\max_B(A)) \quad (180)$$

The remarkable conclusion is that the *maximum filter* and the *minimum filter*, introduced in Section 9.4.2, are gray-level dilation and gray-level erosion for the specific structuring element given by the shape of the filter window with the gray value “0” *inside* the window. Examples of these operations on a simple one-dimensional signal are shown in Figure 45.



a) Effect of 15×1 dilation and erosion

b) Effect of 15×1 opening and closing

Figure 45: Morphological filtering of gray-level data.

For a rectangular window, $J \times K$, the two-dimensional maximum or minimum filter is separable into two, one-dimensional windows. Further, a one-dimensional maximum or minimum filter can be written in incremental form. (See Section 9.3.2.) This means that gray-level dilations and erosions have a computational complexity per pixel that is $O(\text{constant})$, that is, independent of J and K . (See also Table 13.)

The operations defined above can be used to produce morphological algorithms for smoothing, gradient determination and a version of the Laplacian. All are constructed from the primitives for *gray-level dilation* and *gray-level erosion* and in all cases the *maximum* and *minimum* filters are taken over the domain $[j, k] \in \mathbf{B}$.

9.6.11 Morphological smoothing

This algorithm is based on the observation that a *gray-level opening* smooths a gray-value image from above the brightness surface given by the function $a[m, n]$ and the *gray-level closing* smooths from below. We use a structuring element $\mathbf{B}$ based on eqs. (84) and (178).

$$\begin{aligned} \text{MorphSmooth}(\mathbf{A}, \mathbf{B}) &= C_G(O_G(\mathbf{A}, \mathbf{B}), \mathbf{B}) \\ &= \min(\max(\max(\min(\mathbf{A})))) \end{aligned} \quad (181)$$

Note that we have suppressed the notation for the structuring element $\mathbf{B}$ under the *max* and *min* operations to keep the notation simple. Its use, however, is understood.

9.6.12 Morphological gradient

For linear filters the gradient filter yields a vector representation (eq. (103)) with a magnitude (eq. (104)) and direction (eq. (105)). The version presented here generates a morphological estimate of the *gradient magnitude*:

$$\begin{aligned} \text{Gradient}(\mathbf{A}, \mathbf{B}) &= \frac{1}{2} (D_G(\mathbf{A}, \mathbf{B}) - E_G(\mathbf{A}, \mathbf{B})) \\ &= \frac{1}{2} (\max(\mathbf{A}) - \min(\mathbf{A})) \end{aligned} \quad (182)$$

9.6.13 Morphological Laplacian

The morphologically-based Laplacian filter is defined by:

$$\begin{aligned} \text{Laplacian}(\mathbf{A}, \mathbf{B}) &= \frac{1}{2} ((D_G(\mathbf{A}, \mathbf{B}) - \mathbf{A}) - (\mathbf{A} - E_G(\mathbf{A}, \mathbf{B}))) \\ &= \frac{1}{2} (D_G(\mathbf{A}, \mathbf{B}) + E_G(\mathbf{A}, \mathbf{B}) - 2\mathbf{A}) \\ &= \frac{1}{2} (\max(\mathbf{A}) + \min(\mathbf{A}) - 2\mathbf{A}) \end{aligned} \quad (183)$$

9.6.14 Summary of morphological filters

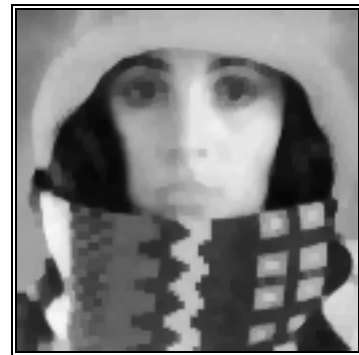
The effect of these filters is illustrated in Figure 46. All images were processed with a 3×3 structuring element as described in eqs. (177) through (183). Figure 46e was contrast stretched for display purposes using eq. (78) and the parameters 1% and 99%. Figures 46c,d,e should be compared to Figures 30, 32, and 33.



a) Dilation



b) Erosion



c) Smoothing

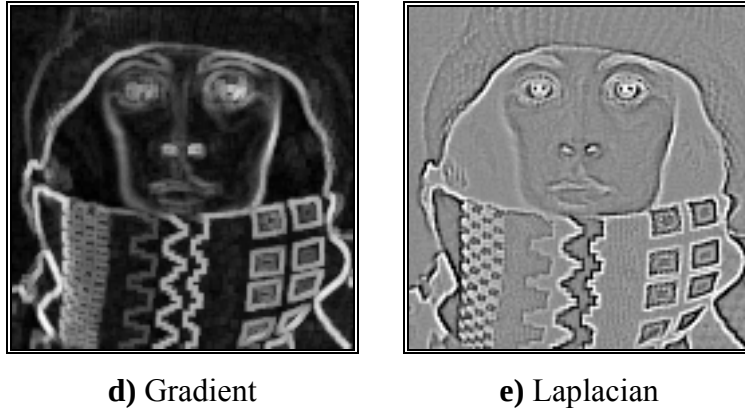


Figure 46: Examples of gray-level morphological filters.

10. Techniques

The algorithms presented in Section 9 can be used to build techniques to solve specific image processing problems. Without presuming to present the solution to all processing problems, the following examples are of general interest and can be used as models for solving related problems.

10.1 SHADING CORRECTION

The method by which images are produced—the interaction between objects in real space, the illumination, and the camera—frequently leads to situations where the image exhibits significant shading across the field-of-view. In some cases the image might be bright in the center and decrease in brightness as one goes to the edge of the field-of-view. In other cases the image might be darker on the left side and lighter on the right side. The shading might be caused by non-uniform illumination, non-uniform camera sensitivity, or even dirt and dust on glass (lens) surfaces. In general this shading effect is undesirable. Eliminating it is frequently necessary for subsequent processing and especially when image analysis or image understanding is the final goal.

10.1.1 Model of shading

In general we begin with a model for the shading effect. The illumination $I_{ill}(x,y)$ usually interacts in a multiplicative with the object $a(x,y)$ to produce the image $b(x,y)$:

$$b(x,y) = I_{ill}(x,y) \cdot a(x,y) \quad (184)$$

with the object representing various imaging modalities such as:

$$a(x, y) = \begin{cases} r(x, y) & \text{reflectance model} \\ 10^{-OD(x, y)} & \text{absorption model} \\ c(x, y) & \text{fluorescence model} \end{cases} \quad (185)$$

where at position (x, y) , $r(x, y)$ is the *reflectance*, $OD(x, y)$ is the *optical density*, and $c(x, y)$ is the concentration of fluorescent material. Parenthetically, we note that the fluorescence model only holds for low concentrations. The camera may then contribute *gain* and *offset* terms, as in eq. (74), so that:

$$\begin{aligned} \text{Total shading} - \quad c[m, n] &= \text{gain}[m, n] \cdot b[m, n] + \text{offset}[m, n] \\ &= \text{gain}[m, n] \cdot I_{ill}[m, n] \cdot a[m, n] + \text{offset}[m, n] \end{aligned} \quad (186)$$

In general we assume that $I_{ill}[m, n]$ is slowly varying compared to $a[m, n]$.

10.1.2 Estimate of shading

We distinguish between two cases for the determination of $a[m, n]$ starting from $c[m, n]$. In both cases we intend to estimate the shading terms $\{\text{gain}[m, n] \cdot I_{ill}[m, n]\}$ and $\{\text{offset}[m, n]\}$. While in the first case we assume that we have only the recorded image $c[m, n]$ with which to work, in the second case we assume that we can record two, additional, calibration images.

- *A posteriori estimate* – In this case we attempt to extract the shading estimate from $c[m, n]$. The most common possibilities are the following.

Lowpass filtering – We compute a smoothed version of $c[m, n]$ where the smoothing is large compared to the size of the objects in the image. This smoothed version is intended to be an estimate of the background of the image. We then subtract the smoothed version from $c[m, n]$ and then restore the desired DC value. In formula:

$$\text{Lowpass} - \quad \hat{a}[m, n] = c[m, n] - \text{LowPass}\{c[m, n]\} + \text{constant} \quad (187)$$

where $\hat{a}[m, n]$ is the estimate of $a[m, n]$. Choosing the appropriate lowpass filter means knowing the appropriate spatial frequencies in the Fourier domain where the shading terms dominate.

Homomorphic filtering – We note that, if the $\text{offset}[m, n] = 0$, then $c[m, n]$ consists solely of multiplicative terms. Further, the term $\{\text{gain}[m, n] \cdot I_{ill}[m, n]\}$ is slowly varying while $a[m, n]$ presumably is not. We therefore take the logarithm of $c[m, n]$

to produce two terms one of which is low frequency and one of which is high frequency. We suppress the shading by high pass filtering the logarithm of $c[m,n]$ and then take the exponent (inverse logarithm) to restore the image. This procedure is based on *homomorphic filtering* as developed by Oppenheim, Schafer and Stockham [34]. In formula:

$$\begin{aligned}
 i) \quad & c[m,n] = \text{gain}[m,n] \cdot I_{ill}[m,n] \cdot a[m,n] \\
 ii) \quad & \ln \{c[m,n]\} = \ln \left\{ \underbrace{\text{gain}[m,n] \cdot I_{ill}[m,n]}_{\text{slowly varying}} \right\} + \ln \left\{ \underbrace{a[m,n]}_{\text{rapidly varying}} \right\} \\
 iii) \quad & \text{HighPass} \{ \ln \{c[m,n]\} \} \approx \ln \{a[m,n]\} \\
 iv) \quad & \hat{a}[m,n] = \exp \{ \text{HighPass} \{ \ln \{c[m,n]\} \} \}
 \end{aligned} \tag{188}$$

Morphological filtering – We again compute a smoothed version of $c[m,n]$ where the smoothing is large compared to the size of the objects in the image but this time using morphological smoothing as in eq. (181). This smoothed version is the estimate of the background of the image. We then subtract the smoothed version from $c[m,n]$ and then restore the desired DC value. In formula:

$$\hat{a}[m,n] = c[m,n] - \text{MorphSmooth} \{c[m,n]\} + \text{constant} \tag{189}$$

Choosing the appropriate morphological filter window means knowing (or estimating) the size of the largest objects of interest.

• *A priori estimate* – If it is possible to record test (calibration) images through the cameras system, then the most appropriate technique for the removal of shading effects is to record two images – $BLACK[m,n]$ and $WHITE[m,n]$. The $BLACK$ image is generated by covering the lens leading to $b[m,n] = 0$ which in turn leads to $BLACK[m,n] = \text{offset}[m,n]$. The $WHITE$ image is generated by using $a[m,n] = 1$ which gives $WHITE[m,n] = \text{gain}[m,n] \cdot I_{ill}[m,n] + \text{offset}[m,n]$. The correction then becomes:

$$\hat{a}[m,n] = \text{constant} \cdot \frac{c[m,n] - BLACK[m,n]}{WHITE[m,n] - BLACK[m,n]} \tag{190}$$

The *constant* term is chosen to produce the desired dynamic range.

The effects of these various techniques on the data from Figure 45 are shown in Figure 47. The shading is a simple, linear ramp increasing from left to right; the objects consist of Gaussian peaks of varying widths.

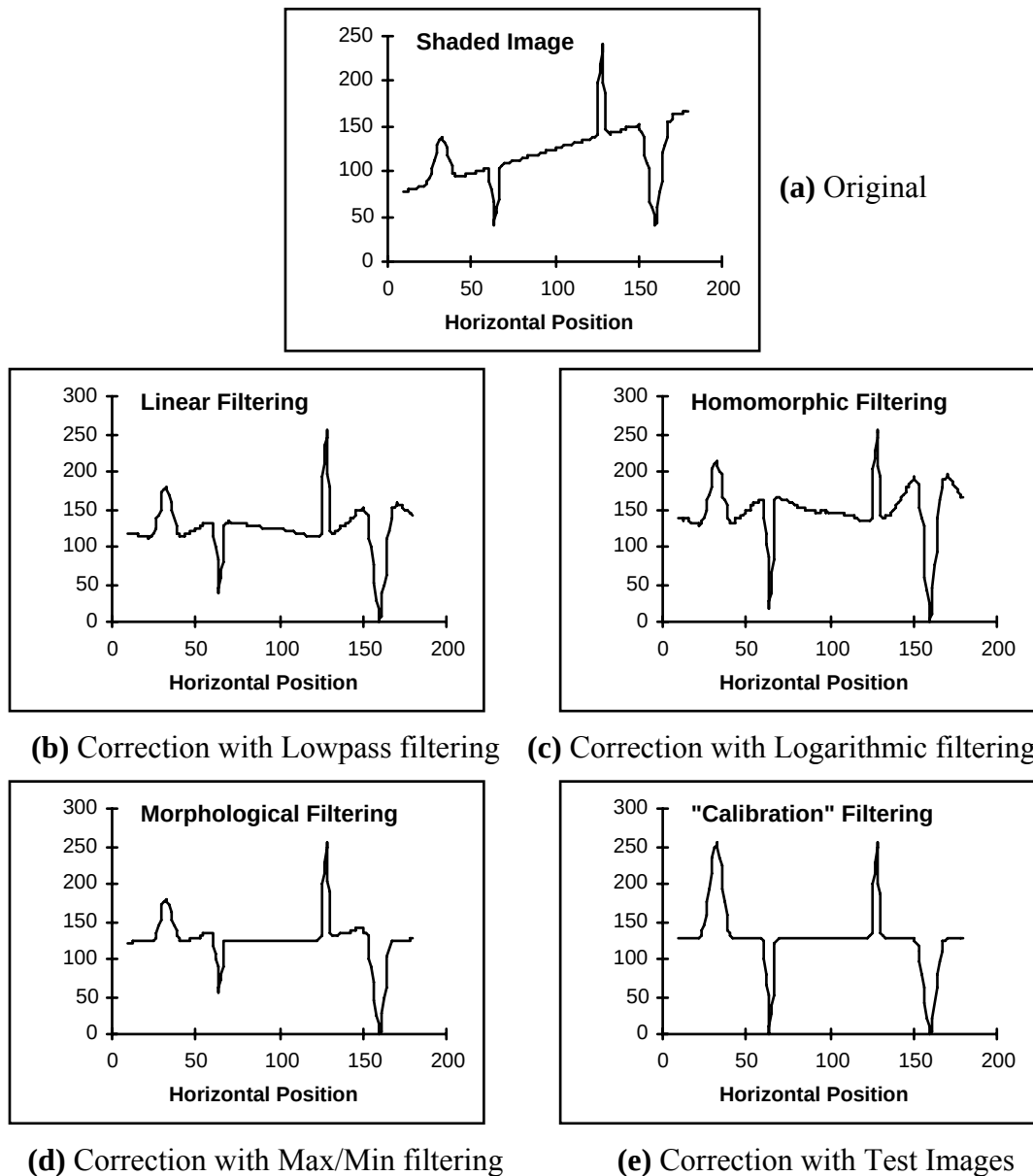


Figure 47: Comparison of various shading correction algorithms. The final result (e) is identical to the original (not shown).

In summary, if it is possible to obtain *BLACK* and *WHITE* calibration images, then eq. (190) is to be preferred. If this is not possible, then one of the other algorithms will be necessary.

10.2 BASIC ENHANCEMENT AND RESTORATION TECHNIQUES

The process of image acquisition frequently leads (inadvertently) to image degradation. Due to mechanical problems, out-of-focus blur, motion, inappropriate illumination, and noise the quality of the digitized image can be inferior to the original. The goal of *enhancement* is— starting from a recorded image $c[m,n]$ —to produce the most visually pleasing image $\hat{a}[m,n]$. The goal of

restoration is—starting from a recorded image $c[m,n]$ —to produce the best possible estimate $\hat{a}[m,n]$ of the original image $a[m,n]$. The goal of enhancement is beauty; the goal of restoration is truth.

The measure of success in restoration is usually an error measure between the original $a[m,n]$ and the estimate $\hat{a}[m,n]$: $\mathfrak{E}\{\hat{a}[m,n], a[m,n]\}$. *No mathematical error function is known that corresponds to human perceptual assessment of error.* The mean-square error function is commonly used because:

1. It is easy to compute;
2. It is differentiable implying that a minimum can be sought;
3. It corresponds to “signal energy” in the total error, and;
4. It has nice properties *vis à vis* Parseval’s theorem, eqs. (22) and (23).

The *mean-square error* is defined by:

$$\mathfrak{E}\{\hat{a}, a\} = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} |\hat{a}[m,n] - a[m,n]|^2 \quad (191)$$

In some techniques an error measure will not be necessary; in others it will be essential for evaluation and comparative purposes.

10.2.1 Unsharp masking

A well-known technique from photography to improve the visual quality of an image is to enhance the edges of the image. The technique is called *unsharp masking*. Edge enhancement means first isolating the edges in an image, amplifying them, and then adding them back into the image. Examination of Figure 33 shows that the Laplacian is a mechanism for isolating the gray level edges. This leads immediately to the technique:

$$\hat{a}[m,n] = a[m,n] - (k \cdot \nabla^2 a[m,n]) \quad (192)$$

The term k is the amplifying term and $k > 0$. The effect of this technique is shown in Figure 48.

The Laplacian used to produce Figure 48 is given by eq. (120) and the amplification term $k = 1$.



Original ↑

↑ Laplacian-enhanced

Figure 48: Edge enhanced compared to original

10.2.2 Noise suppression

The techniques available to suppress noise can be divided into those techniques that are based on temporal information and those that are based on spatial information. By temporal information we mean that a sequence of images $\{a_p[m,n] \mid p=1,2,\dots,P\}$ are available that contain *exactly* the same objects and that differ only in the sense of independent noise realizations. If this is the case and if the noise is additive, then simple averaging of the sequence:

$$\text{Temporal averaging} - \quad \hat{a}[m,n] = \frac{1}{P} \sum_{p=1}^P a_p[m,n] \quad (193)$$

will produce a result where the mean value of each pixel will be unchanged. For each pixel, however, the standard deviation will decrease from σ to $\sigma/\sqrt{P}$.

If temporal averaging is not possible, then spatial averaging can be used to decrease the noise. This generally occurs, however, at a cost to image sharpness. Four obvious choices for spatial averaging are the smoothing algorithms that have been described in Section 9.4 – Gaussian filtering (eq. (93)), median filtering, Kuwahara filtering, and morphological smoothing (eq. (181)).

Within the class of linear filters, the optimal filter for restoration in the presence of noise is given by the *Wiener filter* [2]. The word “optimal” is used here in the sense of minimum mean-square error (*mse*). Because the square root operation is monotonic increasing, the optimal filter also minimizes the root mean-square error (*rms*). The Wiener filter is characterized in the Fourier domain and for additive noise that is independent of the signal it is given by:

$$H_W(u, v) = \frac{S_{aa}(u, v)}{S_{aa}(u, v) + S_{nn}(u, v)} \quad (194)$$

where $S_{aa}(u, v)$ is the power spectral density of an ensemble of random images $\{a[m, n]\}$ and $S_{nn}(u, v)$ is the power spectral density of the random noise. If we have a single image then $S_{aa}(u, v) = |A(u, v)|^2$. In practice it is unlikely that the power spectral density of the uncontaminated image will be available. Because many images have a similar power spectral density that can be modeled by Table 4–T.8, that model can be used as an estimate of $S_{aa}(u, v)$.

A comparison of the five different techniques described above is shown in Figure 49. The Wiener filter was constructed directly from eq. (113) because the image spectrum and the noise spectrum were known. The parameters for the other filters were determined choosing that value (either σ or window size) that led to the minimum *rms*.

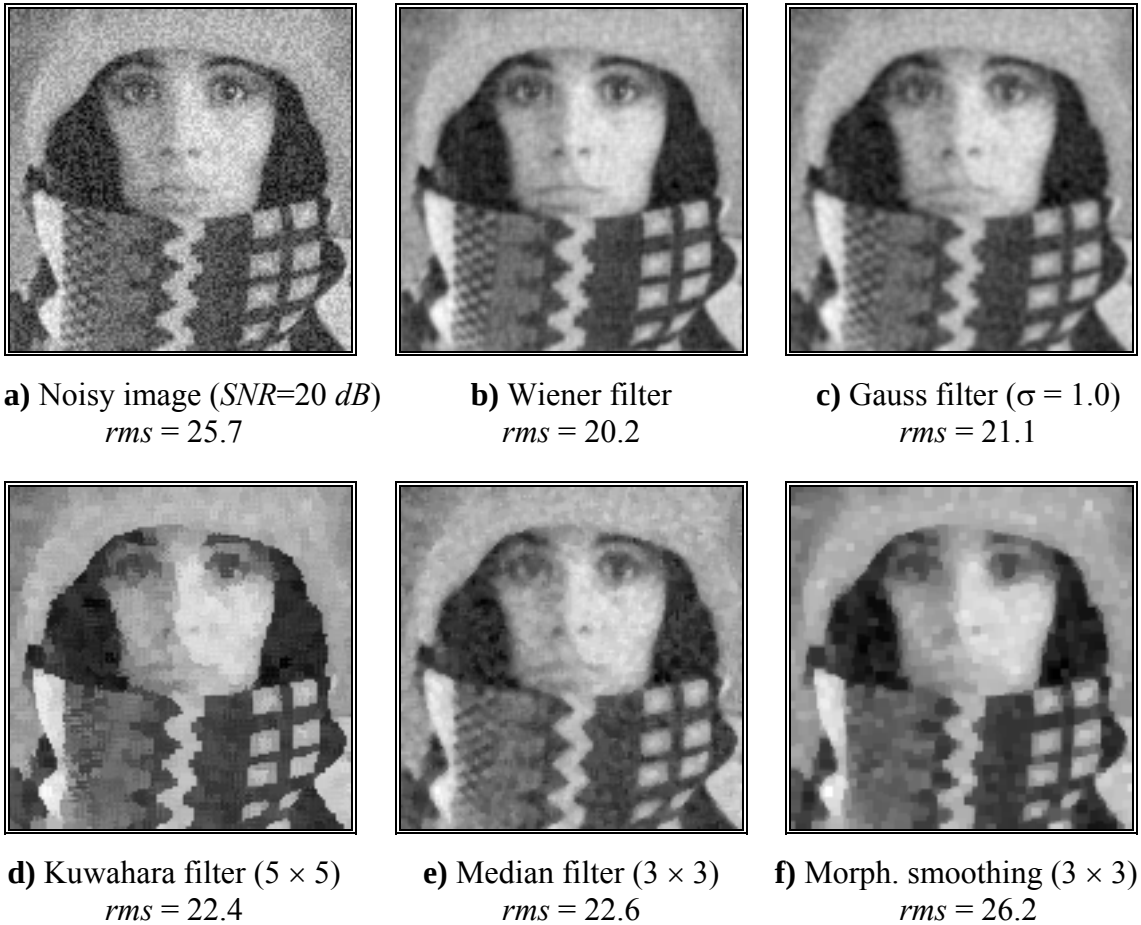


Figure 49: Noise suppression using various filtering techniques.

The root mean-square errors (*rms*) associated with the various filters are shown in Figure 49. For this specific comparison, the Wiener filter generates a lower error

than any of the other procedures that are examined here. The two linear procedures, Wiener filtering and Gaussian filtering, performed slightly better than the three non-linear alternatives.

10.2.3 Distortion suppression

The model presented above—an image distorted solely by noise—is not, in general, sophisticated enough to describe the true nature of distortion in a digital image. A more realistic model includes not only the noise but also a model for the distortion induced by lenses, finite apertures, possible motion of the camera and/or an object, and so forth. One frequently used model is of an image $a[m,n]$ distorted by a linear, shift-invariant system $h_o[m,n]$ (such as a lens) and then contaminated by noise $\kappa[m,n]$. Various aspects of $h_o[m,n]$ and $\kappa[m,n]$ have been discussed in earlier sections. The most common combination of these is the additive model:

$$c[m,n] = (a[m,n] \otimes h_o[m,n]) + \kappa[m,n] \quad (195)$$

The restoration procedure that is based on linear filtering coupled to a minimum mean-square error criterion again produces a Wiener filter [2]:

$$\begin{aligned} H_W(u,v) &= \frac{H_o^*(u,v) S_{aa}(u,v)}{|H_o(u,v)|^2 S_{aa}(u,v) + S_{nn}(u,v)} \\ &= \frac{H_o^*(u,v)}{|H_o(u,v)|^2 + \frac{S_{nn}(u,v)}{S_{aa}(u,v)}} \end{aligned} \quad (196)$$

Once again $S_{aa}(u,v)$ is the power spectral density of an image, $S_{nn}(u,v)$ is the power spectral density of the noise, and $H_o(u,v) = \mathcal{F}\{h_o[m,n]\}$. Examination of this formula for some extreme cases can be useful. For those frequencies where $S_{aa}(u,v) \gg S_{nn}(u,v)$, where the signal spectrum dominates the noise spectrum, the Wiener filter is given by $1/H_o(u,v)$, the *inverse filter* solution. For those frequencies where $S_{aa}(u,v) \ll S_{nn}(u,v)$, where the noise spectrum dominates the signal spectrum, the Wiener filter is proportional to $H_o^*(u,v)$, the *matched filter* solution. For those frequencies where $H_o(u,v) = 0$, the Wiener filter $H_W(u,v) = 0$ preventing overflow.

The Wiener filter is a solution to the restoration problem based upon the hypothesized use of a linear filter and the minimum mean-square (or *rms*) error criterion. In the example below the image $a[m,n]$ was distorted by a bandpass

filter and then white noise was added to achieve an $SNR = 30 \text{ dB}$. The results are shown in Figure 50.

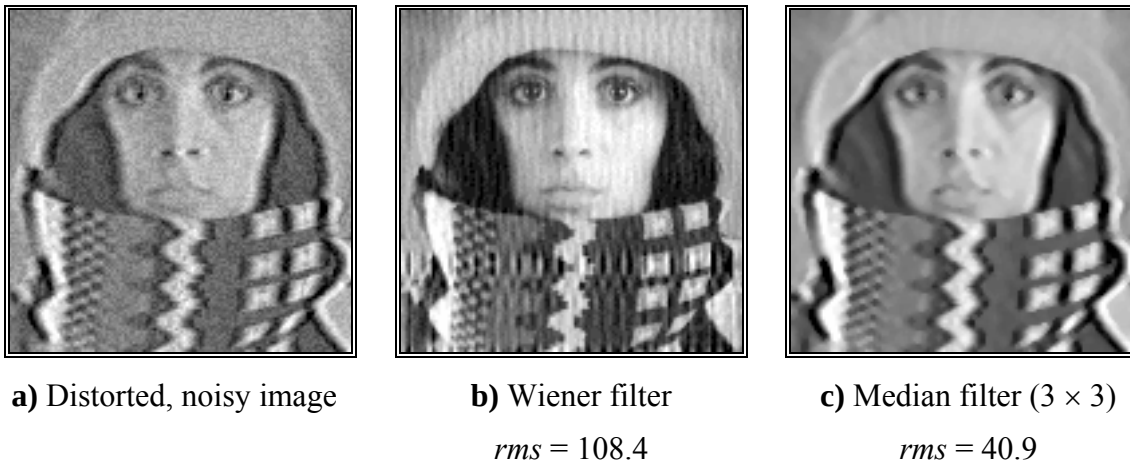


Figure 50: Noise and distortion suppression using the Wiener filter, eq. (196) and the median filter.

The rms after Wiener filtering but before contrast stretching was 108.4; after contrast stretching with eq. (77) the final result as shown in Figure 50b has a mean-square error of 27.8. Using a 3×3 median filter as shown in Figure 50c leads to a rms error of 40.9 before contrast stretching and 35.1 after contrast stretching. Although the Wiener filter gives the minimum rms error over the set of all linear filters, the non-linear median filter gives a lower rms error. The operation *contrast stretching* is itself a non-linear operation. The “visual quality” of the median filtering result is comparable to the Wiener filtering result. This is due in part to periodic artifacts introduced by the linear filter which are visible in Figure 50b.

10.3 SEGMENTATION

In the analysis of the objects in images it is essential that we can distinguish between the objects of interest and “the rest.” This latter group is also referred to as the background. The techniques that are used to find the objects of interest are usually referred to as *segmentation techniques* – segmenting the foreground from background. In this section we will two of the most common techniques—*thresholding* and *edge finding*— and we will present techniques for improving the quality of the segmentation result. It is important to understand that:

- there is no universally applicable segmentation technique that will work for all images, and,
- no segmentation technique is perfect.

10.3.1 Thresholding

This technique is based upon a simple concept. A parameter θ called the *brightness threshold* is chosen and applied to the image $a[m,n]$ as follows:

$$\begin{array}{ll} \text{If } a[m,n] \geq \theta & a[m,n] = \text{object} = 1 \\ \text{Else} & a[m,n] = \text{background} = 0 \end{array} \quad (197)$$

This version of the algorithm assumes that we are interested in light objects on a dark background. For dark objects on a light background we would use:

$$\begin{array}{ll} \text{If } a[m,n] < \theta & a[m,n] = \text{object} = 1 \\ \text{Else} & a[m,n] = \text{background} = 0 \end{array} \quad (198)$$

The output is the label “object” or “background” which, due to its dichotomous nature, can be represented as a Boolean variable “1” or “0”. In principle, the test condition could be based upon some other property than simple brightness (for example, *If (Redness $\{a[m,n]\} \geq \theta_{red}$)*, but the concept is clear.

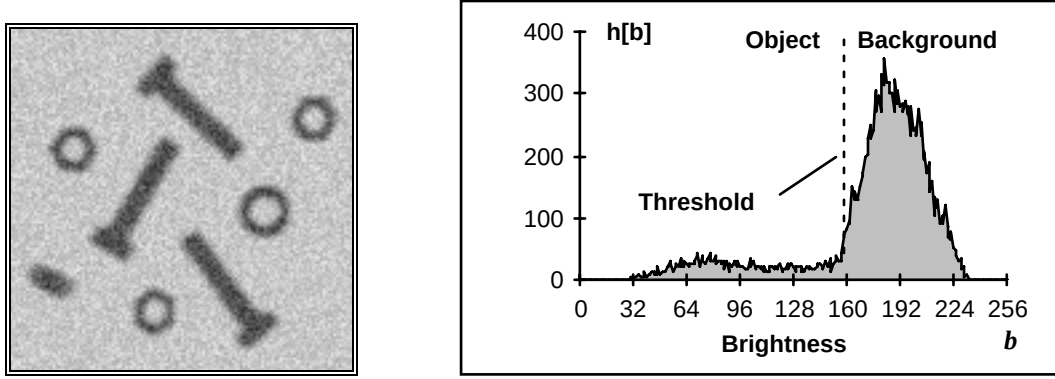
The central question in thresholding then becomes: How do we choose the threshold θ ? While there is no universal procedure for threshold selection that is guaranteed to work on all images, there are a variety of alternatives.

- *Fixed threshold* – One alternative is to use a threshold that is chosen independently of the image data. If it is known that one is dealing with very high-contrast images where the objects are very dark and the background is homogeneous (Section 10.1) and very light, then a constant threshold of 128 on a scale of 0 to 255 might be sufficiently accurate. By accuracy we mean that the number of falsely-classified pixels should be kept to a minimum.

- *Histogram-derived thresholds* – In most cases the threshold is chosen from the brightness histogram of the region or image that we wish to segment. (See Sections 3.5.2 and 9.1.) An image and its associated brightness histogram are shown in Figure 51.

A variety of techniques have been devised to automatically choose a threshold starting from the gray-value histogram, $\{h[b] \mid b = 0, 1, \dots, 2^B-1\}$. Some of the most common ones are presented below. Many of these algorithms can benefit from a smoothing of the raw histogram data to remove small fluctuations but the smoothing algorithm must not shift the peak positions. This translates into a zero-phase smoothing algorithm given below where typical values for W are 3 or 5:

$$h_{smooth}[b] = \frac{1}{W} \sum_{w=-(W-1)/2}^{(W-1)/2} h_{raw}[b-w] \quad W \text{ odd} \quad (199)$$



(a) Image to be *thresholded*

(b) Brightness histogram of the image

Figure 51: Pixels below the threshold ($a[m,n] < \theta$) will be labeled as object pixels; those above the threshold will be labeled as background pixels.

• *Isodata algorithm* – This iterative technique for choosing a threshold was developed by Ridler and Calvard [35]. The histogram is initially segmented into two parts using a starting threshold value such as $\theta_0 = 2^{B-1}$, half the maximum dynamic range. The sample mean ($m_{f,0}$) of the gray values associated with the foreground pixels and the sample mean ($m_{b,0}$) of the gray values associated with the background pixels are computed. A new threshold value θ_1 is now computed as the average of these two sample means. The process is repeated, based upon the new threshold, until the threshold value does not change any more. In formula:

$$\theta_k = (m_{f,k-1} + m_{b,k-1}) / 2 \text{ until } \theta_k = \theta_{k-1} \quad (200)$$

• *Background-symmetry algorithm* – This technique assumes a distinct and dominant peak for the background that is symmetric about its maximum. The technique can benefit from smoothing as described in eq. (199). The maximum peak (*maxp*) is found by searching for the maximum value in the histogram. The algorithm then searches on the *non-object pixel side* of that maximum to find a $p\%$ point as in eq. (39).

In Figure 51b, where the object pixels are located to the *left* of the background peak at brightness 183, this means searching to the right of that peak to locate, as an example, the 95% value. At this brightness value, 5% of the pixels lie to the *right* (are above) that value. This occurs at brightness 216 in Figure 51b. Because of the assumed symmetry, we use as a threshold a displacement to the *left* of the maximum that is equal to the displacement to the right where the $p\%$ is found. For

Figure 51b this means a threshold value given by $183 - (216 - 183) = 150$. In formula:

$$\theta = maxp - (p\% - maxp) \quad (201)$$

This technique can be adapted easily to the case where we have light objects on a dark, dominant background. Further, it can be used if the object peak dominates and we have reason to assume that the brightness distribution around the object peak is symmetric. An additional variation on this symmetry theme is to use an estimate of the sample standard deviation (s in eq. (37)) based on one side of the dominant peak and then use a threshold based on $\theta = maxp \pm 1.96s$ (at the 5% level) or $\theta = maxp \pm 2.57s$ (at the 1% level). The choice of “+” or “-” depends on which direction from $maxp$ is being defined as the object/background threshold. Should the distributions be approximately Gaussian around $maxp$, then the values 1.96 and 2.57 will, in fact, correspond to the 5% and 1 % level.

• *Triangle algorithm* – This technique due to Zack [36] is illustrated in Figure 52. A line is constructed between the maximum of the histogram at brightness b_{max} and the lowest value $b_{min} = (p=0)\%$ in the image. The distance d between the line and the histogram $h[b]$ is computed for all values of b from $b = b_{min}$ to $b = b_{max}$. The brightness value b_o where the distance between $h[b_o]$ and the line is maximal is the threshold value, that is, $\theta = b_o$. This technique is particularly effective when the object pixels produce a weak peak in the histogram.

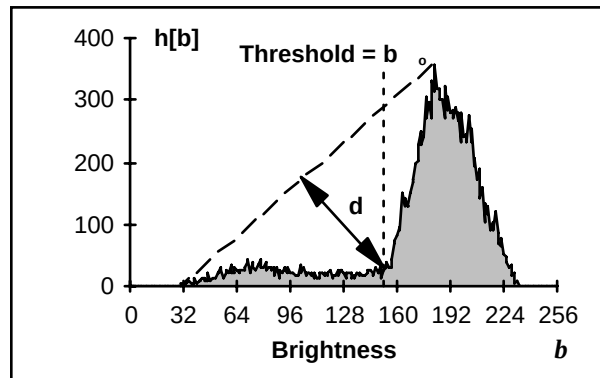


Figure 52: The triangle algorithm is based on finding the value of b that gives the maximum distance d .

The three procedures described above give the values $\theta = 139$ for the Isodata algorithm, $\theta = 150$ for the background symmetry algorithm at the 5% level, and $\theta = 152$ for the triangle algorithm for the image in Figure 51a.

Thresholding does not have to be applied to entire images but can be used on a region by region basis. Chow and Kaneko [37] developed a variation in which the

$M \times N$ image is divided into non-overlapping regions. In each region a threshold is calculated and the resulting threshold values are put together (interpolated) to form a thresholding surface for the entire image. The regions should be of “reasonable” size so that there are a sufficient number of pixels in each region to make an estimate of the histogram and the threshold. The utility of this procedure—like so many others—depends on the application at hand.

10.3.2 Edge finding

Thresholding produces a segmentation that yields all the pixels that, in principle, belong to the object or objects of interest in an image. An alternative to this is to find those pixels that belong to the borders of the objects. Techniques that are directed to this goal are termed *edge finding techniques*. From our discussion in Section 9.6 on mathematical morphology, specifically eqs. (79), (163), and (170), we see that there is an intimate relationship between edges and regions.

- *Gradient-based procedure* – The central challenge to edge finding techniques is to find procedures that produce *closed* contours around the objects of interest. For objects of particularly high SNR, this can be achieved by calculating the gradient and then using a suitable threshold. This is illustrated in Figure 53.

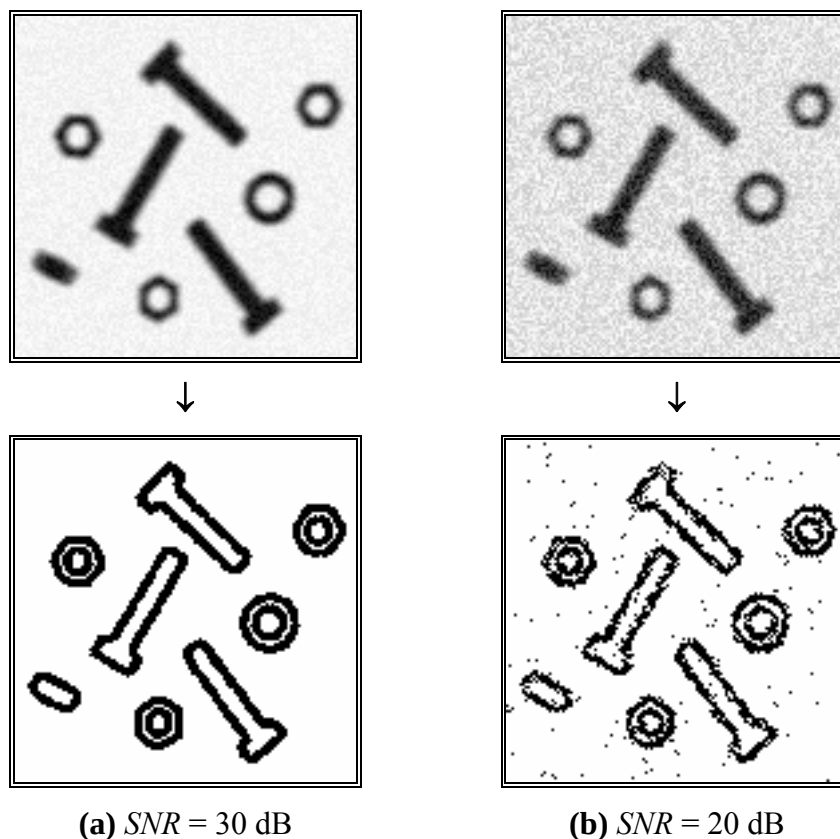


Figure 53: Edge finding based on the Sobel gradient, eq. (110), combined with the Isodata thresholding algorithm eq. (92).

While the technique works well for the 30 dB image in Figure 53a, it fails to provide an accurate determination of those pixels associated with the object edges for the 20 dB image in Figure 53b. A variety of smoothing techniques as described in Section 9.4 and in eq. (181) can be used to reduce the noise effects before the gradient operator is applied.

- *Zero-crossing based procedure* – A more modern view to handling the problem of edges in noisy images is to use the zero crossings generated in the Laplacian of an image (Section 9.5.2). The rationale starts from the model of an ideal edge, a step function, that has been blurred by an *OTF* such as Table 4 T.3 (out-of-focus), T.5 (diffraction-limited), or T.6 (general model) to produce the result shown in Figure 54.

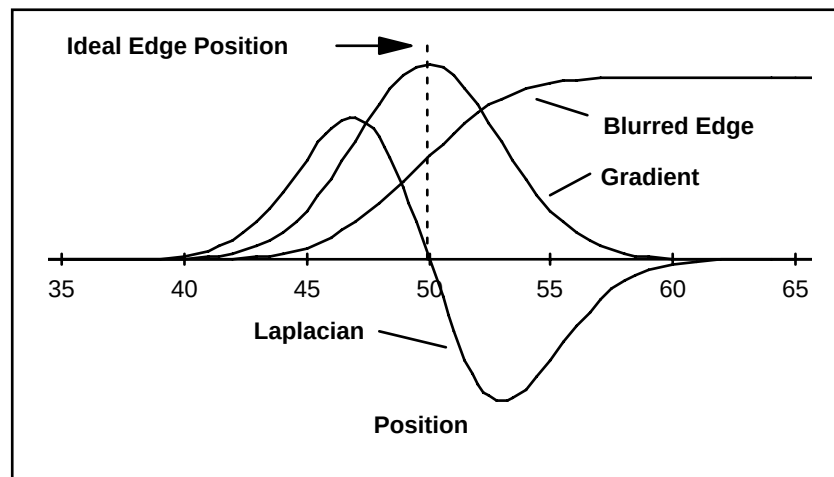


Figure 54: Edge finding based on the zero crossing as determined by the second derivative, the Laplacian. The curves are not to scale.

The edge location is, according to the model, at that place in the image where the Laplacian changes sign, the zero crossing. As the Laplacian operation involves a second derivative, this means a potential enhancement of noise in the image at high spatial frequencies; see eq. (114). To prevent enhanced noise from dominating the search for zero crossings, a smoothing is necessary.

The appropriate smoothing filter, from among the many possibilities described in Section 9.4, should according to Canny [38] have the following properties:

- In the frequency domain, (u,v) or (Ω,Ψ) , the filter should be as narrow as possible to provide suppression of high frequency noise, and;
- In the spatial domain, (x,y) or $[m,n]$, the filter should be as narrow as possible to provide good localization of the edge. A too wide filter generates uncertainty as to precisely where, within the filter width, the edge is located.

The smoothing filter that simultaneously satisfies both these properties—minimum bandwidth and minimum spatial width—is the Gaussian filter described in Section 9.4. This means that the image should be smoothed with a Gaussian of an appropriate σ followed by application of the Laplacian. In formula:

$$\text{ZeroCrossing}\{a(x, y)\} = \{(x, y) \mid \nabla^2 \{g_{2D}(x, y) \otimes a(x, y)\} = 0\} \quad (202)$$

where $g_{2D}(x, y)$ is defined in eq. (93). The derivative operation is linear and shift-invariant as defined in eqs. (85) and (86). This means that the order of the operators can be exchanged (eq. (4)) or combined into one single filter (eq. (5)). This second approach leads to the Marr-Hildreth formulation of the “Laplacian-of-Gaussians” (*LoG*) filter [39]:

$$\text{ZeroCrossing}\{a(x, y)\} = \{(x, y) \mid \text{LoG}(x, y) \otimes a(x, y) = 0\} \quad (203)$$

where

$$\text{LoG}(x, y) = \frac{x^2 + y^2}{\sigma^4} g_{2D}(x, y) - \frac{2}{\sigma^2} g_{2D}(x, y) \quad (204)$$

Given the circular symmetry this can also be written as:

$$\text{LoG}(r) = \left(\frac{r^2 - 2\sigma^2}{2\pi\sigma^6} \right) e^{-(r^2/2\sigma^2)} \quad (205)$$

This two-dimensional convolution kernel, which is sometimes referred to as a “Mexican hat filter”, is illustrated in Figure 55.

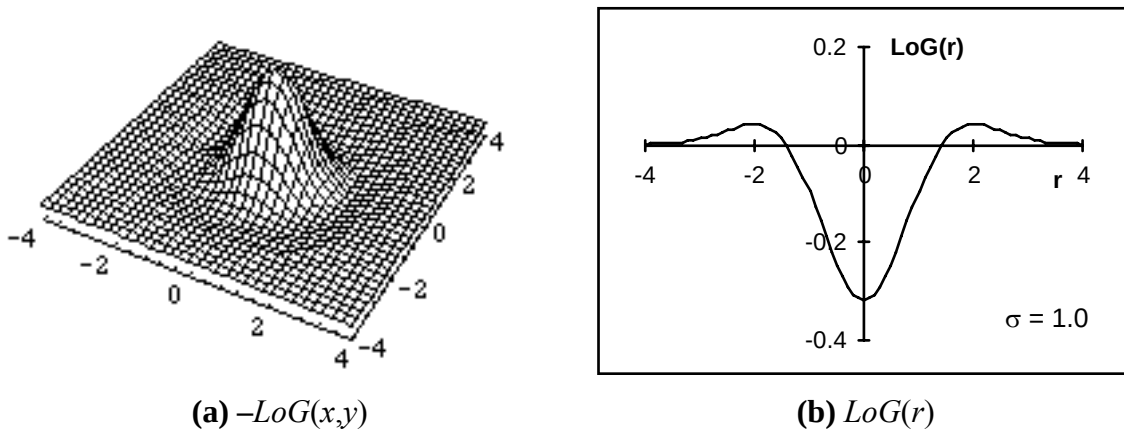


Figure 55: *LoG* filter with $\sigma = 1.0$.

•*PLUS-based procedure* – Among the zero crossing procedures for edge detection, perhaps the most accurate is the *PLUS* filter as developed by Verbeek and Van Vliet [40]. The filter is defined, using eqs. (121) and (122), as:

$$\begin{aligned} PLUS(a) &= SDGD(a) + Laplace(a) \\ &= \left(\frac{A_{xx}A_x^2 + 2A_{xy}A_xA_y + A_{yy}A_y^2}{A_x^2 + A_y^2} \right) + (A_{xx} + A_{yy}) \end{aligned} \quad (206)$$

Neither the derivation of the *PLUS*'s properties nor an evaluation of its accuracy are within the scope of this section. Suffice it to say that, for positively curved edges in gray value images, the Laplacian-based zero crossing procedure *overestimates* the position of the edge and the *SDGD*-based procedure *underestimates* the position. This is true in both two-dimensional and three-dimensional images with an error on the order of $(\sigma/R)^2$ where R is the radius of curvature of the edge. The *PLUS* operator has an error on the order of $(\sigma/R)^4$ if the image is sampled at, at least, $3\times$ the usual Nyquist sampling frequency as in eq. (56) or if we choose $\sigma \geq 2.7$ and sample at the usual Nyquist frequency.

All of the methods based on zero crossings in the Laplacian must be able to distinguish between zero *crossings* and zero *values*. While the former represent edge positions, the latter can be generated by regions that are no more complex than bilinear surfaces, that is, $a(x,y) = a_0 + a_1 \cdot x + a_2 \cdot y + a_3 \cdot x \cdot y$. To distinguish between these two situations, we first find the zero crossing positions and label them as “1” and all other pixels as “0”. We then multiply the resulting image by a measure of the *edge strength* at each pixel. There are various measures for the edge strength that are all based on the gradient as described in Section 9.5.1 and eq. (182). This last possibility, use of a morphological gradient as an edge strength measure, was first described by Lee, Haralick, and Shapiro [41] and is particularly effective. After multiplication the image is then thresholded (as above) to produce the final result. The procedure is thus as follows [42]:

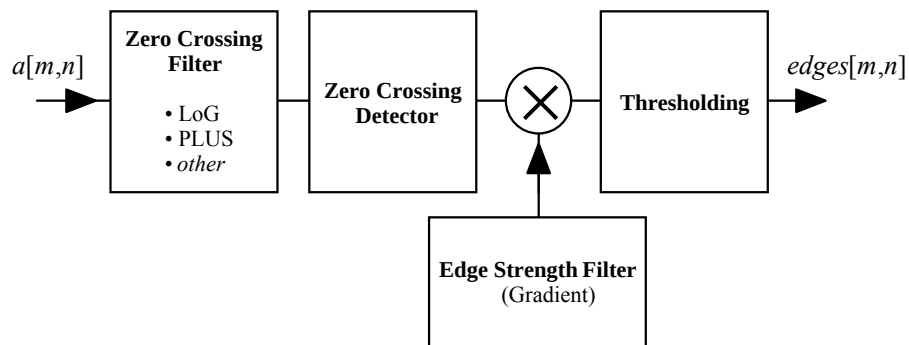


Figure 56: General strategy for edges based on zero crossings.

The results of these two edge finding techniques based on zero crossings, *LoG* filtering and *PLUS* filtering, are shown in Figure 57 for images with a 20 dB SNR.

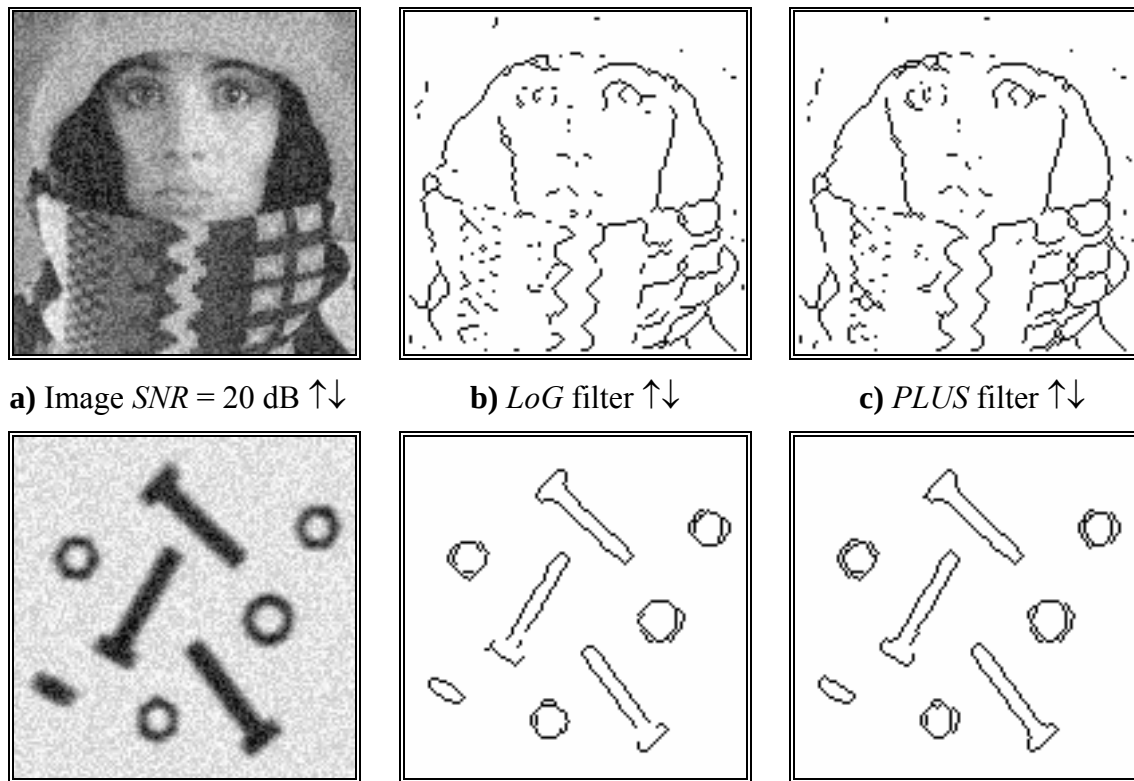


Figure 57: Edge finding using zero crossing algorithms *LoG* and *PLUS*. In both algorithms $\sigma = 1.5$.

Edge finding techniques provide, as the name suggests, an image that contains a collection of edge pixels. Should the edge pixels correspond to objects, as opposed to say simple lines in the image, then a region-filling technique such as eq. (170) may be required to provide the complete objects.

10.3.3 Binary mathematical morphology

The various algorithms that we have described for mathematical morphology in Section 9.6 can be put together to form powerful techniques for the processing of binary images and gray level images. As binary images frequently result from segmentation processes on gray level images, the morphological processing of the binary result permits the improvement of the segmentation result.

- *Salt-or-pepper filtering* – Segmentation procedures frequently result in isolated “1” pixels in a “0” neighborhood (salt) or isolated “0” pixels in a “1” neighborhood (pepper). The appropriate neighborhood definition must be chosen as in Figure 3. Using the lookup table formulation for Boolean operations in a 3×3 neighborhood that was described in association with Figure 43, *salt filtering* and

pepper filtering are straightforward to implement. We weight the different positions in the 3×3 neighborhood as follows:

$$Weights = \begin{bmatrix} w_4 = 16 & w_3 = 8 & w_2 = 4 \\ w_5 = 32 & w_0 = 1 & w_1 = 2 \\ w_6 = 64 & w_7 = 128 & w_8 = 256 \end{bmatrix} \quad (207)$$

For a 3×3 window in $a[m,n]$ with values “0” or “1” we then compute:

$$\begin{aligned} sum = & w_0 a[m,n] + w_1 a[m+1,n] + w_2 a[m+1,n-1] + \\ & w_3 a[m,n-1] + w_4 a[m-1,n-1] + w_5 a[m-1,n] + \\ & w_6 a[m-1,n+1] + w_7 a[m,n+1] + w_8 a[m+1,n-1] \end{aligned} \quad (208)$$

The result, sum , is a number bounded by $0 \leq sum \leq 511$.

• *Salt Filter* – The 4-connected and 8-connected versions of this filter are the same and are given by the following procedure:

$$\begin{aligned} & \text{i) Compute } sum \\ & \text{ii) If } (sum == 1) \quad c[m,n] = 0 \\ & \quad \text{Else} \quad c[m,n] = a[m,n] \end{aligned} \quad (209)$$

• *Pepper Filter* – The 4-connected and 8-connected versions of this filter are the following procedures:

<i>4-connected</i>	<i>8-connected</i>	
i) Compute sum	i) Compute sum	(210)
ii) If ($sum == 170$)	ii) If ($sum == 510$)	
$c[m,n] = 1$	$c[m,n] = 1$	
Else	Else	
$c[m,n] = a[m,n]$	$c[m,n] = a[m,n]$	

• *Isolate objects with holes* – To find objects with holes we can use the following procedure which is illustrated in Figure 58.

- i) *Segment* image to produce binary mask representation (211)
- ii) Compute *skeleton* without end pixels – eq. (169)
- iii) Use *salt* filter to remove single skeleton pixels
- iv) *Propagate* remaining skeleton pixels into original binary mask – eq. (170)

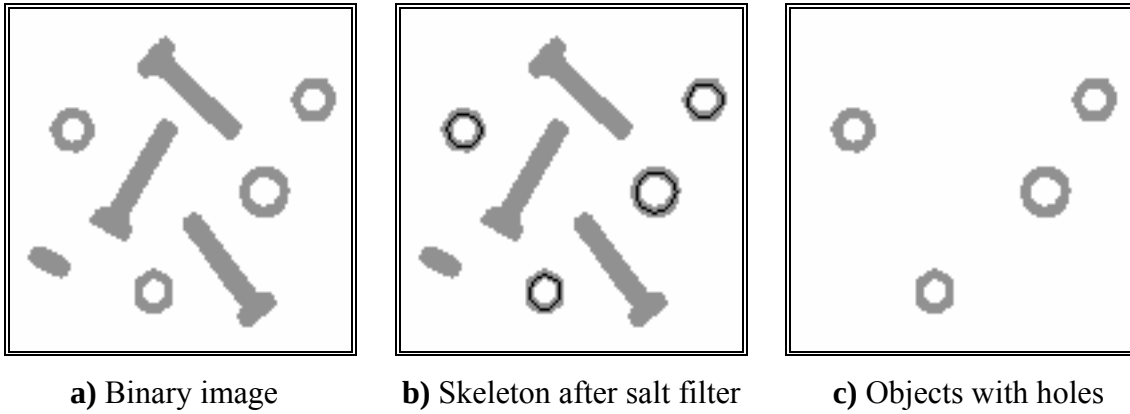


Figure 58: Isolation of objects with holes using morphological operations.

The binary objects are shown in gray and the skeletons, after application of the salt filter, are shown as a black overlay on the binary objects. Note that this procedure uses no parameters other than the fundamental choice of connectivity; it is free from “magic numbers.” In the example shown in Figure 58, the 8-connected definition was used as well as the structuring element $B = N_8$.

• *Filling holes in objects* – To fill holes in objects we use the following procedure which is illustrated in Figure 59.

- i) *Segment* image to produce binary representation of objects (212)
- ii) Compute *complement* of binary image as a *mask image*
- iii) Generate a *seed image* as the border of the image
- iv) *Propagate* the *seed* into the *mask* – eq. (97)
- v) *Complement* result of propagation to produce final result

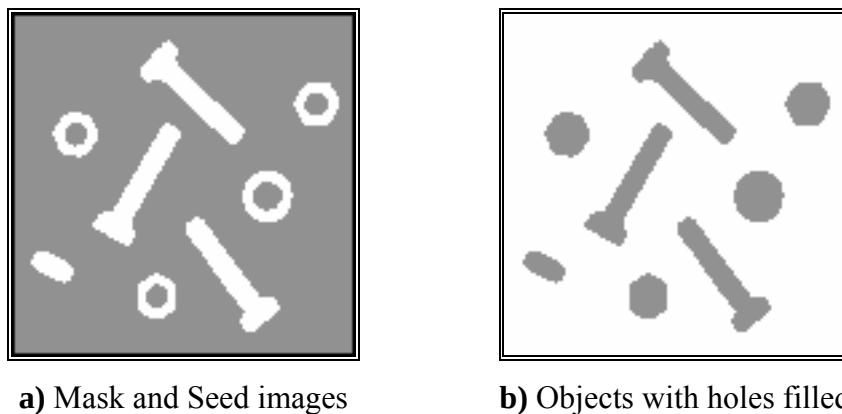


Figure 59: Filling holes in objects.

The *mask image* is illustrated in gray in Figure 59a and the *seed image* is shown in black in that same illustration. When the object pixels are specified with a connectivity of $C = 8$, then the propagation into the mask (background) image

should be performed with a connectivity of $C = 4$, that is, dilations with the structuring element $\mathbf{B} = \mathbf{N}_4$. This procedure is also free of “magic numbers.”

• *Removing border-touching objects* – Objects that are connected to the image border are not suitable for analysis. To eliminate them we can use a series of morphological operations that are illustrated in Figure 60.

- i) *Segment* image to produce binary *mask image* of objects (213)
- ii) Generate a *seed image* as the border of the image
- iv) *Propagate* the *seed* into the *mask* – eq. (97)
- v) Compute *XOR* of the propagation result and the *mask image* as final result

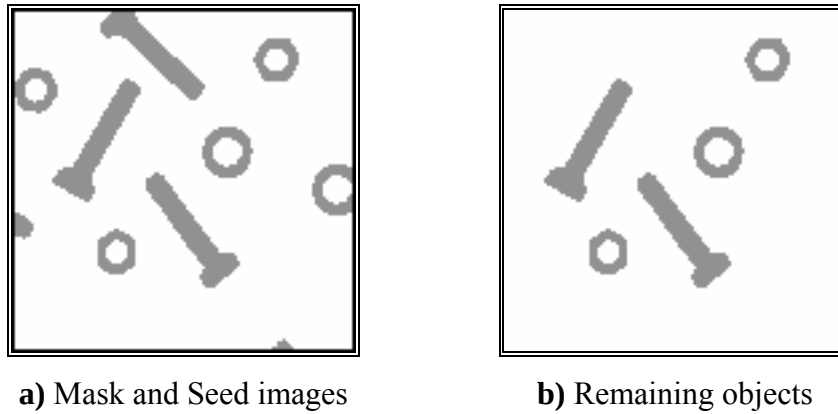


Figure 60: Removing objects touching borders.

The *mask image* is illustrated in *gray* in Figure 60a and the *seed image* is shown in *black* in that same illustration. If the structuring element used in the propagation is $\mathbf{B} = \mathbf{N}_4$, then objects are removed that are 4-connected with the image boundary. If $\mathbf{B} = \mathbf{N}_8$ is used then objects that 8-connected with the boundary are removed.

• *Exo-skeleton* – The *exo-skeleton* of a set of objects is the skeleton of the background that contains the objects. The *exo-skeleton* produces a partition of the image into regions each of which contains one object. The actual skeletonization (eq. (169)) is performed without the preservation of end pixels and with the border set to “0.” The procedure is described below and the result is illustrated in Figure 61.

- i) *Segment* image to produce binary image (214)
- ii) Compute *complement* of binary image
- iii) Compute *skeleton* using eq. (169) i+ii with border set to “0”

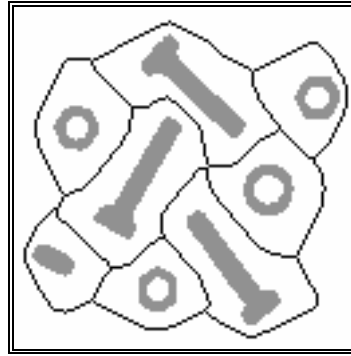
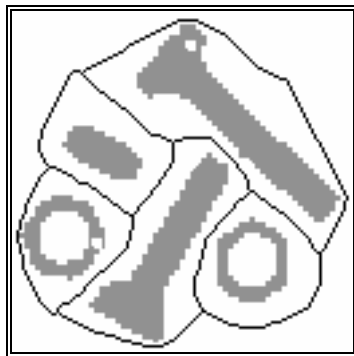


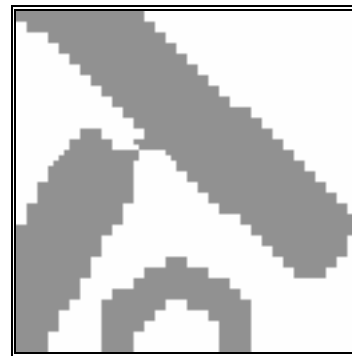
Figure 61: Exo-skeleton.

• *Touching objects* – Segmentation procedures frequently have difficulty separating slightly touching, yet distinct, objects. The following procedure provides a mechanism to separate these objects and makes minimal use of “magic numbers.” The exo-skeleton produces a partition of the image into regions each of which contains one object. The actual skeletonization is performed without the preservation of end pixels and with the border set to “0.” The procedure is illustrated in Figure 62.

- i) *Segment* image to produce binary image
- (215)
- ii) Compute a “small number” of *erosions* with $B = N_4$
- iii) Compute *exo-skeleton* of eroded result
- iv) Complement *exo-skeleton* result
- v) Compute *AND* of original binary image and the complemented exo-skeleton



a) Eroded and exo-skeleton images



b) Objects separated (detail)

Figure 62: Separation of touching objects.

The *eroded binary image* is illustrated in *gray* in Figure 62a and the *exo-skeleton image* is shown in *black* in that same illustration. An enlarged section of the final result is shown in Figure 62b and the separation is easily seen. This procedure involves choosing a small, minimum number of erosions but the number is not critical as long as it initiates a coarse separation of the desired objects. The actual

separation is performed by the exo-skeleton which, itself, is free of “magic numbers.” If the exo-skeleton is 8-connected than the background separating the objects will be 8-connected. The objects, themselves, will be disconnected according to the 4-connected criterion. (See Section 9.6 and Figure 36.)

10.3.4 Gray-value mathematical morphology

As we have seen in Section 10.1.2, gray-value morphological processing techniques can be used for practical problems such as shading correction. In this section several other techniques will be presented.

- *Top-hat transform* – The isolation of gray-value objects that are convex can be accomplished with the *top-hat transform* as developed by Meyer [43, 44]. Depending upon whether we are dealing with light objects on a dark background or dark objects on a light background, the transform is defined as:

$$\text{Light objects} - \quad TopHat(\mathbf{A}, \mathbf{B}) = \mathbf{A} - (\mathbf{A} \circ \mathbf{B}) = \mathbf{A} - \max_{\mathbf{B}} \left(\min_{\mathbf{B}} (\mathbf{A}) \right) \quad (216)$$

$$\text{Dark objects} - \quad TopHat(\mathbf{A}, \mathbf{B}) = (\mathbf{A} \bullet \mathbf{B}) - \mathbf{A} = \min_{\mathbf{B}} \left(\max_{\mathbf{B}} (\mathbf{A}) \right) - \mathbf{A} \quad (217)$$

where the structuring element $\mathbf{B}$ is chosen to be bigger than the objects in question and, if possible, to have a convex shape. Because of the properties given in eqs. (155) and (158), $TopHat(\mathbf{A}, \mathbf{B}) \geq 0$. An example of this technique is shown in Figure 63.

The original image including shading is processed by a 15×1 structuring element as described in eqs. (216) and (217) to produce the desired result. Note that the transform for dark objects has been defined in such a way as to yield “positive” objects as opposed to “negative” objects. Other definitions are, of course, possible.

- *Thresholding* – A simple estimate of a locally-varying threshold surface can be derived from morphological processing as follows:

$$\text{Threshold surface} - \quad \theta[m, n] = \frac{1}{2} (\max(\mathbf{A}) + \min(\mathbf{A})) \quad (218)$$

Once again, we suppress the notation for the structuring element $\mathbf{B}$ under the *max* and *min* operations to keep the notation simple. Its use, however, is understood.

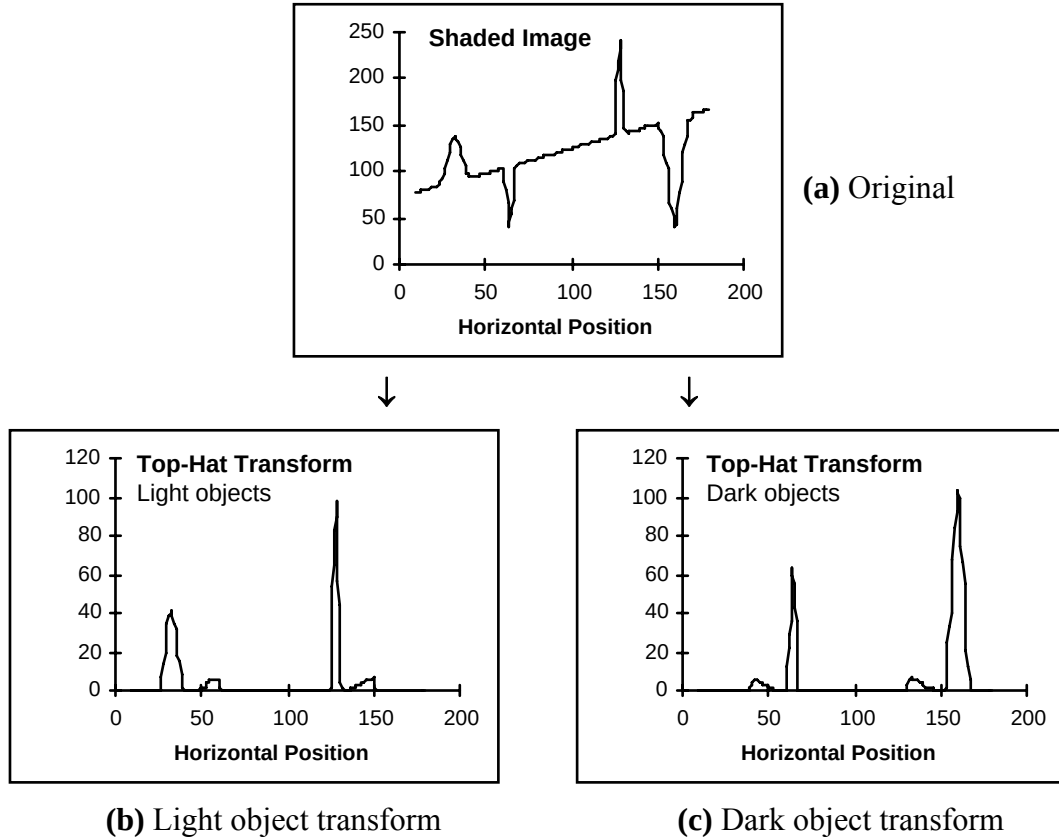


Figure 63: Top-hat transforms.

• *Local contrast stretching* – Using morphological operations we can implement a technique for *local contrast stretching*. That is, the amount of stretching that will be applied in a neighborhood will be controlled by the original contrast in that neighborhood. The morphological gradient defined in eq. (182) may also be seen as related to a measure of the local contrast in the window defined by the structuring element $\mathbf{B}$:

$$LocalContrast(\mathbf{A}, \mathbf{B}) = \max(\mathbf{A}) - \min(\mathbf{A}) \quad (219)$$

The procedure for local contrast stretching is given by:

$$c[m, n] = scale \cdot \frac{\mathbf{A} - \min(\mathbf{A})}{\max(\mathbf{A}) - \min(\mathbf{A})} \quad (220)$$

The *max* and *min* operations are taken over the structuring element $\mathbf{B}$. The effect of this procedure is illustrated in Figure 64. It is clear that this *local* operation is an extended version of the *point* operation for contrast stretching presented in eq. (77).

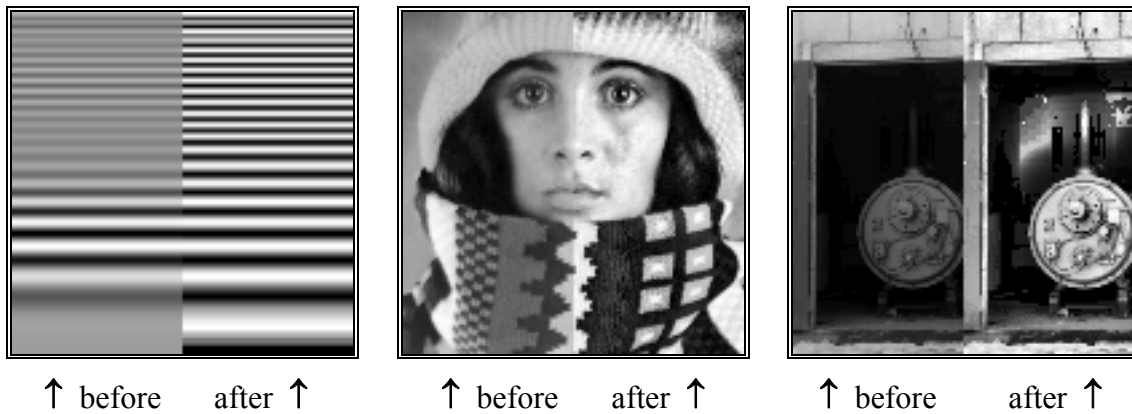


Figure 64: Local contrast stretching.

Using standard test images (as we have seen in so many examples) illustrates the power of this local morphological filtering approach.

11. Acknowledgments

This work was partially supported by the Netherlands Organization for Scientific Research (NWO) Grant 900-538-040, the Foundation for Technical Sciences (STW) Project 2987, the ASCI PostDoc program, and the Rolling Grants program of the Foundation for Fundamental Research in Matter (FOM). Images presented above were processed using *TCL-Image* and *SCIL-Image* (both from the TNO-TPD, Stieltjesweg 1, Delft, The Netherlands) and Adobe *Photoshop*TM.

12. References

1. Dudgeon, D.E. and R.M. Mersereau, *Multidimensional Digital Signal Processing*. 1984, Englewood Cliffs, New Jersey: Prentice-Hall.
2. Castleman, K.R., *Digital Image Processing*. Second ed. 1996, Englewood Cliffs, New Jersey: Prentice-Hall.
3. Oppenheim, A.V., A.S. Willsky, and I.T. Young, *Systems and Signals*. 1983, Englewood Cliffs, New Jersey: Prentice-Hall.
4. Papoulis, A., *Systems and Transforms with Applications in Optics*. 1968, New York: McGraw-Hill.
5. Russ, J.C., *The Image Processing Handbook*. Second ed. 1995, Boca Raton, Florida: CRC Press.
6. Giardina, C.R. and E.R. Dougherty, *Morphological Methods in Image and Signal Processing*. 1988, Englewood Cliffs, New Jersey: Prentice-Hall. 321.

7. Gonzalez, R.C. and R.E. Woods, *Digital Image Processing*. 1992, Reading, Massachusetts: Addison-Wesley. 716.
8. Goodman, J.W., *Introduction to Fourier Optics*. McGraw-Hill Physical and Quantum Electronics Series. 1968, New York: McGraw-Hill. 287.
9. Heijmans, H.J.A.M., *Morphological Image Operators*. Advances in Electronics and Electron Physics. 1994, Boston: Academic Press.
10. Hunt, R.W.G., *The Reproduction of Colour in Photography, Printing & Television*,. Fourth ed. 1987, Tolworth, England: Fountain Press.
11. Freeman, H., *Boundary encoding and processing*, in *Picture Processing and Psychopictorics*, B.S. Lipkin and A. Rosenfeld, Editors. 1970, Academic Press: New York. p. 241-266.
12. Stockham, T.G., *Image Processing in the Context of a Visual Model*. Proc. IEEE, 1972. **60**: p. 828 - 842.
13. Murch, G.M., *Visual and Auditory Perception*. 1973, New York: Bobbs-Merrill Company, Inc. 403.
14. Frisby, J.P., *Seeing: Illusion, Brain and Mind*. 1980, Oxford, England: Oxford University Press. 160.
15. Blakemore, C. and F.W.C. Campbell, *On the existence of neurons in the human visual system selectively sensitive to the orientation and size of retinal images*. J. Physiology, 1969. **203**: p. 237-260.
16. Born, M. and E. Wolf, *Principles of Optics*. Sixth ed. 1980, Oxford: Pergamon Press.
17. Young, I.T., *Quantitative Microscopy*. IEEE Engineering in Medicine and Biology, 1996. **15**(1): p. 59-66.
18. Dorst, L. and A.W.M. Smeulders, *Length estimators compared*, in *Pattern Recognition in Practice II*, E.S. Gelsema and L.N. Kanal, Editors. 1986, Elsevier Science: Amsterdam. p. 73-80.
19. Young, I.T., *Sampling density and quantitative microscopy*. Analytical and Quantitative Cytology and Histology, 1988. **10**(4): p. 269-275.
20. Kulpa, Z., *Area and perimeter measurement of blobs in discrete binary pictures*. Computer Vision, Graphics and Image Processing, 1977. **6**: p. 434-454.
21. Vossepoel, A.M. and A.W.M. Smeulders, *Vector code probabilities and metrication error in the representation of straight lines of finite length*. Computer Graphics and Image Processing, 1982. **20**: p. 347-364.
22. Photometrics Ltd., *Signal Processing and Noise*, in *Series 200 CCD Cameras Manual*. 1990: Tucson, Arizona.

23. Huang, T.S., G.J. Yang, and G.Y. Tang, *A Fast Two-Dimensional Median Filtering Algorithm*. IEEE Transactions on Acoustics, Speech, and Signal Processing, 1979. **ASSP-27**: p. 13-18.
24. Groen, F.C.A., R.J. Ekkers, and R. De Vries, *Image processing with personal computers*. Signal Processing, 1988. **15**: p. 279-291.
25. Verbeek, P.W., H.A. Vrooman, and L.J. Van Vliet, *Low-Level Image Processing by Max-Min Filters*. Signal Processing, 1988. **15**: p. 249-258.
26. Young, I.T. and L.J. Van Vliet, *Recursive Implementation of the Gaussian Filter*. Signal Processing, 1995. **44**(2): p. 139-151.
27. Kuwahara, M., *et al.*, *Processing of RI-angiocardigraphic images*, in *Digital Processing of Biomedical Images*, K. Preston and M. Onoe, Editors. 1976, Plenum Press: New York. p. 187-203.
28. Van Vliet, L.J., *Grey-scale measurements in multi-dimensional digitized images*, PhD Thesis: Delft University of Technology, 1993.
29. Serra, J., *Image Analysis and Mathematical Morphology*. 1982, London: Academic Press.
30. Vincent, L., *Morphological transformations of binary images with arbitrary structuring elements*. Signal Processing, 1991. **22**(1): p. 3-23.
31. Van Vliet, L.J. and B.J.H. Verwer, *A Contour Processing Method for Fast Binary Neighbourhood Operations*. Pattern Recognition Letters, 1988. **7**(1): p. 27-36.
32. Young, I.T., *et al.*, *A new implementation for the binary and Minkowski operators*. Computer Graphics and Image Processing, 1981. **17**(3): p. 189-210.
33. Lantuéjoul, C., *Skeletonization in Quantitative Metallography*, in *Issues of Digital Image Processing*, R.M. Haralick and J.C. Simon, Editors. 1980, Sijthoff and Noordhoff: Groningen, The Netherlands.
34. Oppenheim, A.V., R.W. Schafer, and T.G. Stockham, Jr., *Non-Linear Filtering of Multiplied and Convolved Signals*. Proc. IEEE, 1968. **56**(8): p. 1264-1291.
35. Ridler, T.W. and S. Calvard, *Picture thresholding using an iterative selection method*. IEEE Trans. on Systems, Man, and Cybernetics, 1978. **SMC-8**(8): p. 630-632.
36. Zack, G.W., W.E. Rogers, and S.A. Latt, *Automatic Measurement of Sister Chromatid Exchange Frequency*. 1977. **25**(7): p. 741-753.

37. Chow, C.K. and T. Kaneko, *Automatic boundary detection of the left ventricle from cineangiograms*. Computers and Biomedical Research, 1972. **5**: p. 388-410.
38. Canny, J., *A Computational Approach to Edge Detection*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986. **PAMI-8**(6): p. 679-698.
39. Marr, D. and E.C. Hildreth, *Theory of edge detection*. Proc. R. Soc. London Ser. B., 1980. **207**: p. 187-217.
40. Verbeek, P.W. and L.J. Van Vliet, *On the Location Error of Curved Edges in Low-Pass Filtered 2D and 3D Images*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1994. **16**(7): p. 726-733.
41. Lee, J.S.L., R.M. Haralick, and L.S. Shapiro. *Morphologic Edge Detection*. in *8th International Conference on Pattern Recognition*. 1986. Paris: IEEE Computer Society.
42. Van Vliet, L.J., I.T. Young, and A.L.D. Beckers, *A Non-linear Laplace Operator as Edge Detector in Noisy Images*. Computer Vision, Graphics, and Image Processing, 1989. **45**: p. 167-195.
43. Meyer, F. and S. Beucher, *Morphological Segmentation*. J. Visual Comm. Image Rep., 1990. **1**(1): p. 21-46.
44. Meyer, F., *Iterative Image Transformations for an Automatic Screening of Cervical Cancer*. Journal of Histochemistry and Cytochemistry, 1979. **27**: p. 128-135.